

ARM926EJ-S[™]

Revision: r0p5

Implementation Guide

Confidential

The ARM logo, consisting of the letters "ARM" in a bold, sans-serif font, followed by a registered trademark symbol (®).

ARM926EJ-S

Implementation Guide

Copyright © 2002-2003, 2005 ARM Limited. All rights reserved.

Release Information

Change history		
Date	Issue	Change
15th October 2002	A	First release
7th February 2003	B	Second release
04 July 2003	C	Revised to conform with Reference Methodology RM4.1 for ARM926EJ-S r0p4
5 December 2003	D	Release for r0p5
30 June 2005	E	Tools list revised.

Proprietary Notice

Words and logos marked with ® or ™ are registered trademarks or trademarks of ARM Limited in the EU and other countries, except as otherwise stated below in this proprietary notice. Other brands and names mentioned herein may be the trademarks of their respective owners.

Neither the whole nor any part of the information contained in, or the product described in, this document may be adapted or reproduced in any material form except with the prior written permission of the copyright holder.

The product described in this document is subject to continuous developments and improvements. All particulars of the product and its use contained in this document are given by ARM in good faith. However, all warranties implied or expressed, including but not limited to implied warranties of merchantability, or fitness for purpose, are excluded.

This document is intended only to assist the reader in the use of the product. ARM Limited shall not be liable for any loss or damage arising from the use of any information in this document, or any error or omission in such information, or any incorrect use of the product.

Confidentiality Status

This document is Confidential. This document may only be used and distributed in accordance with the terms of the agreement entered into by ARM and the party that ARM delivered this document to.

Product Status

The information in this document is final, that is for a developed product.

Web Address

<http://www.arm.com>

Contents

ARM926EJ-S Implementation Guide

Preface

About this document	xii
Feedback	xvi

Chapter 1

Introduction

1.1 About the macrocell development platform, tools, and deliverables	1-2
---	-----

Chapter 2

RAM Integration

2.1 About RAM integration	2-2
2.2 Supported RAM Integration flow	2-3
2.3 RAM organization	2-4
2.4 Integrating your RAM blocks	2-22
2.5 Integrating RAM test logic	2-31

Chapter 3

RTL Validation

3.1 About RTL validation	3-2
3.2 Supported RTL validation flow	3-7
3.3 Simulator and platform support for RTL validation	3-8
3.4 Setting up your validation environment	3-9
3.5 Building the model	3-10
3.6 Running the RTL validation	3-12
3.7 Debugging a single validation test	3-18

3.8	Validation script usage	3-19
3.9	val_report script usage	3-22
3.10	Script file generation	3-25

Chapter 4

Floorplanning

4.1	About floorplanning	4-2
4.2	Supported flow	4-3
4.3	Guidelines	4-4
4.4	Floorplanning	4-5
4.5	Placement	4-7
4.6	Floorplan example	4-8

Chapter 5

Synthesis

5.1	About synthesis	5-2
5.2	Supported synthesis flow	5-4
5.3	Synthesis scripts	5-6
5.4	Setting up synthesis	5-9
5.5	Running synthesis	5-12
5.6	Synthesis process	5-15
5.7	Floorplanning and SoCTest	5-32
5.8	Reports	5-34

Chapter 6

CTS and Routing

6.1	About CTS and routing	6-2
6.2	Supported CTS and routing flow	6-3
6.3	CTS and routing scripts	6-4
6.4	Set up for Astro layout	6-7
6.5	CTS and clock skew	6-8
6.6	Routing	6-9
6.7	Post-routing	6-10
6.8	Static Timing Analysis	6-13

Chapter 7

STA and Timing Model Generation

7.1	About STA and timing model generation	7-2
7.2	Supported STA and timing model generation flow	7-3
7.3	STA and timing model generation scripts	7-4
7.4	Setting up STA and timing model generation	7-6
7.5	Running STA and timing model extraction	7-7
7.6	The STA and timing model generation process	7-8
7.7	Extracting the library cell timing model	7-13
7.8	Verifying the extracted timing model	7-14
7.9	Outputs	7-17

Chapter 8	Functional Verification of Netlist	
8.1	About netlist verification	8-2
8.2	Verification using equivalence checking tools	8-3
8.3	Netlist verification preparation	8-11
8.4	Functional verification in conjunction with the macrocell RTL model	8-12
8.5	Verification using test vectors	8-15
8.6	Replaying test vectors in a Verilog environment	8-18
Chapter 9	Production Test	
9.1	About production test	9-2
9.2	Test flow with non-Synopsys test tools	9-3
9.3	Macrocell test ports	9-4
9.4	Test wrapper insertion	9-5
9.5	Performing ATPG using TetraMAX	9-6
Chapter 10	Sign-Off	
10.1	RTL integration phase	10-2
10.2	Post synthesis, and post place and route phases	10-3
Appendix A	Synthesizable Register File Replacement	
A.1	About synthesizable register file replacement	A-2
A.2	Replacing the synthesizable register file	A-3
	Glossary	

List of Tables

ARM926EJ-S Implementation Guide

	Change history	ii
Table 1-1	Development tools and version	1-2
Table 1-2	Design flow process cross references	1-6
Table 2-1	RAM integration process cross references	2-3
Table 2-2	MMU RAM implementation options	2-6
Table 2-3	ICache data RAM implementation options	2-8
Table 2-4	Instruction cache TAG RAM implementation options	2-9
Table 2-5	ICache valid RAM implementation options	2-11
Table 2-6	DCache data RAM implementation options	2-13
Table 2-7	Data cache TAG RAM implementation options	2-15
Table 2-8	DCache valid RAM implementation options	2-16
Table 2-9	DCache dirty RAM implementation options	2-17
Table 2-10	CACHE_SIZE encodings	2-24
Table 2-11	ram-config file variable settings	2-26
Table 2-12	ram-config file variable replacement	2-27
Table 2-13	ARM926EJSCore RAM test ports and associated functionality	2-32
Table 2-14	DCBISTA port connections	2-34
Table 3-1	RTL validation test suites	3-4
Table 3-2	Validation configuration files	3-5
Table 3-3	Simulator support for platforms	3-8
Table 5-1	Synthesis technology parameters	5-9
Table 5-2	Options	5-10
Table 5-3	Summary of operation, option settings and design flow requirements	5-12

Table 5-4	Synthesis reports	5-34
Table 7-1	STA options	7-6
Table 7-2	STA reports	7-17
Table 8-1	ARM926EJS_fm.tcl variables	8-7
Table 8-2	Verplex equivalence checking files	8-8
Table 8-3	926-rtl-vs-tsmc_sicg.do	8-9
Table 9-1	Macrocell test ports	9-4
Table 9-2	Pins that are not wrapped by the test wrapper	9-5
Table A-1	a9ejsRam3r2wSDff interface description	A-3

List of Figures

ARM926EJ-S Implementation Guide

Figure 1-1	Design flow	1-5
Figure 1-2	Release directory structure	1-7
Figure 1-3	Macrocell top-level module block diagram	1-9
Figure 2-1	RAM integration design flow	2-3
Figure 2-2	MMU RAM organization	2-6
Figure 2-3	ICache data RAM organization	2-7
Figure 2-4	ICache TAG RAM organization	2-8
Figure 2-5	ICache valid RAM organization	2-11
Figure 2-6	DCache data RAM organization	2-12
Figure 2-7	DCache TAG RAM organization	2-14
Figure 2-8	DCache valid RAM organization	2-16
Figure 2-9	DCache dirty RAM organization	2-17
Figure 2-10	Producing byte-write memory	2-18
Figure 2-11	Optimizing for power	2-20
Figure 2-12	Optimizing for Speed	2-21
Figure 2-13	Test access using the DCache dirty RAM	2-31
Figure 3-1	Building and validating the macrocell	3-7
Figure 3-2	Validation report, normal status	3-16
Figure 3-3	Validation report, full status	3-17
Figure 4-1	Floorplanning flow	4-3
Figure 4-2	Floorplan example	4-8
Figure 5-1	Synthesis directory structure	5-2
Figure 5-2	Supported synthesis flow	5-5

Figure 5-3	Synthesis script association	5-6
Figure 6-1	CTS and routing flow	6-3
Figure 6-2	Example clock tree	6-8
Figure 7-1	Timing model extraction process	7-3
Figure 7-2	STA and timing model generation script association	7-4
Figure 8-1	Equivalence checking flow	8-4
Figure 8-2	Verilog tools directory	8-15
Figure 8-3	Vectors directory	8-15
Figure 8-4	Verilog testbench directory	8-16

Preface

This preface introduces the *ARM926EJ-S Implementation Guide*. It contains the following sections:

- *About this document* on page xii
- *Feedback* on page xvi.

About this document

This document is the implementation guide for the ARM926EJ-S macrocell.

Note

- In this guide the term macrocell refers to the ARM926EJ-S macrocell, unless explicit reference is made to other macrocells.
 - The term processor refers to the ARM926EJ-S processor unless explicit reference is made to other processors.
-

Product revision status

The *rnpn* identifier indicates the revision status of the product described in this document, where:

- | | |
|------------------|--|
| <i>rn</i> | Identifies the major revision of the product. |
| <i>pn</i> | Identifies the minor revision or modification status of the product. |

Intended audience

This document has been written for experienced hardware engineers who might or might not have experience of ARM products, but who have experience of writing Verilog and of performing synthesis, and who want to implement the macrocell in an *System-on-Chip* (SoC) design.

Using this manual

This document is organized into the following chapters:

Chapter 1 *Introduction*

Read this chapter for an overview of the macrocell implementation giving details of:

- the platform and tools used in the development of the macrocell
- the directory structure of the supplied release.

Chapter 2 *RAM Integration*

Read this chapter to learn how to integrate the MMU and cache RAM blocks at the ARM926EJS level, and how the RAM blocks interface to the ARM926EJSCore level in the macrocell.

Chapter 3 *RTL Validation*

Read this chapter to learn how to build the macrocell model for use with various vendor simulators, and how to validate the model using the validation suite.

Chapter 4 *Floorplanning*

Read this chapter to learn how to create a floorplan for the macrocell to use with synthesis and place and route tools.

Chapter 5 *Synthesis*

Read this chapter to learn how to synthesize the macrocell. This chapter also gives details of the synthesis files and script variables.

Chapter 6 *CTS and Routing*

Read this chapter for advice on:

- Clock tree synthesis and clock skew
- layout and routing.

Chapter 7 *STA and Timing Model Generation*

Read this chapter to learn how to:

- use *Static Timing Analysis* (STA) tools to analyze the timing performance of a design, both after synthesis, and after layout
- produce an accurate context-independent timing model for the macrocell for use throughout the SoC design flow.

Chapter 8 *Functional Verification of Netlist*

Read this chapter to learn on how to verify the functionality of the synthesized netlist using formal verification tools.

Chapter 9 *Production Test*

Read this chapter for information on production testing, including how to perform automatic scan insertion.

Chapter 10 *Sign-Off*

Read this chapter for details of the ARM verification criteria, and how to sign off your macrocell design.

Appendix A *Synthesizable Register File Replacement*

This appendix contains guidelines on how to replace the synthesizable register file.

Conventions

This section describes the conventions that this document uses:

- *Typographical*
- *Signal naming*
- *Numbering* on page xv.

Typographical

This document uses the following typographical conventions:

<i>italic</i>	Highlights important notes, introduces special terminology, denotes internal cross-references, and citations.
bold	Highlights interface elements, such as menu names. Denotes signal names. Also used for terms in descriptive lists, where appropriate.
<code>monospace</code>	Denotes text that you can enter at the keyboard, such as commands, file and program names, and source code.
<u><code>monospace</code></u>	Denotes a permitted abbreviation for a command or option. You can enter the underlined text instead of the full command or option name.
<i>monospace italic</i>	Denotes arguments to monospace text where the argument is to be replaced by a specific value.
<code>monospace bold</code>	Denotes language keywords when used outside example code.
< and >	Angle brackets enclose replaceable terms for assembler syntax where they appear in code or code fragments. They appear in normal font in running text. For example: • MRC p15, 0 <Rd>, <CRn>, <CRm>, <Opcode_2> • The Opcode_2 value selects which register is accessed.

Signal naming

The level of an asserted signal depends on whether the signal is active-HIGH or active-LOW. Asserted means HIGH for active-HIGH signals and LOW for active-LOW signals:

Prefix H	Denotes <i>Advanced High-performance Bus</i> (AHB) signals.
Prefix n	Denotes Active-LOW signals except in the case of AHB or <i>Advanced Peripheral Bus</i> APB reset signals. These are named HRESETn and PRESETn respectively.
Prefix P	Denotes an APB signal.

Numbering

`<size in bits>'<base><number>`

This is a Verilog method of abbreviating constant numbers. For example:

- `'h7B4` is an unsized hexadecimal value.
- `'o7654` is an unsized octal value.
- `8'd9` is an eight-bit wide decimal value of 9.
- `8'h3F` is an eight-bit wide hexadecimal value of 0x3F. This is equivalent to `b0011111`.
- `8'b1111` is an eight-bit wide binary value of `b00001111`.

Further reading

This section lists publications by ARM Limited, and by third parties.

ARM periodically provides updates and corrections to its documentation. See <http://www.arm.com> for current errata sheets and addenda, and ARM Frequently Asked Questions list.

ARM publications

This document contains information that is specific to the ARM926EJ-S macrocell. Refer to the following documents for other relevant information:

- *ARM926EJ-S Technical Reference Manual* (ARM DDI 0198)
- *ARM9EJ-S Technical Reference Manual* (ARM DDI 0222)
- *Jazelle v1 Technical Reference Manual* (ARM DDI 0225).

Feedback

ARM Limited welcomes feedback both on the ARM926EJ-S macrocell and its documentation.

Feedback on this product

If you have any comments or suggestions about this product, contact your supplier giving:

- the product name
- a concise explanation of your comments.

Feedback on this document

If you have any comments about this document, send email to errata@arm.com giving:

- the title
- the number
- the page number(s) to which your comments refer
- a concise explanation of your comments.

ARM Limited also welcomes general suggestions for additions and improvements.

Chapter 1

Introduction

This chapter gives an overview of the macrocell. It contains the following section:

- *About the macrocell development platform, tools, and deliverables* on page 1-2.

1.1 About the macrocell development platform, tools, and deliverables

This section provides details of the platform and tools used in the development of the macrocell, and gives the directory structure of the supplied release:

- *Development platform*
- *Development tools*
- *Macrocell design flow* on page 1-4
- *Directory structure* on page 1-6
- *Design hierarchy* on page 1-7.

1.1.1 Development platform

The macrocell has been developed and tested on Sun UNIX workstations using the Solaris 5.8 operating system. The supplied scripts are either C-shell scripts, or Perl scripts written for Perl version 5.

1.1.2 Development tools

The tools used to develop the macrocell are shown in Table 1-1.

Table 1-1 Development tools and version

Purpose	Vendor	Tool	Version
<i>Automatic Test Pattern Generation (ATPG)</i>	Synopsys	TetraMAX	2003.12
STA	Synopsys	PrimeTime	2003.12
Synthesis	Synopsys	Design Compiler, Test Compiler, Physical Compiler	2003.12
Test wrapper insertion	Synopsys	SoCTest	2003.12
Timing Model Extraction	Synopsys	PrimeTime	2003.12
Equivalence checking	Cadence	Conformal LEC	4.2.0.a
	Synopsys	Formality	2003.12

Table 1-1 Development tools and version (continued)

Purpose	Vendor	Tool	Version
Validation	ModelTech	vsim	5.7e
	Cadence	XL	LDV5.0
	Synopsys	VCS	7.1.1
	Cadence	NC	LDV5.0
	ARM	<i>ARM Developer Suite (ADS):</i> armasm, armlink, fromelf	1.1
	Open source	Perl	5.005-02
	ARM	Modelgen	4.3.2
Place and route	Synopsys	Astro	2001.2.3.5.0.3
Parasitic extraction	Synopsys	StarExtract	2002.2.1.b02

Note

The macrocell has been verified using the tools defined in Table 1-1 on page 1-2. You are recommended to use this, or a later version of the tools.

If your installation of perl is nonstandard you must modify the headers in the following scripts:

- verilog/MAIN_ARM926EJS/automem
- verilog/MAIN_ARM926EJS/intest_schain_mask.perl
- verilog/modelgen/build
- verilog/arm926ejs/tools/bin/Crf2Vrf
- verilog/arm926ejs/tools/bin/verilog_testgen
- verilog/arm926ejs/tools/bin/write_clockgate_sdf
- verilog/arm926ejs/tools/bin/gen_connecttb
- verilog/arm926ejs/tools/lib/perl5/ParseHDL/GenXCon.pm
- verilog/arm926ejs/tools/lib/perl5/ParseHDL/ParseEntity.pm
- verilog/arm926ejs/tools/lib/perl5/ParseHDL/ParseHDL.pm
- verilog/arm926ejs/tools/lib/perl5/ParseHDL/ParseVHDL.pm
- verilog/arm926ejs/tools/lib/perl5/ParseHDL/ParseVeriEnt.pm
- verilog/arm926ejs/tools/lib/perl5/ParseHDL/ParseVerilog.pm
- synopsys/build_dir/scripts/extract_essence.pl
- synopsys/build_dir/scripts/critical_paths

1.1.3 Macrocell design flow

The steps required to implement the macrocell are shown in the design flow in Figure 1-1 on page 1-5.

First create a top-level module into which you integrate your library RAM (see Chapter 2 *RAM Integration*). You can then proceed with validating your RTL (see Chapter 3 *RTL Validation*), producing a floorplan (see Chapter 4 *Floorplanning*), and synthesizing your design (see Chapter 5 *Synthesis*). You can do validation and synthesis in parallel because these two tasks are time-intensive.

Note

If validation fails you might have to start the synthesis process again after recreating the top-level module (see Chapter 2 *RAM Integration*).

After validating and synthesizing the macrocell RTL, you produce a hardened macrocell (see Chapter 6 *CTS and Routing*).

After macrocell hardening you characterize (see Chapter 7 *STA and Timing Model Generation*) and verify (Chapter 8 *Functional Verification of Netlist*) the macrocell. To finish the implementation flow you must sign off the macrocell (see Chapter 10 *Sign-Off*).

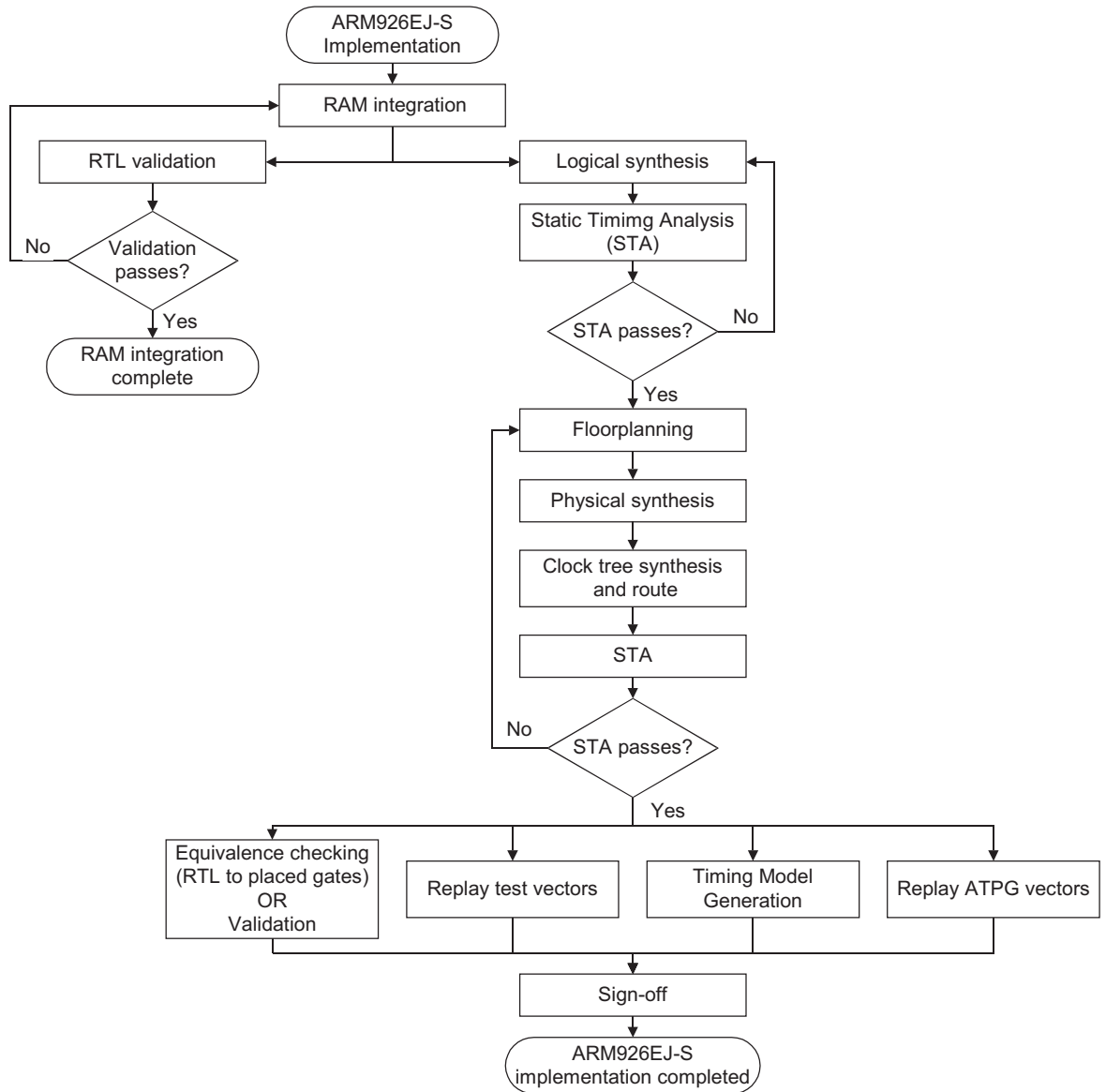


Figure 1-1 Design flow

Table 1-2 shows the chapters in this guide that relate to the design flow processes.

Table 1-2 Design flow process cross references

Process	Chapter
RAM integration	see Chapter 2 <i>RAM Integration</i>
RTL validation	see Chapter 3 <i>RTL Validation</i>
Logical synthesis	see Chapter 5 <i>Synthesis</i>
STA	see Chapter 7 <i>STA and Timing Model Generation</i>
Floorplanning	see Chapter 4 <i>Floorplanning</i>
Physical synthesis	see Chapter 5 <i>Synthesis</i>
Clock tree synthesis and route	see Chapter 6 <i>CTS and Routing</i>
Equivalence checking (RTL to placed gates)	see Chapter 8 <i>Functional Verification of Netlist</i>
Netlist validation	see Chapter 8 <i>Functional Verification of Netlist</i>
Replay test vectors	see Chapter 8 <i>Functional Verification of Netlist</i>
Timing model generation	see Chapter 7 <i>STA and Timing Model Generation</i>
Replay ATPG vectors	see Chapter 9 <i>Production Test</i>
Sign-off	see Chapter 10 <i>Sign-Off</i>

1.1.4 Directory structure

Figure 1-2 on page 1-7 shows the principal directory structure of the macrocell design, as it appears when you have unpacked the deliverables. The `./synopsys/build_dir` contains directories required to implement the macrocell design. If want to explore multiple implementations of the macrocell design in parallel, you must:

1. copy and rename the `build_dir` directory
2. place the renamed directory in `./synopsys`.

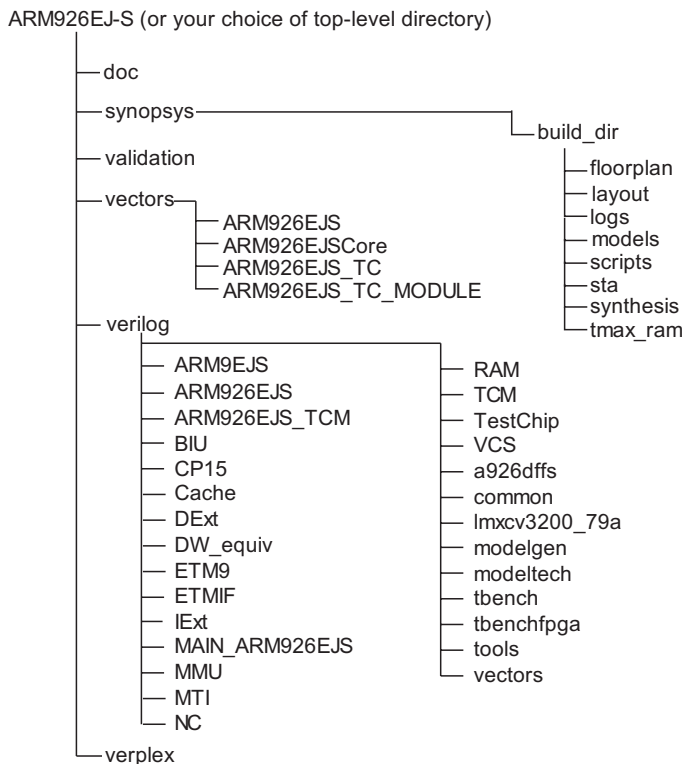


Figure 1-2 Release directory structure

1.1.5 Design hierarchy

Figure 1-3 on page 1-9 shows the macrocell design hierarchy.

The ARM926EJSCore is the RAM-independent level that is common within all ARM926EJS module variants. The main modules instantiated at ARM926EJSCore level are:

ARM9EJS	This is the processor core.
a926ejsBIU	This is the AHB <i>Bus Interface Unit</i> (BIU).
a926ejsClkBlk	This contains the clock control logic.
a926ejsCP15	This contains the system control processor and external coprocessor interface.
a926ejsDCache	This is the data cache controller.

a926ejsDExt	This is the control block for external data accesses.
a926ejsDRoute	This is the data routing block between the ARM926EJSCore modules and the ARM9EJS module.
a926ejsETMIF	This is the <i>Embedded Trace Macrocell</i> (ETM) interface block.
a926ejsICache	This is the instruction cache controller.
a926ejsIExt	This is the control block for external instruction accesses.
a926ejsIRoute	This is the instruction routing block between the ARM926EJSCore modules and the ARM9EJS module.
a926ejsMMU	This is the <i>Memory Management Unit</i> (MMU).
a926ejsSysCtl	This is the system controller logic.
a926ejsTCM	This is the <i>Tightly-Coupled Memory</i> (TCM) controller.

The ARM926EJS level instantiates the ARM926EJSCore and the RAM blocks. The ARM926EJS.v file, supplied with the release, contains example behavioral RAM blocks.

Note

You must create a copy of the ARM926EJS.v file for each RAM size variant that you require and integrate your own library RAM blocks into this module. You also have to integrate RAM test logic at the ARM926EJS level.

Two *Design Simulation Models* (DSMs) are supplied:

- ARM926EJS-level DSM.
- ARM926EJSCore-level DSM. This is also supplied because the ARM926EJS level DSM does not contain your specific RAM test logic and does not match the pinout of your ARM926EJS level if you have added RAM test pins.

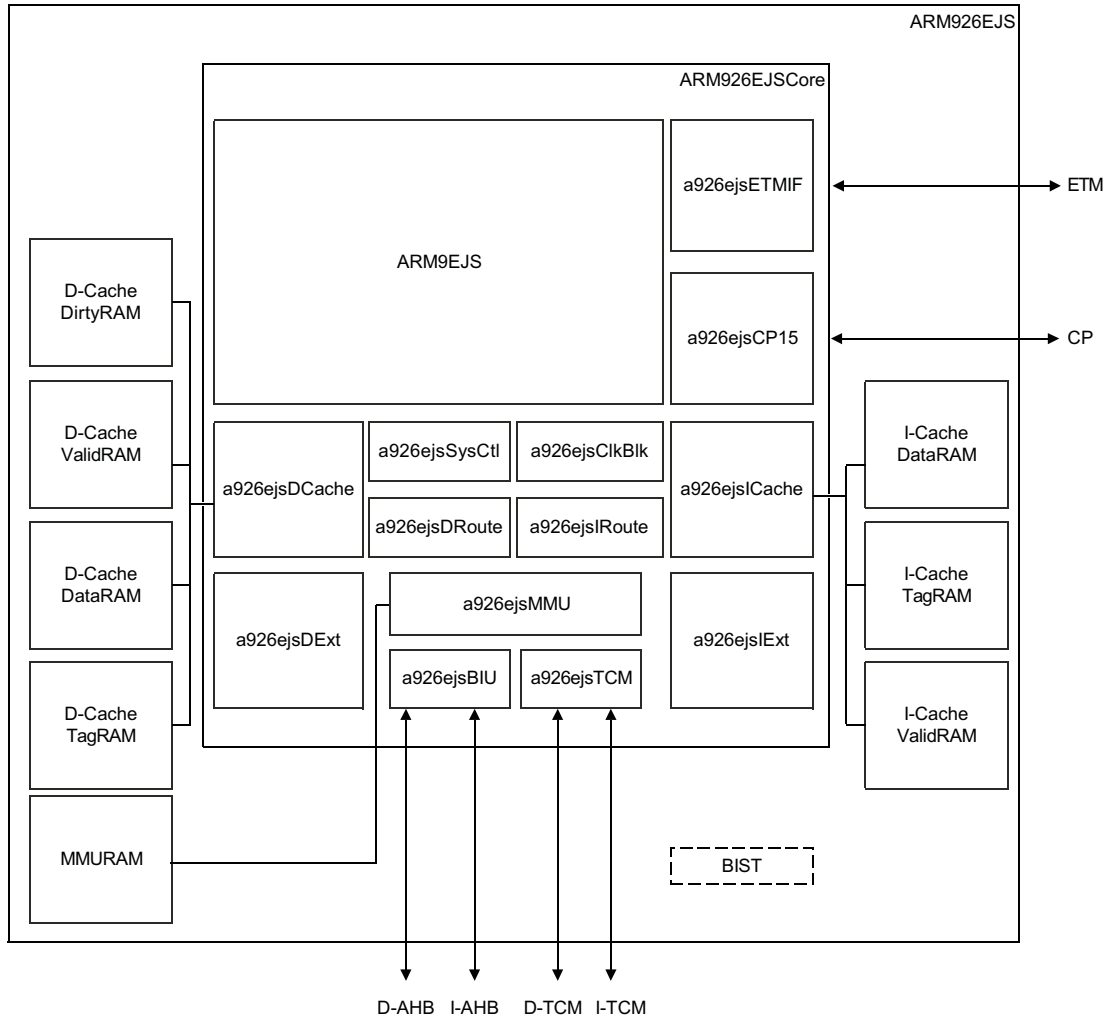


Figure 1-3 Macrocell top-level module block diagram

1.1.6 Silicon revision information

This manual is for revision r0p5 of the ARM926EJ-S macrocell, see *Product revision status* on page xii for details of revision numbering. The implementation methodology has been updated in line with *Reference Methodology version 4.1* (RM4.1) from Synopsys, including the addition of back-end scripts for *Clock Tree Synthesis* (CTS) and routing.

Chapter 2

RAM Integration

This chapter describes the macrocell RAM organization, and how to integrate your RAM blocks into the macrocell. It contains the following sections:

- *About RAM integration* on page 2-2
- *Supported RAM Integration flow* on page 2-3
- *RAM organization* on page 2-4
- *Integrating your RAM blocks* on page 2-22
- *Integrating RAM test logic* on page 2-31.

2.1 About RAM integration

The RAM blocks used in the macrocell are all instantiated at the ARM926EJS level (see Figure 1-3 on page 1-9). Having all the RAM blocks at a single level of hierarchy eases the RAM integration task and enables you to produce macrocell cache size variants by only having to edit a single module.

The macrocell is provided with example behavioral RAM models. You must create your own version of the ARM926EJS level with your own library RAM models instantiated.

This chapter describes the required RAM organization and the RAM implementation options available (see *RAM organization* on page 2-4), and how to integrate your RAM (see *Integrating your RAM blocks* on page 2-22).

Note

The *Tightly-Coupled Memories* (TCMs) are instantiated external to the macrocell and are not discussed in this chapter.

2.1.1 Requirements

Your library RAM must satisfy the following requirements:

- Synchronous, so that all timings are relative to the rising clock edge.
- Single cycle access.
- Chip select (RAM enable) control required.
- RAM outputs must always be valid (do not tristate).
- Byte-write (or bit-write) control is required for some RAM blocks. If byte-write (or bit-write) RAM blocks are not available you have to use blocks of byte-wide RAM, which increases the overall RAM area.
- RAM setup times must be less than 10-15% and access times must be less than 40-50% of the target cycle time. Violation of these requirements results in a slower design. You can balance setup and access times by skewing the RAM clock.

2.2 Supported RAM Integration flow

Figure 2-1 shows the flow that you must use when integrating RAM blocks into the macrocell.

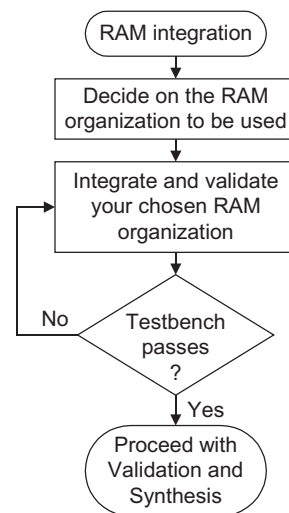


Figure 2-1 RAM integration design flow

Table 2-1 shows the sections and chapters in this guide that relate to the RAM integration flow.

Table 2-1 RAM integration process cross references

Process	Chapter
Decide on RAM organization	see <i>RAM organization</i> on page 2-4
Integrate and validate RAM	see <i>Integrating your RAM blocks</i> on page 2-22
Validation	see Chapter 3 <i>RTL Validation</i>
Synthesis	see Chapter 5 <i>Synthesis</i>

2.3 RAM organization

The RAM organization at the ARM926EJS level for the MMU and cache RAM blocks, and how these RAM blocks interface to the ARM926EJSCore level, are described in the following sections:

- *MMU RAM* on page 2-6
- *Instruction cache data RAM* on page 2-7
- *Instruction cache TAG RAM* on page 2-8
- *Instruction cache valid RAM* on page 2-11
- *Data cache data RAM* on page 2-12
- *Data cache TAG RAM* on page 2-14
- *Data cache valid RAM* on page 2-16
- *Data cache dirty RAM* on page 2-17.

You must ensure your implementation follows the information in these sections. Where an alternative implementation is suggested, you can choose the implementation that best suits your requirements.

In the ideal case, you can produce a single block of compiled RAM for each block of RAM as shown in Figure 2-2 on page 2-6 to Figure 2-9 on page 2-17. This might not always be possible if:

- Your RAM does not have the required byte-write control. In this case you must construct the RAM out of four blocks of byte-wide RAM. See *Producing byte-write memory from word-write RAM* on page 2-18.
- Your compiler cannot produce a single RAM block that is the required size, or a single RAM block might not meet the timing requirements. In this case, you must produce the RAM out of two or more blocks of smaller RAM. See *Producing a large memory from smaller RAM blocks* on page 2-19.

You can implement some RAM blocks (MMU RAM and TAG RAM) as a single wide RAM instead of four narrower RAM blocks. You must consider these trade-offs:

- The overall RAM area of the single wide RAM is lower. However, the aspect ratio of the wide RAM might be difficult to integrate into your floorplan and might be detrimental to the overall macrocell area (especially for larger cache sizes).
- Depending on the RAM architecture, a single wide RAM might have a lower power consumption than four narrower RAM blocks. You must compare the power consumption of the RAM to confirm this. Power saving is the only real advantage of using a single wide RAM over four narrower RAM blocks.
- The access time of the single wide RAM might be slower than the access time of the narrower RAM blocks.

Note

Figure 2-3 on page 2-7 to Figure 2-9 on page 2-17 show the address connections required to support the maximum cache size. For smaller caches some address bits are redundant, and do not connect to the RAM blocks. The address connections required for each cache size are described in the appropriate sections of this chapter.

You must only connect the specified address bits for your required cache size to your RAM blocks. If a particular RAM instantiation is larger than required, for example if your RAM generator cannot produce a small enough RAM, you must tie the upper bit(s) of the RAM address port on your RAM to 1'b0.

2.3.1 MMU RAM

Figure 2-2 shows the MMU RAM organization at the ARM926EJS level.

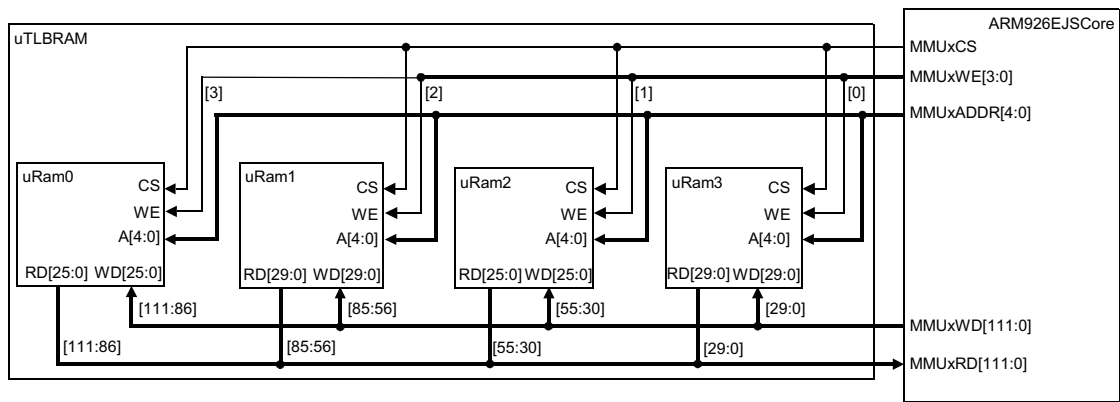


Figure 2-2 MMU RAM organization

Table 2-2 shows the implementation options for the MMU RAM.

Table 2-2 MMU RAM implementation options

RAM organization (#RAM blocks x words x bits)	Notes
1 x 32 x 112	Bit-write RAM ^a : MMUxWE[3] connects to WE[111:86] MMUxWE[2] connects to WE[85:56] MMUxWE[1] connects to WE[55:30] MMUxWE[0] connects to WE[29:0] .
1 x 32 x 128	Byte-write RAM. Redundant write data bits must be tied off. Redundant read data bits must be ignored.
2 x 32 x 30 (uRam1, uRam3) 2 x 32 x 26 (uRam0, uRam2)	Word-write RAM, as shown in Figure 2-2.

a. Implementing the MMU RAM out of four separate RAM blocks, as shown in Figure 2-2, is preferable to implementing the MMU RAM out of a single RAM block.

2.3.2 Instruction cache data RAM

Figure 2-3 shows the *Instruction Cache (ICache)* data RAM organization at the ARM926EJS level.

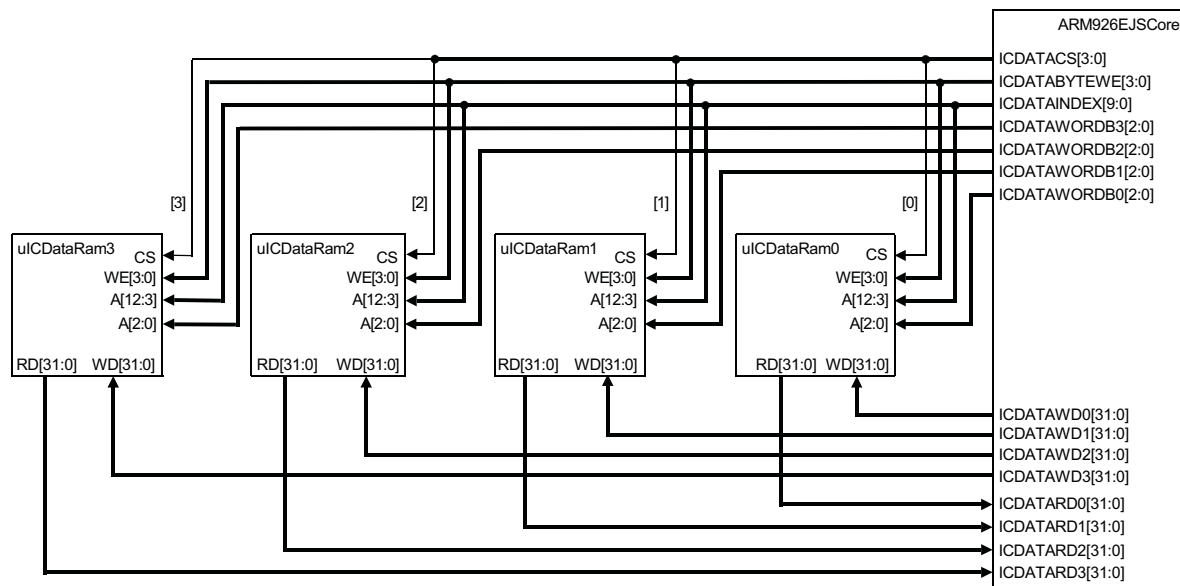


Figure 2-3 ICache data RAM organization

Table 2-3 on page 2-8 shows the ICache data RAM implementation options. All reads and writes to the ICache data RAM are word-wide and so you can implement this RAM using either word or byte-wide RAM. Byte-write control signals are only driven to this RAM to be compatible with the DCache data RAM, which requires byte-write control. If you implement the ICache data RAM using word-wide RAM, you have to create a single write-enable for the RAM by ORing the four byte-write bits:

```
assign WE = | ({ICDATABYTEWE[3],ICDATABYTEWE[2],ICDATABYTEWE[1],ICDATABYTEWE[0]});
```

Table 2-3 also shows the bits of **ICDATAINDEX[9:0]** that connect to the RAM address port for each cache size. The three LSBs of the RAM address connect to the relevant **ICDATAWORDB** bus as shown in Figure 2-3 on page 2-7.

Table 2-3 ICache data RAM implementation options

Cache size	RAM organization (#RAM blocks x words x bits)	MSB address connection	Notes
4KB	4 x 256 x 32	ICDATAINDEX[4:0]	Word or byte-write RAM
8KB	4 x 512 x 32	ICDATAINDEX[5:0]	Word or byte-write RAM
16KB	4 x 1024 x 32	ICDATAINDEX[6:0]	Word or byte-write RAM
32KB	4 x 2048 x 32	ICDATAINDEX[7:0]	Word or byte-write RAM
64KB	4 x 4096 x 32	ICDATAINDEX[8:0]	Word or byte-write RAM
128KB	4 x 8192 x 32	ICDATAINDEX[9:0]	Word or byte-write RAM

2.3.3 Instruction cache TAG RAM

Figure 2-4 shows the ICache TAG RAM organization at the ARM926EJS level.

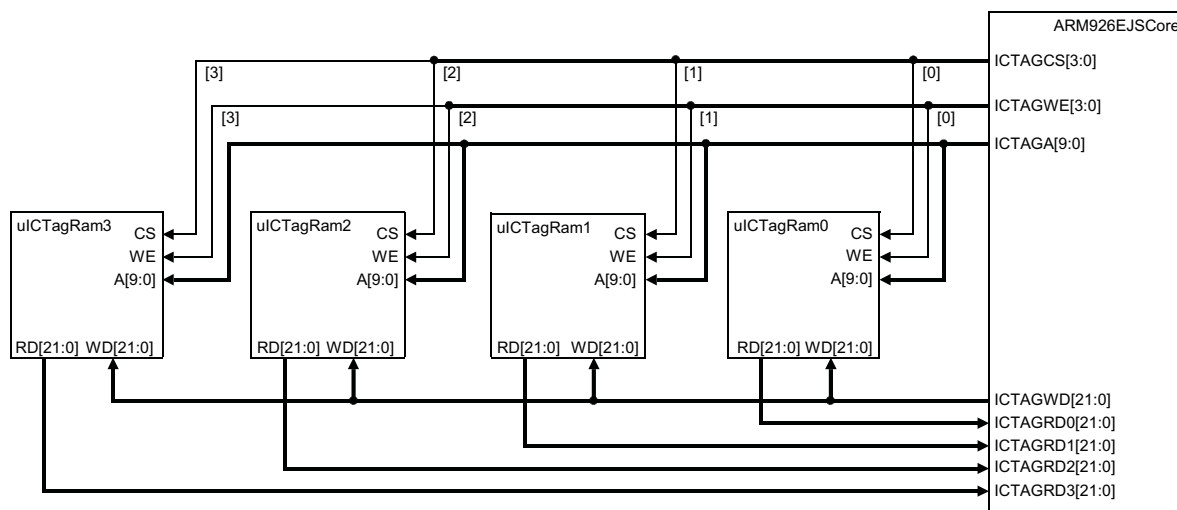


Figure 2-4 ICache TAG RAM organization

Table 2-4 shows the ICache TAG RAM implementation options. You can implement the TAG RAM as four banks of 22-bit wide RAM as shown in Figure 2-4 on page 2-8, or as a single 88-bit wide RAM if your RAM has bit-write control. Implementing the RAM as four banks of 22-bit wide RAM should be the default option.

The third column for the 4KB cache shows how the **WE** and **CS** signals must be connected if you are implementing the TAG RAM with a single RAM. These connections are the same for all cache sizes. The table also shows the bits of **ICTAGA[9:0]** that connect to the RAM address for each cache size.

———— **Note** —————

ICTAGA[10] is redundant and is not connected to your RAM blocks.

Table 2-4 Instruction cache TAG RAM implementation options

Cache size	RAM organization (#RAM blocks x words x bits)	Notes	Address connection
4KB	1 x 32 x 88	Bit-write RAM WE[87:66] = ICTAGWE[3] WE[65:44] = ICTAGWE[2] WE[43:22] = ICTAGWE[1] WE[21:0] = ICTAGWE[0] CS = 1 (ICTAGCS[3:0])	ICTAGA[4:0]
	4 x 32 x 22	Word-write RAM. Connections as shown in Figure 2-4 on page 2-8.	
8KB	1 x 64 x 88	Bit-write RAM	ICTAGA[5:0]
	4 x 64 x 22	Word-write RAM	
16KB	1 x 128 x 88	Bit-write RAM	ICTAGA[6:0]
	4 x 128 x 22	Word-write RAM	
32KB	1 x 256 x 88	Bit-write RAM	ICTAGA[7:0]
	4 x 256 x 22	Word-write RAM	

Table 2-4 Instruction cache TAG RAM implementation options (continued)

Cache size	RAM organization (#RAM blocks x words x bits)	Notes	Address connection
64KB	1 x 512 x 88	Bit-write RAM	ICTAGA[8:0]
	4 x 512 x 22	Word-write RAM	
128KB	1 x 1024 x 88	Bit-write RAM	ICTAGA[9:0]
	4 x 1024 x 22	Word-write RAM	

2.3.4 Instruction cache valid RAM

Figure 2-5 shows the ICache valid RAM organization at the ARM926EJS level.

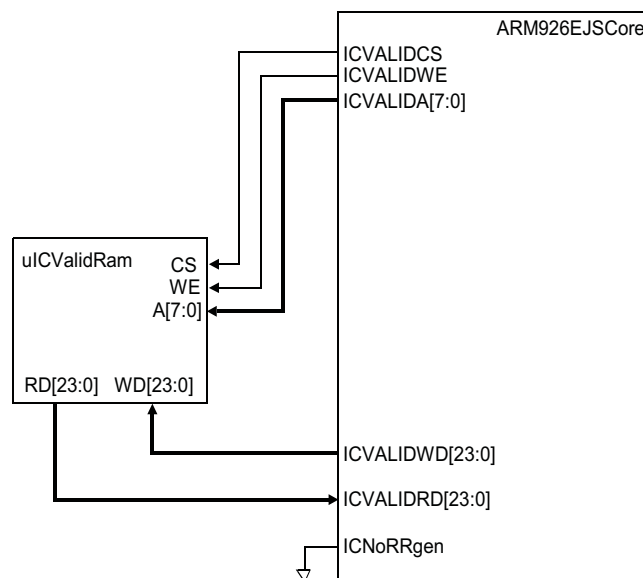


Figure 2-5 ICache valid RAM organization

Table 2-5 shows the implementation options for the ICache valid RAM. Only word-write control is required for the valid RAM.

Note

Ensure that **ICNoRRgen** is connected to 1'b0.

Table 2-5 ICache valid RAM implementation options

Cache size	RAM organization (# RAM blocks x words x bits)	Address connection	Notes
4KB	1 x 8 x 24	ICVALIDA[2:0]	Word-write RAM
8KB	1 x 16 x 24	ICVALIDA[3:0]	Word-write RAM
16KB	1 x 32 x 24	ICVALIDA[4:0]	Word-write RAM

Table 2-5 ICache valid RAM implementation options (continued)

Cache size	RAM organization (# RAM blocks x words x bits)	Address connection	Notes
32KB	1 x 64 x 24	ICVALIDA[5:0]	Word-write RAM
64KB	1 x 128 x 24	ICVALIDA[6:0]	Word-write RAM
128KB	1 x 256 x 24	ICVALIDA[7:0]	Word-write RAM

2.3.5 Data cache data RAM

Figure 2-6 shows the *Data Cache* (DCache) data RAM organization at the ARM926EJS level.

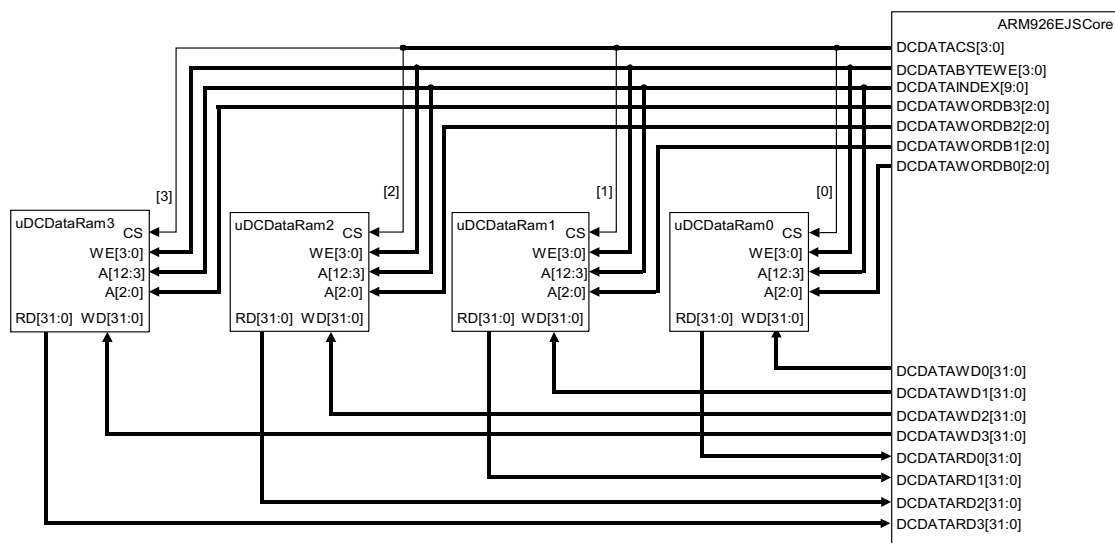


Figure 2-6 DCache data RAM organization

Table 2-6 shows the DCache data RAM implementation options. Byte-write control is required for the data RAM. If byte (or bit) writeable RAM is not available, then each RAM block must be constructed out of four byte-wide RAM blocks. The table also shows the bits of **DCDATAINDEX[9:0]** that connect to the RAM address for each cache size. The three LSBs of the RAM address connect to the relevant **DCDATAWORDB** bus as shown in Figure 2-6.

Table 2-6 DCache data RAM implementation options

Cache size	RAM organization (#RAM blocks x words x bits)	MSB address connection	Notes
4KB	4 x 256 x 32	DCDATAINDEX[4:0]	Byte-write RAM
8KB	4 x 512 x 32	DCDATAINDEX[5:0]	Byte-write RAM
16KB	4 x 1024 x 32	DCDATAINDEX[6:0]	Byte-write RAM
32KB	4 x 2048 x 32	DCDATAINDEX[7:0]	Byte-write RAM
64KB	4 x 4096 x 32	DCDATAINDEX[8:0]	Byte-write RAM
128KB	4 x 8192 x 32	DCDATAINDEX[9:0]	Byte-write RAM

2.3.6 Data cache TAG RAM

Figure 2-7 shows the DCache TAG RAM organization at the ARM926EJS level.

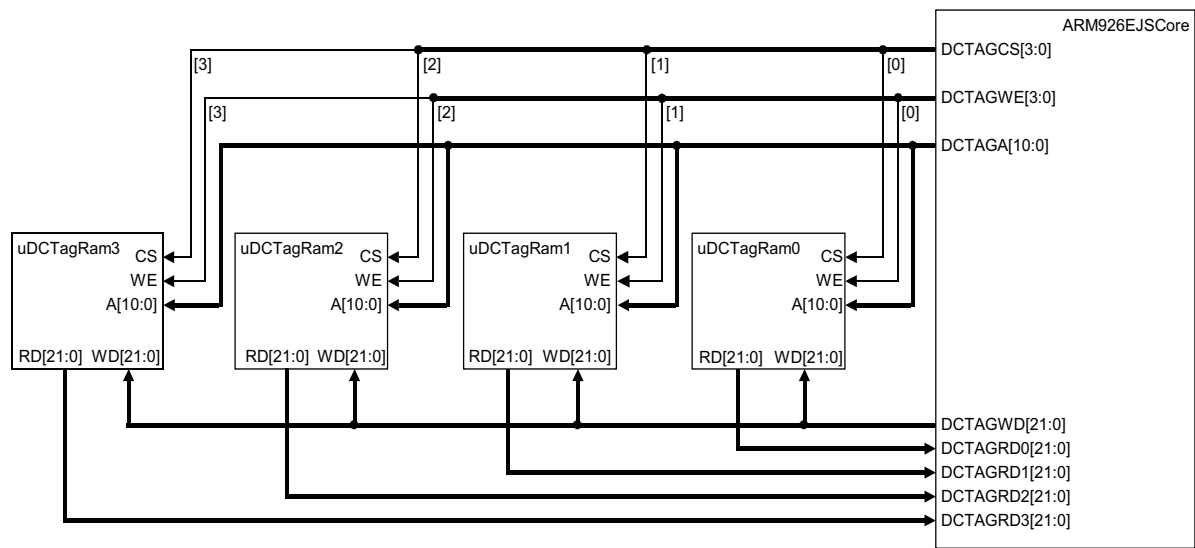


Figure 2-7 DCache TAG RAM organization

Table 2-7 on page 2-15 shows the DCache TAG RAM implementation options. You can implement the TAG RAM as four banks of 22-bit wide RAM (as shown in Figure 2-7) or as a single 88-bit wide RAM if your RAM has bit-write control. Implementing the RAM as four banks of 22-bit wide RAM should be the default option.

It is recommended that you implement the RAM as four banks of 22-bit wide RAM.

The third column in the table for the 4KB cache size entry shows how the **WE** and **CS** signals must be connected if you implement the TAG RAM with a single RAM. These connections are the same for all cache sizes. The table also shows the bits of **DCTAGA[9:0]** that connect to the RAM address port for each cache size.

Table 2-7 Data cache TAG RAM implementation options

Cache size	Ram organization (#RAM blocks x words x bits)	Notes	Address connection
4KB	1 x 64 x 88	Bit-write RAM: WE[87:66] = DCTAGWE[3] WE[65:44] = DCTAGWE[2] WE[43:22] = DCTAGWE[1] WE[21:0] = DCTAGWE[0] CS = 1 (DCTAGCS[3:0])	{ DCTAGA[10] , DCTAGA[4:0] }
	4 x 64 x 22	Word-write RAM. Connections as shown in Figure 2-7 on page 2-14.	
8KB	1 x 128 x 88	Bit-write RAM	{ DCTAGA[10] , DCTAGA[5:0] }
	4 x 128 x 22	Word-write RAM	
16KB	1 x 256 x 88	Bit-write RAM	{ DCTAGA[10] , DCTAGA[6:0] }
	4 x 256 x 22	Word-write RAM	
32KB	1 x 512 x 88	Bit-write RAM	{ DCTAGA[10] , DCTAGA[7:0] }
	4 x 512 x 22	Word-write RAM	
64KB	1 x 1024 x 88	Bit-write RAM	{ DCTAGA[10] , DCTAGA[8:0] }
	4 x 1024 x 22	Word-write RAM	
128KB	1 x 2048 x 88	Bit-write RAM	{ DCTAGA[10] , DCTAGA[9:0] }
	4 x 2048 x 22	Word-write RAM	

Note

If you are using the macrocell BIST ports to access the DCache TAG RAM, you must ensure that the top bit of the BIST address is mapped to bit10 of **DCTAGA**. See *BIST to data cache TAG RAM special considerations* on page 2-33 for more information.

2.3.7 Data cache valid RAM

Figure 2-8 shows the DCache valid RAM organization at the ARM926EJS level.

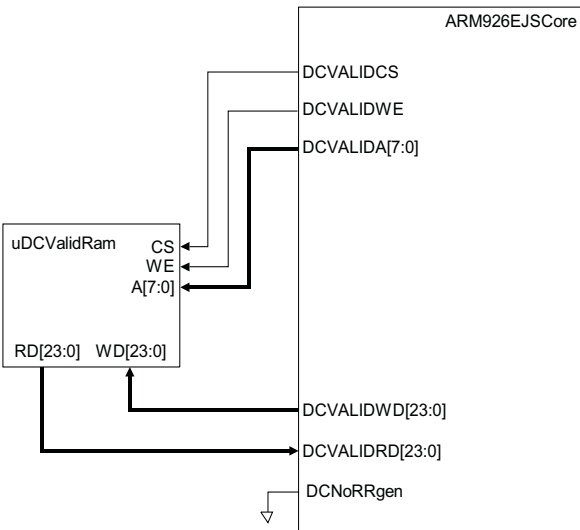


Figure 2-8 DCache valid RAM organization

Note

 Ensure that **DCNoRRgen** is connected to 1'b0.

Table 2-8 shows the implementation options for the DCache valid RAM. Only word-write control is required for the valid RAM.

Table 2-8 DCache valid RAM implementation options

Cache size	Ram organization (# RAM blocks x words x bits)	Address connection	Notes
4KB	1 x 8 x 24	DCVALIDA[2:0]	Word-write RAM
8KB	1 x 16 x 24	DCVALIDA[3:0]	Word-write RAM
16KB	1 x 32 x 24	DCVALIDA[4:0]	Word-write RAM
32KB	1 x 64 x 24	DCVALIDA[5:0]	Word-write RAM
64KB	1 x 128 x 24	DCVALIDA[6:0]	Word-write RAM
128KB	1 x 256 x 24	DCVALIDA[7:0]	Word-write RAM

2.3.8 Data cache dirty RAM

Figure 2-9 shows the DCache dirty RAM organization at the ARM926EJS level.

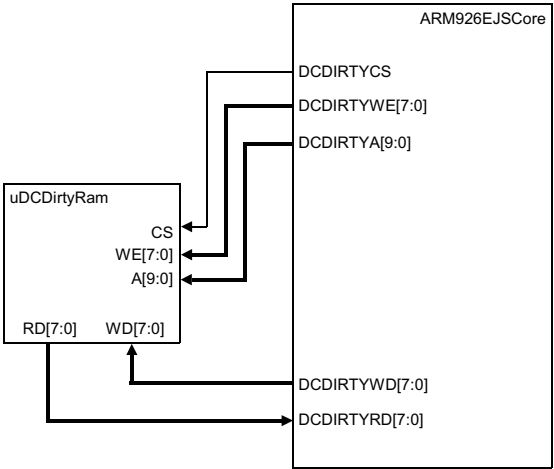


Figure 2-9 DCache dirty RAM organization

Table 2-9 shows the DCache dirty RAM implementation options. The dirty RAM requires bit-write control. If bit-write RAM is not available, the RAM must be implemented with flip-flops. The table shows the bits of **DCDIRTYA[9:0]** that connect to the RAM for each cache size.

Table 2-9 DCache dirty RAM implementation options

Cache size	Ram organization (#RAM blocks x words x bits)	Address connection	Notes
4KB	1 x 32 x 8	DCDIRTYA[4:0]	Bit-write RAM
8KB	1 x 64 x 8	DCDIRTYA[5:0]	Bit-write RAM
16KB	1 x 128 x 8	DCDIRTYA[6:0]	Bit-write RAM
32KB	1 x 256 x 8	DCDIRTYA[7:0]	Bit-write RAM
64KB	1 x 512 x 8	DCDIRTYA[8:0]	Bit-write RAM
128KB	1 x 1024 x 8	DCDIRTYA[9:0]	Bit-write RAM

2.3.9 Producing byte-write memory from word-write RAM

The DCache data memory requires byte-write RAM. If you do not have memories with byte-write control, you must construct the data memories out of four byte-wide RAM blocks. Figure 2-10 shows how to do this, using uDCDataRam0 of the DCache data RAM (see Figure 2-6 on page 2-12) as an example. Each RAM block now represents one byte of uDCDataRam0. The approach is exactly the same for any RAM that has to be split into bytes.

The rules for connecting the four RAM blocks are:

- Each byte-wide RAM has the same address and chip select control as the word-wide RAM.
- The following connections are made:
 - **WE[0], WD[7:0], RD[7:0]** connect to RAM byte0
 - **WE[1], WD[15:8], RD[15:8]** connect to RAM byte1
 - **WE[2], WD[23:16], RD[23:16]** connect to RAM byte2
 - **WE[3], WD[31:24], RD[31:24]** connect to RAM byte3.

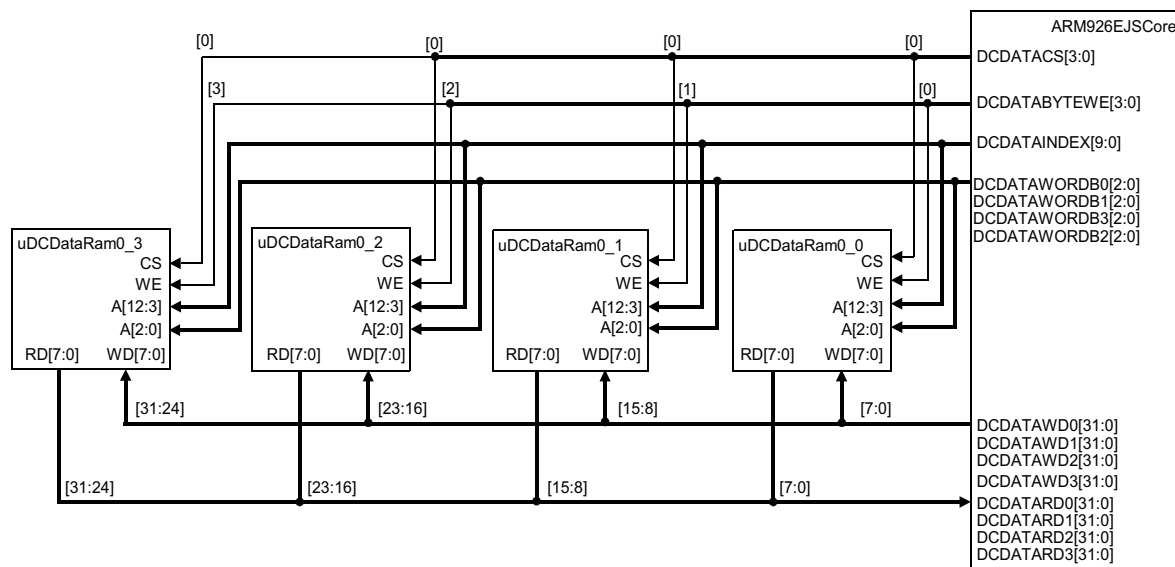


Figure 2-10 Producing byte-write memory

2.3.10 Producing a large memory from smaller RAM blocks

You might have to create a large memory out of smaller RAM blocks, for one of the following reasons:

- your RAM compiler cannot produce a RAM of the required size
- a single large RAM is too slow for your performance requirements
- a single large RAM does not fit into your floorplan.

Two ways of producing a memory from smaller RAM blocks are described using uDCDataRam0 of the DCache data RAM (see Figure 2-6 on page 2-12) as an example:

- if minimizing power consumption is more important than a fast design, follow the example described in *Optimizing for power*
- if a faster design is more important than minimizing power consumption, follow the example described in *Optimizing for speed* on page 2-20.

The rules for producing a memory out of smaller RAM blocks are:

- there must be an even number of RAM blocks ($b = 2, 4, 8$, for example).
- if the address width of the required memory size is n bits, the address port of the smaller RAM blocks is $m = \lceil n - (\log b / \log 2) \rceil$ bits wide
- address bits $[m-1:0]$ are applied to all the RAM blocks.
- address bits $[n-1:m]$ are gated with chip select signals for a power-optimized solution, or with write enable signals for a speed-optimized solution.
- pipelined address bits $[n-1:m]$ are used to select the correct RAM read data.

The approach is exactly the same for any memory that has to be constructed from smaller RAM blocks.

Optimizing for power

Figure 2-11 on page 2-20 shows how to produce uDCDataRam0 from two smaller RAM blocks, (uDCDataRam0_0 and uDCDataRam0_1, if you are optimizing for power. A separate chip select is required for each RAM block:

```
CS_0 = ~DCDATAINDEX[9] & DCDATACS[0];
CS_1 = DCDATAINDEX[9] & DCDATACS[0];
```

where:

- CS_0 is chip select for uDCDataRam0_0
- CS_1 is chip select for uDCDataRam0_1.

This ensures that only the RAM being accessed is enabled, minimizing power consumption.

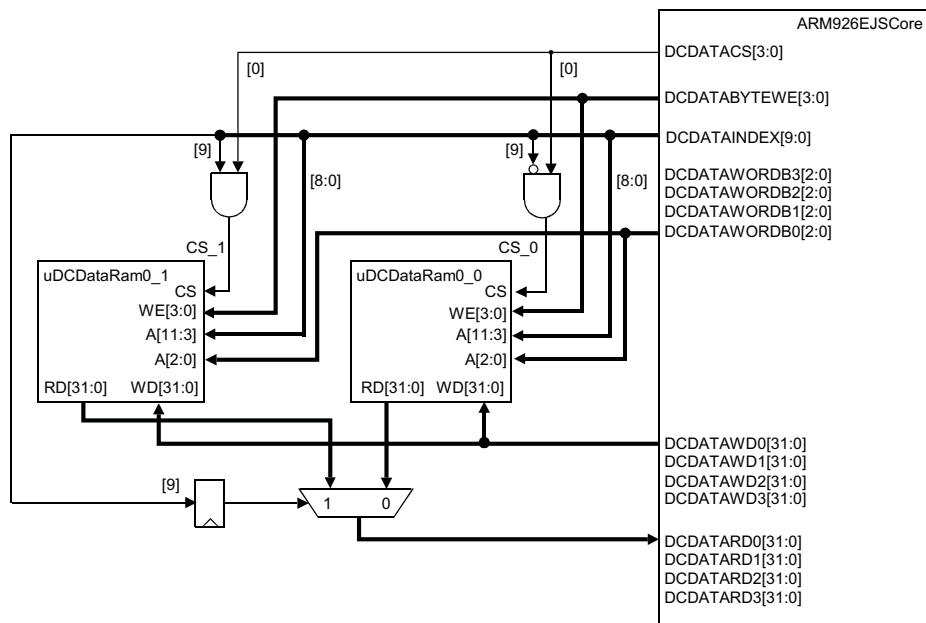


Figure 2-11 Optimizing for power

Optimizing for speed

Figure 2-12 on page 2-21 shows you how to produce uDCDataRam0 from two smaller RAM blocks (uDCDataRam0_0 and uDCDataRam0_1) if you are optimizing for speed. A separate write enable is required for each RAM block:

$$\begin{aligned} \text{WE_0}[3:0] &= \{4'b\{\sim\text{DCDATAINDEX}[9]\}\} \& \text{DCDATABYTEWE}[3:0]; \\ \text{WE_1}[3:0] &= \{4'b\{\text{DCDATAINDEX}[9]\}\} \& \text{DCDATABYTEWE}[3:0]; \end{aligned}$$

where:

- WE_0[3:0] are the write enables for uDCDataRam0_0
- WE_1[3:0] are the write enables for uDCDataRam0_1.

No logic is added to the critical **DCDATACS** path but both RAM blocks are enabled whenever **DCDATACS[0]** is asserted resulting in higher power consumption.

———— Note ————

The AND gate in Figure 2-12 on page 2-21 is four bits wide. If the RAM has word-write control, the AND gate (and resulting write enable signal) is one bit wide.

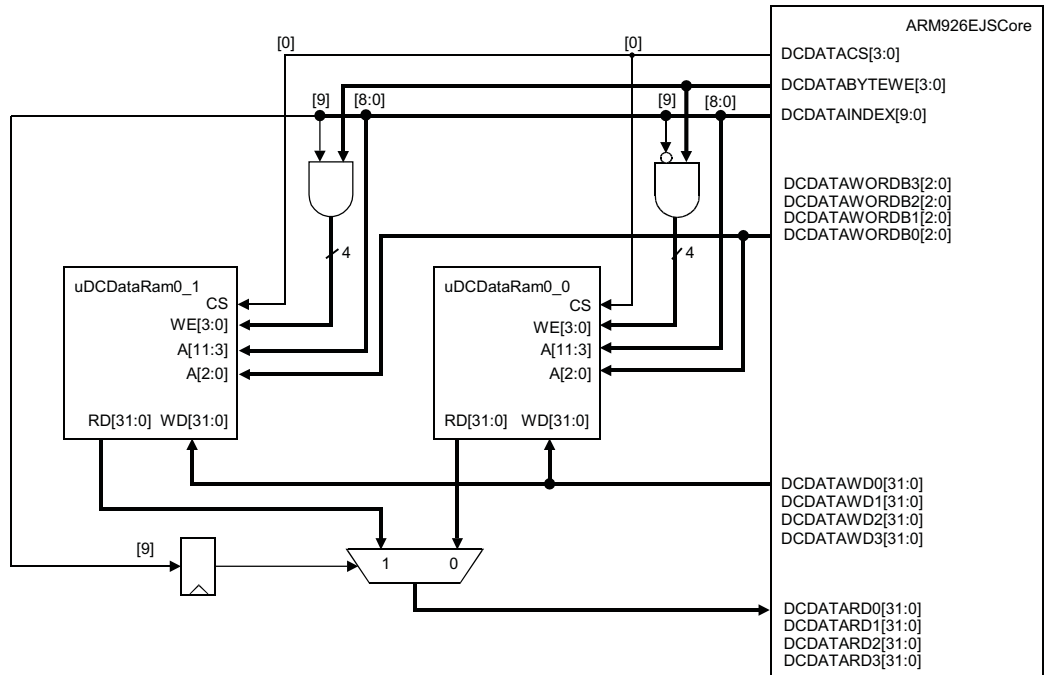


Figure 2-12 Optimizing for Speed

2.4 Integrating your RAM blocks

This section describes how you integrate your RAM blocks. RAM integration is performed at the ARM926EJS level. All RTL edits required to perform RAM integration are done within your verilog/MAIN_ARM926EJS directory. There are two methods:

- *Manual integration.*
- *Automatic integration* on page 2-25. A script is provided to automate the RAM integration process. If you have several macrocell variants to create, you can use this script to reduce the RAM integration effort.

2.4.1 Manual integration

To perform manual RAM integration:

1. Navigate to the verilog/MAIN_ARM926EJS directory:

```
cd verilog/MAIN_ARM926EJS
```
2. Copy the example top-level ARM926EJS.v from the verilog/ARM926EJS directory to the verilog/MAIN_ARM926EJS directory and rename it using the convention ARM926EJS_<ICache size><DCache size>.v. For example, if your ICache is 8KB and your DCache is 16KB type:

```
cp ../ARM926EJS/ARM926EJS.v ARM926EJS_8K16K.v
```
3. See *RAM organization* on page 2-4 for information on how you must connect RAM for each cache size. Use this to identify the RAM blocks that you require then generate them using your library RAM generator.

————— Note —————

The example ARM926EJS.v contains a gasket, a926CacheRamGasket, that interfaces between the ARM926EJSCore and the RAM blocks. This gasket enables ARM to simulate different cache sizes with a single RAM model, masking out the upper address bits depending on the cache size.

To prevent the upper address bits being masked out, you must remove this gasket from ARM926EJS.v that you created in step 2, and connect up the address bits as described in *Instruction cache data RAM* on page 2-7 to *Data cache dirty RAM* on page 2-17.

4. Replace the example RAM blocks in your new top-level ARM926EJS module, that you generated in step 2, with the RAM blocks that you generated in step 3. The ARM926EJSCore level drives all RAM control signals as active HIGH. If your RAM blocks have active LOW control signals, then you must add the logic to create the inversion. For example, to create an active LOW version of the DCache RAM enable signal, add the following code:


```
wire [3:0] DCDATACS_activeLOW;
assign DCDATACS_activeLOW = ~DCDATACS;
```

Then you must connect DCDATACS_activeLOW to your RAM chip select port instead of the **DCDATACS** signal.

5. Integrate any RAM test logic required into your top-level ARM926EJS module. See *Integrating RAM test logic* on page 2-31 for more details.
6. Take a copy of the example ARM926EJS.vc file from the verilog/ARM926EJS directory to the verilog/MAIN_ARM926EJS directory and rename it using the convention ARM926EJS_<ICache size><DCache size>.vc. For example, if your ICache is 8KB and your DCache is 16KB type:

```
cp ../ARM926EJS/ARM926EJS.vc ARM926EJS_8K16K.vc
```
7. Edit your copy of the example ARM926EJS.vc file to:
 - a. add links to any libraries and verilog files as required that contain the logic added in step 5
 - b. add links for any other logic that you have added.

———— **Note** ————

Do not add a link to your RAM library path in this file.

8. Edit the libraries.vc file in the verilog/MAIN_ARM926EJS directory adding a link to the directory where your RAM verilog files are. The libraries.vc file also contains links to the directories where the *DesignWare* (DW) components used in the ARM926EJS level are. Edit these paths to reflect your Synopsys site installation.
 If you use any DW components from a directory other than those listed in this file, add a link to that directory path.
9. Take a copy of the example ARM926EJS.vh file from the verilog/ARM926EJS directory to the verilog/MAIN_ARM926EJS directory and rename it using the convention ARM926EJS_<ICache size><DCache size>.vh. For example, if your ICache is 8KB and your DCache is 16KB type:

```
cp ../ARM926EJS/ARM926EJS.vh ARM926EJS_8K16K.vh
```
10. Edit your copy of the example ARM926EJS.vh:
 - a. If architectural clock gating is required set the define:

```
'define BLOCK_LEVEL_CLOCK_GATING
```

 Otherwise remove it.

———— **Note** ————

For information on the BLOCK_LEVEL_CLOCK_GATING define and its impact on architectural clock gating contact ARM Limited.

- b. Remove the define:

```
'define TESTCHIP
```

- c. Remove the define:

```
'define FPGA
```

Caution

If this define is set, your macrocell has extra ports that do not match the expected port list.

- d. Set the DATA_CACHE_SIZE and INSTR_CACHE_SIZE defines to the value that represents your cache sizes for your macrocell. Valid encodings are shown in Table 2-10.

Table 2-10 CACHE_SIZE encodings

Cache size	DATA_CACHE_SIZE	INSTR_CACHE_SIZE
4KB	4'b0011	4'b0011
8KB	4'b0100	4'b0100
16KB	4'b0101	4'b0101
32KB	4'b0110	4'b0110
64KB	4'b0111	4'b0111
128KB	4'b1000	4'b1000

For example, if your ICache is 8KB and your DCache is 16KB, you set the defines to:

```
'define DATA_CACHE_SIZE 4'b0101
'define INSTR_CACHE_SIZE 4'b0100
```

11. Create links to the three files that you have created:

```
ln -s ARM926EJS_8K816K.v ARM926EJS.v
ln -s ARM926EJS_8K816K.vc ARM926EJS.vc
ln -s ARM926EJS_8K816K.vh ARM926EJS.vh
```

These links enable the synthesis and validation environments to automatically pick up your files. If you have several macrocell variants, you can select the version to be synthesized and simulated by changing the links to point to the variant that you require.

Note

You can use the command `make_default_links` to set up links to the default ARM926EJ-S supplied with the macrocell release. You only do this if you want to simulate the default ARM926EJ-S macrocell.

12. Check that the RAM integration is correct by running the *RAM Integration testbench* on page 2-29.

2.4.2 Automatic integration

A script, `automem`, is provided that automatically creates a top-level macrocell with your RAM blocks integrated. This script is in the `verilog/MAIN_ARM926EJS` directory.

To perform automatic RAM integration:

1. Navigate to the `verilog/MAIN_ARM926EJS` directory.
`cd verilog/MAIN_ARM926EJS`
2. Copy the `ram-config` file and rename it using the convention `ARM926EJS_<ICache size><DCache size>`. For example, if your instruction cache size is 8KB and your data cache size is 16KB, type:
`cp ram-config ram-config_8K16K`
3. The `ram-config` file describes the RAM to be used for each of the RAM blocks described in *MMU RAM* on page 2-6 to *Data cache dirty RAM* on page 2-17. Edit the `ram-config` file that you created in step 2 and set the variables as defined in Table 2-11 on page 2-26.

The `automem` script automatically detects if a selected RAM is either too narrow or not deep enough for the required size. It instantiates multiple banks of the selected RAM to build the necessary width and/or depth. The selected RAM must satisfy the conditions outlined in *Producing byte-write memory from word-write RAM* on page 2-18 or *Producing a large memory from smaller RAM blocks* on page 2-19.

Table 2-11 ram-config file variable settings

Variable	Description
optimise	If you create large memory out of smaller RAM blocks, the optimise parameter selects between power or speed optimization: - Select <code>optimise power</code> to produce logic as described in <i>Optimizing for power</i> on page 2-19 - Select <code>optimise speed</code> to produce logic as described in <i>Optimizing for speed</i> on page 2-20.
replacement	You must select <code>replacement round-robin</code> .
clockgating	Selects if architectural block level clock gating is enabled. For more information on architectural clock gating contact ARM Limited. - Select <code>clockgating true</code> to enable architectural clock gating - Select <code>clockgating false</code> to disable architectural clock gating.
dcsize	Size of the DCache. Valid options are 4k, 8k, 16k, 32k, 64k, and 128k.
dcdata	Module name of the RAM to be used for the DCache data RAM.
dctag	Module name of the RAM to be used for the DCache TAG RAM. <code>automem</code> does not support the 88-bit wide RAM option.
dcvalid	Module name of the RAM to be used for the DCache valid RAM.
dcdirty	Module name of the RAM to be used for the DCache dirty RAM.
icsize	Size of the ICache. Valid options are 4k, 8k, 16k, 32k, 64k, and 128k.
icdata	Module name of the RAM to be used for the ICache data RAM.
ictag	Module name of the RAM to be used for the ICache TAG RAM. <code>automem</code> does not support the 88-bit wide RAM option.
icvalid	Module name of the RAM to be used for the ICache valid RAM.
mmu112bit	If the MMU RAM is implemented with a 112-bit wide RAM, set the module name here, otherwise set to <code>none</code> .
mmu128bit	If the MMU RAM is implemented with a 128-bit wide RAM, set the module name here, otherwise set to <code>none</code> .
mmu30bit	If the MMU is implemented with 2x30-bit RAM blocks and 2x26 RAM blocks, <code>mmu30bit</code> sets the module name of the 30-bit RAM and <code>mmu26bit</code> sets the name of the 26-bit RAM.
mmu26bit	
	If the MMU is implemented with <code>mmu128bit</code> or <code>mmu112bit</code> options, then <code>mmu30bit</code> and <code>mmu26bit</code> must both be set to <code>none</code> .

4. Edit the ram-config file that you created in step 2 and describe the RAM blocks that you selected in step 3. There must be a ram_definition description for each RAM used. The format of the ram_definition is:

```
ram_definition RAM_module_name
    clock = clk_port
    <not_>chip_select = chip_select_port
    address = address_port[x:0]
    read_data = read_data_port[x:0]
    write_data = write_data_port[x:0]
    <not_>output_enable = output_enable_port
    <not_>write_enable = write_enable_port[x:0]
    misc_ports = misc_port_list
    misc_pins = misc_pin_list
end_ram_definition
```

You must set each of the variables described in Table 2-12 to those that represent your RAM.

Table 2-12 ram-config file variable replacement

Variable	Description
RAM_module_name	Name of the RAM module that this ram_definition represents.
chip_select_port	Name of your RAM chip select (RAM Enable) port. If the chip select is active LOW, this must be set to: not_chip_select = <chip_select_port> If the chip select is active HIGH, this must be set to: chip_select = <chip_select_port>
address_port	Name of your RAM address port. The name must contain the port range. For example: address = A[12:0] represents a 13-bit address bus. If your RAM is larger than required then you must: • specify the expected address size not the size of the address port on your RAM. • tie off the additional address bit(s) manually in the Verilog file produced by automem.
read_data_port	Name of your RAM read data port. The name must contain the port range.
write_data_port	Name of your RAM write data port. The name must contain the port range.
output_enable_port	Name of your RAM output enable port. Omit this line from the definition if your RAM does not have an output enable port. If the output enable is active LOW, you must set this to: not_output_enable = <output_enable_port> If the output enable is active HIGH, you must set this to: output_enable = <output_enable_port>

Table 2-12 ram-config file variable replacement (continued)

Variable	Description
write_enable_port	Name of your RAM write enable port. If your RAM is bit or byte-writable, the name must contain the port range. Omit the port range if the write enable port is a single bit. If the write enable is active LOW, you must set this to: <pre>not_write_enable = <write_enable_port></pre> If the write enable is active HIGH, you must set this to: <pre>write_enable = <write_enable_port></pre>
misc_port_list	This is a comma-separated list of ports, other than those described in the above variables, that you want to be added to the module instance. For example: <pre>misc_ports = A, B, C</pre>
misc_pin_list	This is a comma-separated list of connections to the ports described in misc_ports. You must describe concatenations in quotes. For example: <pre>misc_pins = A1, "{B1,B2}", C1</pre>

- Run the automem script to perform RAM integration. The command for running the script is:

```
automem ../ARM926EJS/ARM926EJS.v [destination_file] [ram-config]
```

Where:

```
../ARM926EJS/ARM926EJS.v
```

This is the template file that you must always use.

destination_file This is the output file using the naming convention
ARM926EJS_<ICache size><DCache size>.v

ram-config This is the file that you created in step 4.

The script produces three files:

```
ARM926EJS_<ICache size><DCache size>.v
ARM926EJS_<ICache size><DCache size>.vc
ARM926EJS_<ICache size><DCache size>.vh
```

- Create links to the three files that you have created:

```
ln -s ARM926EJS_<ICache size><DCache size>.v ARM926EJS.v
ln -s ARM926EJS_<ICache size><DCache size>.vc ARM926EJS.vc
ln -s ARM926EJS_<ICache size><DCache size>.vh ARM926EJS.vh
```
- Edit the libraries.vc file in the verilog/MAIN_ARM926EJS directory and add a link to the directory where your RAM verilog files are. libraries.vc also contains links to the directories that the DW components used in the ARM926EJS level. Edit these paths to reflect your Synopsys site installation.

Note

If any DW components are used from a directory other than those listed in this file, you must add a link to that directory path.

8. Check that the RAM integration is correct by running the *RAM Integration testbench*.

2.4.3 RAM Integration testbench

A testbench is provided to enable you to check that your RAM blocks are correctly integrated.

Caution

Do not run the example RAM blocks on the RAM integration testbench. Failures are expected if the example RAM blocks are run on the RAM integration testbench.

Before running the testbench you must complete all steps in either:

- *Manual integration* on page 2-22
- *Automatic integration* on page 2-25.

To run the RAM integration testbench using the Verilog-XL simulator, type the command:

```
verilog -f ARM926EJS_ram_testbench.vc -f ARM926EJS.vc -f libraries.vc\
ARM926EJS_ram_testbench.v
```

To run the RAM integration testbench using the ModelTech simulator, type the commands:

```
vlib work
vlog -f ARM926EJS_ram_testbench.vc -f ARM926EJS.vc -f libraries.vc\
ARM926EJS_ram_testbench.v
vsim -do "run -all" ARM926EJS_ram_testbench
```

The testbench is self-running and self-checking. If the test has completed successfully the following message is displayed:

RAM integration passed. Test completed with no errors

If the test fails, the simulation completes with the message:

!!! RAM integration FAILED. Test completed with errors !!!

The logfile lists which of the RAM block(s) failed integration.

Note

If your RAM model performs setup and hold timing checks, you must disable the checks when running the RAM integration testbench by adding the `+notimingchecks` option to the command line.

2.5 Integrating RAM test logic

You must integrate your RAM test logic into the top-level ARM926EJS module that you created in *Integrating your RAM blocks* on page 2-22.

The normal method of achieving test access to a RAM is to add a collar of multiplexors around the RAM inputs, and route the RAM read data bus to the test logic. This method is not acceptable for the ARM926EJ-S macrocell because:

- The address and chip select signals to the cache RAM blocks are timing critical. Adding any additional logic to these paths results in performance degradation.
- The RAM read data paths are timing critical. You must minimize the load on these paths.

To enable the RAM blocks to be tested without degrading the macrocell performance, the test multiplexors required for the address and chip select signals have been built into the ARM926EJSCore logic, and structured so that they do not affect the critical path. When you collar your RAM blocks, you must collar the write enable, byte-write enable, and write data ports in the normal way, but access the address and chip select ports through the ARM926EJSCore. If your RAM collaring is performed automatically by a test tool, then you manually disconnect the multiplexors and wire the test signals to the ARM926EJSCore. Figure 2-13 shows how the test access is achieved, using the DCache dirty RAM as an example.

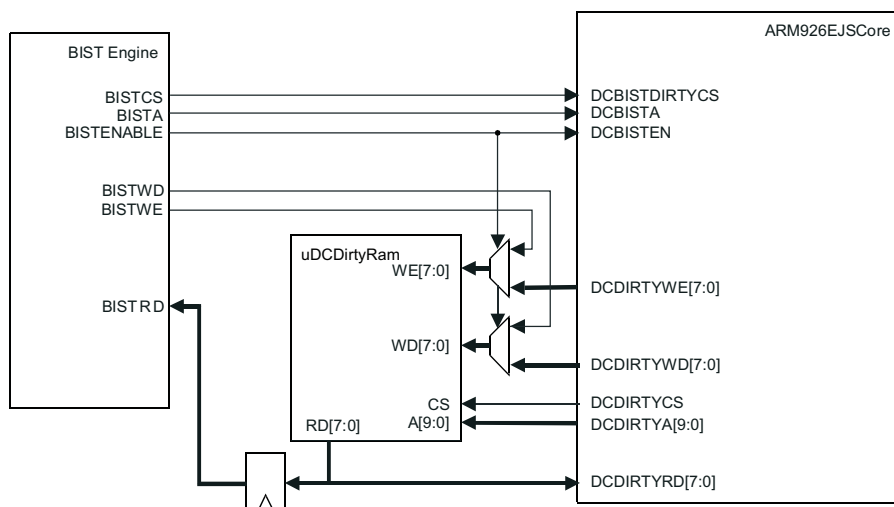


Figure 2-13 Test access using the DCache dirty RAM

Note

The test multiplexors built into the ARM926EJSCore logic have been implemented with OR gates. To ensure that the ARM926EJSCore only drives out the BIST addresses and chip selects, and not functional addresses and chip selects, the ARM926EJSCore must be held in reset during BIST testing. To achieve this, you must use a separate reset for your BIST logic than that used for the ARM926EJSCore.

The load on the RAM read data buses can be minimized by either:

- optimizing the layout so that routing to the RAM test logic is minimized
- registering the RAM output to decouple the BIST routing capacitance from the functional signal, as Figure 2-13 on page 2-31.

Table 2-13 lists the ARM926EJSCore RAM test ports and associated functionality.

Table 2-13 ARM926EJSCore RAM test ports and associated functionality

Port name	Functionality
DCBISTEN	Selects between test mode and normal functionality. When this signal is HIGH, the DCache RAM address and chip select outputs are driven with the test values. When this signal is LOW, the DCache RAM address and chip select signals are driven with the functional mode values. This signal must be driven LOW during normal operation. This signal must be tied LOW if you do not implement any test logic.
DCBISTA[12:0]	Test mode address for all DCache RAM blocks. This value is driven to all DCache RAM blocks when DCBISTEN is HIGH. See <i>BIST to data cache TAG RAM special considerations</i> on page 2-33 for more information.
DCBISTTAGCS[3:0]	Test mode chip select for DCache TAG RAM. DCTAGCS[3:0] is driven with this value when DCBISTEN is HIGH.
DCBISTDATAACS[3:0]	Test mode chip select for DCache data RAM. DCDATAACS[3:0] is driven with this value when DCBISTEN is HIGH.
DCBISTVALIDCS	Test mode chip select for DCache valid RAM. DCVALIDCS is driven with this value when DCBISTEN is HIGH.
DCBISTDIRTYCS	Test mode chip select for DCache dirty RAM. DCDIRTYCS is driven with this value when DCBISTEN is HIGH.
ICBISTEN	Selects between test mode and normal functionality. When this signal is HIGH, the ICache RAM address and chip select outputs are driven with the test values. When this signal is LOW, the ICache RAM address and chip select signals are driven with the functional mode values. This signal must be driven LOW during normal operation. This signal must be tied LOW if you do not implement any test logic.

**Table 2-13 ARM926EJSCore RAM test ports and associated functionality
(continued)**

Port name	Functionality
ICBISTA[12:0]	Test mode address for all ICache RAM blocks. This value is driven to all ICache RAM blocks when ICBISTEN is HIGH.
ICBISTTAGCS[3:0]	Test mode chip select for ICache TAG RAM. ICTAGCS[3:0] is driven with this value when ICBISTEN is HIGH.
ICBISTDATAACS[3:0]	Test mode chip select for ICache data RAM. ICDATAACS[3:0] is driven with this value when ICBISTEN is HIGH.
ICBISTVALIDCS	Test mode chip select for ICache valid RAM. ICVALIDCS is driven with this value when ICBISTEN is HIGH.

Note

Test multiplexors for the MMU RAM are not built into the ARM926EJSCore logic because the MMU RAM interface is less time critical than the cache RAM interfaces.

2.5.1 BIST to data cache TAG RAM special considerations

DCTAGA[10] always drives the top address bit to the DCache TAG RAM, as described in Table 2-7 on page 2-15. **DCBISTA[10:0]** directly maps to **DCTAGA[10:0]**. You must ensure that your top BIST address bit maps to **DCTAGA[10]**. Table 2-14 on page 2-34 shows the **DCBISTA** ports that your BIST engine must connect to for each cache size.

Note

You cannot run BIST on the DCache data RAM when you are running BIST on the DCache TAG RAM because this address mapping is not valid for the DCache data RAM.

Table 2-14 DCBISTA port connections

Cache size	BIST address size	DCBISTA port connections
4KB	6 bits	DCBISTA[10], DCBISTA[4:0]
8KB	7 bits	DCBISTA[10], DCBISTA[5:0]
16KB	8 bits	DCBISTA[10], DCBISTA[6:0]
32KB	9 bits	DCBISTA[10], DCBISTA[7:0]
64KB	10 bits	DCBISTA[10], DCBISTA[8:0]
128KB	11 bits	DCBISTA[10], DCBISTA[9:0]

2.5.2 Integrated test logic scan consideration

The BIST inputs to the ARM926EJSCore must be controllable during scan testing. You can achieve this by:

- Gating the ARM926EJSCore BIST inputs with TESTMODE to ensure that the BIST inputs are tied LOW during test.

———— **Note** ————

The stuck-at fault coverage of the BIST paths is lost using this method, however these paths are tested during BIST.

- Adding the inputs of your BIST controller to the INTEST wrapper. This enables you to produce vectors that enable the BIST inputs to the ARM926EJSCore to be put into a known state.

Chapter 3

RTL Validation

This chapter gives a general explanation of the validation tools, validation suites, and configurations used for RTL validation. It also describes how to build and simulate the macrocell, and how to run the validation program. It contains the following sections:

- *About RTL validation* on page 3-2
- *Supported RTL validation flow* on page 3-7
- *Simulator and platform support for RTL validation* on page 3-8
- *Setting up your validation environment* on page 3-9
- *Building the model* on page 3-10
- *Running the RTL validation* on page 3-12
- *Debugging a single validation test* on page 3-18
- *Validation script usage* on page 3-19
- *val_report script usage* on page 3-22
- *Script file generation* on page 3-25.

3.1 About RTL validation

RTL validation is the process of running the validation test suites on the RTL simulation model. Macrocell RTL validation comprises:

Validation tests

Self-checking test programs written in ARM assembler for macrocell functional validation.

Validation suites

A collection of validation tests that validate specific areas of the macrocell functionality.

Validation configurations

These supply a number of additional external conditions to the core while a validation test is run. A test run with no configurations invoked is termed *native*.

Validation tools

The Perl programs used to automate RTL validation.

The collection of validation suites, validation tools, and validation configurations are in the `./validation` directory.

This section gives an overview of the:

- *Validation tools*
- *RTL validation configuration files* on page 3-3
- *RTL validation test suites* on page 3-4
- *RTL validation suite configurations* on page 3-5.

3.1.1 Validation tools

The validation tools are programs written in Perl that are used to perform validation of the macrocell RTL simulation model. The two principal tools are:

- *val_report* on page 3-3
- *validation* on page 3-3.

val_report

Macrocell validation consists of many validation tests, organized into several validation suites. `val_report` is the primary tool that controls the process of running the macrocell validation tests. In most cases you can complete the entire RTL validation process with the `val_report` tool. This tool:

- generates formatted reports of validation test results by parsing validation test log files
- optionally generates filtered reports according to test results (passed, failed, or abandoned)
- optionally generates job files consisting of per test validation commands for batch submission of validation suites.

validation

This tool runs an individual test, and is not directly used because `val_report` invokes it for each test to be run. However, `validation` is used when interactive debugging of failing tests is required. This tool:

- automates the whole process of building and running an individual validation test
- optionally configures validation tests
- optionally configures the RTL simulation model
- generates log files containing validation test results
- invokes the chosen simulation tool in batch or interactive mode.

3.1.2 RTL validation configuration files

There are three configuration files that you might have to edit to set up for validation of the RTL simulation model:

- `dotcshrc`
- `validation.cfg`
- `val_report.cfg` on page 3-4.

dotcshrc

This is a csh script file, that you can modify, that adds all the validation tools required for macrocell validation onto the `PATH` environment variable.

validation.cfg

This is a text file that you can modify to configure validation and `val_report` for macrocell validation. It also configures the validation tool for use with different simulators. See *Running validation test suites* on page 3-12 for information on how to use `validation.cfg`.

val_report.cfg

This is a text file, that you can modify, that defines:

- validation suite specific parameters used by the val_report script to generate batch job files
- what suite configurations are available.

3.1.3 RTL validation test suites

The validation tests are grouped into several suites that target specific areas of functionality. Table 3-1 lists the available macrocell test suites that are specified in the validation.cfg file using the \$ValSrcGeneric and \$ValSrcProject variables. The individual validation tests that comprise a validation suite are specified in source_list files located within each suite directory.

Table 3-1 RTL validation test suites

Test suite	Description
arm9e	Validation tests that verify ARM9E-S core level behavior
arm9ej	Validation tests that verify ARM9EJ-S core level behavior
arm926ej	Validation tests that verify macrocell level behavior
arm9tdmi	Validation tests that verify ARM9TDMI core level behavior
armv4	Validation tests that verify the ARMv4T architectural behavior
armv5	Validation tests that verify the ARMv5TE architectural behavior
cache	Validation tests that verify the correct operation of cache
debug	Validation tests that verify the hardware debug functionality of the macrocell
etm	Validation tests that verify the processor ETM interface
hand	Validation tests that verify the real-time debug functionality of the macrocell
hazard	Validation tests that verify pipeline forwarding and interlock logic, checking that cycle times of operations are as expected
mult16	Validation tests that verify the multiplier instructions added in the ARMv5TE architecture
multiplier	Contains tests for the long multiplier instructions defined in the ARMv4T architecture
multiplerv5	Additional long multiplier validation tests for the ARMv4 architecture
ris_9es	Validation tests that apply randomly generated ARMv5TE instruction sequences
thumb	Validation tests that verify the Thumb instruction set and the switch mechanisms between ARM and Thumb states

Table 3-1 RTL validation test suites (continued)

Test suite	Description
fcse	Validation tests that verify <i>Fast Context Switch Extension</i> (FCSE) write support
mmu	Validation tests that verify correct <i>Memory Management Unit</i> (MMU) operation
mmue	Validation tests that verify correct MMU extensions operation
jazelle	Validation tests that check Java bytecode functionality in the macrocell
ris_926	Random instruction sequences to that verify macrocell functionality

3.1.4 RTL validation suite configurations

You can run many of the validation tests with extra functionality enabled, such as the caches or *Tightly-Coupled Memories* (TCMs), to increase the RTL verification coverage without the requirement for a completely new test. This functionality is provided by validation test *configuration files* (configs), and applied on the validation script command line using the `-config` switch. You can apply combinations of multiple configurations using a dot (.) separated list argument, for example:

```
validation -config hivecs.randomHCLK.randomSPLIT T1-32
```

The application of configs results in the test name being modified so that it can be uniquely distinguished from other tests of the same name. For the example above the name can be:

```
T1-32.hivecs.randomHCLK.randomSPLIT
```

The configurations listed in Table 3-2 are applicable to macrocell validation tests.

Configurations are normally applied on a suite-by-suite basis, and the ones supported by each suite are listed in the `val_report.cfg` file. For each suite, configuration-specific `source_lists` are required to select only the tests that can be run with a given set of configurations. These `source_lists` are in the appropriate suite directory.

Table 3-2 Validation configuration files

Configuration name	Behavior
AHBmultilayer	Run tests with TCMs in multilayer mode.
bigend	Run most tests in big-endian mode (big-endian configurable memory model is required).
boot	Link test for hivecs address (0xFFFF0000), enables ITCM of 16KB at 16KB base address, and copies test code down to 4K address base. Therefore, tests greater than 28KB are executed from external memory.
dcacheon	Enable DCache memory regions.

Table 3-2 Validation configuration files (continued)

Configuration name	Behavior
dtcmn	Enable DTCM and copies code to DTCM (instruction accesses go external but data accesses are first out to DTCM).
etmEnTrc	Enable the ETM Trace.
etmLarge_p16_ad	Enable a Large ETM config, with 16-bit wide trace port, trace address and data.
hclk1to<x>	HCLK bus ratios where x is 2 - 8.
icacheon	Enable ICache memory regions.
itcmn	Enable ITCM and copies code to ITCM for execution.
large	Use for mmue suite to configure page tables to use large pages.
ncb	Enable data cache with code running in a NCB area.
ncnb	Enable data cache with code running in a NCNB area.
randomAHB	Enable random waits, splits, and retries on the AHB interface, Passes randomWAIT, randomRetry, randomHCLK, and randomSplit arguments to tbench.v in one go at simulation time.
randomTCM	Perform random waits on the DTCM and ITCM interfaces.
ris_boot	Use for RIS tests that use the data table offset to copy code into ITCM, and enable DTCM for data accesses (good for getting ITCM and DTCM going together).
SampleInt	Enable deterministic testing of the ARM9E-S core. This configuration is required because of the reduced interrupt latency of the ARM9E-S core.
section	Use for mmue suite to configure page tables to use section pages.
small	Use for mmue suite to configure page tables to use small pages.
TCMWAIT<xy>	Deterministic TCM wait states where x is the number of waits for NSEQ accesses and y is the number of waits for SEQ accesses.
tiny	Use for mmue suite to configure page tables to use tiny pages.
wb	Enable data cache with code running in a write-back area.
wdog	Link in the watchdog initialization code to enable the test bench watchdog in the Trick Box.
wt	Enable data cache with code running in a write-through area.

3.2 Supported RTL validation flow

The supported RTL validation flow shown in Figure 3-1 enables you to:

1. Set up the appropriate validation environment, see *Setting up your validation environment* on page 3-9.
2. Build the RTL model with the simulator, see *Building the model* on page 3-10.
3. Run the validation suites, see *Running the RTL validation* on page 3-12.
4. Check that the validation test has completed satisfactorily, see *RTL validation status* on page 3-15.
5. Debug the validation test if it has failed to complete satisfactorily. It can be run interactively using the features available in your chosen simulator, see *Debugging a single validation test* on page 3-18.

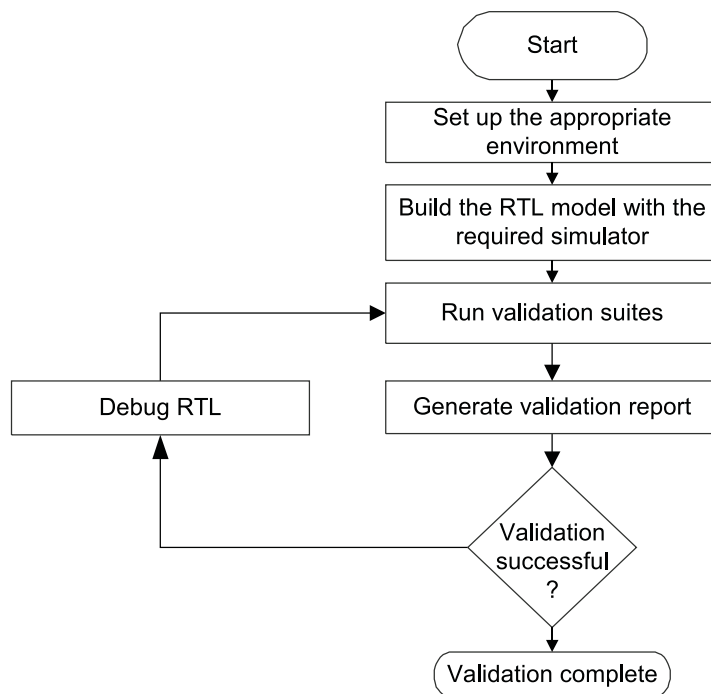


Figure 3-1 Building and validating the macrocell

3.3 Simulator and platform support for RTL validation

The RTL validation kit supports the following commonly used simulators:

- MTI Vsim
- Cadence Verilog_XL
- Cadence NC_Verilog
- Synopsys VCS.

Additionally, support exists for compute platforms other than Solaris, see Table 3-3.

The RTL testbench contains some Modelgen components, notably the BST and the disass. Builds of these models are provided for Solaris, Linux, and HP-UX. The RTL compile scripts described in *Building the model* on page 3-10 automatically detect the platform OS, and link in the appropriate builds of these modelgen models. The validation tools `validation` and `val_report` also support these platforms.

Note

There is no support for Microsoft Windows based platforms.

Table 3-3 shows which simulator is supported on which platform. VCS with HP-UX is untested by ARM Limited.

Table 3-3 Simulator support for platforms

Simulator	Solaris	Linux	HP-UX
NC_Verilog	Supported	Supported	Supported
Verilog_XL	Supported	Supported	Supported
VCS	Supported	Supported	-
MTI Vsim	Supported	Supported	Supported

3.4 Setting up your validation environment

You must ensure there is a link from your validation directory to your verilog directory. If your verilog directory is in the same directory as your validation directory, you do this from within the validation directory by typing:

```
ln -s ../verilog verilog
```

Note

You must ensure that all the EDA tools necessary to build and simulate the RTL are included on your search path.

The following scripts assume that the Perl binary is in `/usr/local/bin/perl`:

```
validation/tools/etm/bin/Convert.pl
validation/tools/etm/bin/Decomp.pl
validation/tools/etm/bin/EtmCompare
validation/tools/etm/bin/EtmDefs.pl
validation/tools/etm/bin/EtmRsrcSel
validation/tools/etm/bin/EtmMksrclists
validation/tools/etm/dotperl
```

If your site has Perl installed in a different location, edit the first line of the scripts to point to the correct location. For example, if Perl is available at `/usr/bin/perl`, change:

```
#!/usr/local/bin/perl -w
```

to read:

```
#!/usr/bin/perl -w
```

3.5 Building the model

This section describes how to build the macrocell model, ready for simulation, using the provided compile scripts for the common simulators and UNIX operating systems. The steps are:

- *Setting up the environment*
- *Compiling the model.*

3.5.1 Setting up the environment

Each compile script uses a common top-level options file, `verilog/tbenchfpga/tbench926.vc`, which specifies all the source files and library directories that comprise the RTL model. This options file references a site-specific options file, `verilog/MAIN_ARM926EJS/libraries.vc`, containing references to the Synopsys GTECH and DesignWare Verilog source files, and your RAM libraries. You must change the following lines of this file to reflect your site installation:

```
// site-specific synopsys supplied library dirs
-y /eda/tools/synopsys/default/dw/dw01/src_ver
-y /eda/tools/synopsys/default/dw/dw04/src_ver

// site-specific synopsys supplied source files
-y /eda/tools/synopsys/default/packages/gtech/src_ver
-v /eda/tools/synopsys/default/packages/gtech/src_ver/gtech_lib.v

// other site specific libraries

-y verilog/RAM
```

Where:

default must be replaced with your Synopsys tool version, for example:
2003.03-1

3.5.2 Compiling the model

Before you attempt to compile the model, make sure that you have correctly configured the environment as described in *Setting up the environment*.

———— Note ————

Two testbenches are supplied with the macrocell model. One testbench is located in the `tbench` directory, and the other is located in the `tbenchfpga` directory. You must use the testbench in the `tbenchfpga` directory for RTL validation simulation.

To compile the macrocell model and its testbench:

1. Choose an operating system on which to compile. The supported operating systems are:
 - SunOS/Solaris
 - Linux
 - HP-UX.
2. Log onto a machine with your chosen operating system and change directory to the verilog directory:
`cd verilog`
3. Choose a simulator to compile the model under. The supported simulators are:
 - VCS
 - NC
 - verilog XL
 - MTI Modelsim.
4. Ensure your chosen simulator is available on your search path.
5. Change directory to your chosen simulator compile directory by typing:
 - a. `cd VCS` for VCS simulator
 - b. `cd NC` for NC Verilog simulator
 - c. `cd MTI` for MTI Modelsim simulator.

———— **Note** ————

For Verilog-XL there is no requirement to precompile the model because the model is compiled with each simulation invocation by the validation script.

6. Run the simulator specific compile script by typing:
 - a. `compile_VCS` for VCS
 - b. `compile_NC` for NC Verilog
 - c. `compile_MTI` for MTI Modelsim.

———— **Note** ————

Current compile scripts optimize for speed. To run interactively with a GUI you must have additional compile options. Contact ARM Limited for further information.

3.6 Running the RTL validation

This section gives details on:

- *Running validation test suites*
- *RTL validation status* on page 3-15.

Note

Ensure that the compilation of your model is complete before running RTL validation.

3.6.1 Running validation test suites

The validation suites contain about 5 600 tests. The sequential simulation time for complete RTL validation is approximately 200 hours (This was achieved by ARM using Synopsys VCS running on Linux platforms). To reduce the time required to complete the RTL validation, you can spread the simulations across several computers where possible. There are two ways to run validation test suites:

- *Running with LSF*
- *Running without LSF* on page 3-13.

Running with LSF

RTL validation can use the *Load Share Facility* (LSF) workload software suite from Platform Computing. To run validation test suites with LSF:

1. Ensure your chosen simulator and the RTL validation tools are on your search path. To make the RTL validation tools available:
 - a. change to the validation directory
 - b. type:
source dotcshrc
2. Modify the validation.cfg file to:
 - a. Ensure the LSF enable setting is enabled:
\$UseLsf = 1;
 - b. Change the temporary run directory path prefix to suit your file system. The default is:
\$ValRunHome = "/home/\$ENV{USER}";
 - c. Change the bsub command prefix to suit your cluster practices. The default is:
\$BsubCmd= "bsub -F 400000 -o /dev/null";
where:
-F 400000 is the maximum permissible file size specified in Kilobytes.
-o /dev/null writes the LSF log file to dev/null.

- d. Change the default resource settings as required. The default settings for each operating system are:


```
if ($OpSys =~ /^SunOS/) {
    $LsfDefaultResource = "solaris";
}
elseif ($OpSys =~ /^Linux/) {
    $LsfDefaultResource = "linux";
}
elseif ($OpSys =~ /^HP-UX/) {
    $LsfDefaultResource = "hpux";
}
```
- e. Change the simulator resource requirement settings as required. The settings are specified per simulator in the simulator setup subroutine. For example, for VCS the value is:


```
sub vcs_setup {
    $LsfResource= "select[VCSRuntimeNet && mem > 400]";
```
- f. Change the queue to where the validation jobs are submitted. The default setting is:


```
$LsfDefaultQueue = "normal";
```
- g. Change the LsfShortQueue setting to specify the queue to be used for simulations that take less than 3 600 seconds to complete. The default setting is:


```
$LsfShortQueue = "short 3600";
```
- h. Change the LsfQuickQueue setting to specify the queue to be used for simulations that take less than 60 seconds to complete. The default setting is:


```
$LsfQuickQueue = "minute 60";
```
3. Submit all the tests to your compute cluster. If you are using Synopsys VCS as your simulator, type:


```
val_report -sub -vcs
```

Running without LSF

To run validation test suites without LSF:

1. Ensure your chosen simulator and the RTL validation tools are on your search path. To make the RTL validation tools available:
 - a. change to the validation directory
 - b. type:


```
source dotcshrc
```
2. Modify the validation.cfg file to:
 - a. Ensure the LSF enable setting is disabled:


```
$UseLsf = 0;
```

- b. Change the temporary run directory path prefix to support your file system.
The default is:
`$ValRunHome = "/home/$ENV{USER}";`
3. Create batch job script files for all tests:
`val_report logs -s`
 For each suite in the `val_report.cfg` file, `val_report` produces one shell script file, for every validation configuration listed, and writes these script files to the `validation/jobs` directory.
4. Execute the batch scripts in the `validation/jobs` directory. To reduce the time required to complete the RTL validation, you must distribute the batch scripts onto your compute cluster as appropriate:
 - To run the scripts in parallel:
 1. Identify the number of job scripts needing to be run and identify the available machines in your compute cluster.
 2. Decide how job scripts are to be shared to each machine.
For example:
 - if you have 100 scripts and 20 machines you can run 5 scripts per machine.
 - run all the scripts for a given suite on a particular machine.
 3. For each share of job scripts, log onto a machine, then:
 - a. Set up the validation environment on the machine to enable RTL validation.
 - b. Run the scripts, for example at the `cs`h prompt type:


```
cd validation
foreach i (jobs/<list of scripts>)
$i
end
```
 - To run the scripts sequentially:
 1. You can use:
`val_report logs -s -exec`
 2. At the `cs`h prompt:


```
foreach i (jobs/*)
$i
end
```

3.6.2 RTL validation status

To review the status of the entire RTL validation, use the command:

```
val_report
```

`val_report` parses all the available log files suite by suite, producing an output report.

val_report example

This section gives two examples of using `val_report` to produce individual reports for the validation status of the armv5 suite:

- Figure 3-2 on page 3-16 shows an example report using the command:
`val_report armv5 -config 'wdog$'`
- Figure 3-3 on page 3-17 shows an example report using the command:
`val_report armv5 -config 'wdog$' -full`

For a full explanation of how to use the `val_report` script see *val_report script usage* on page 3-22.

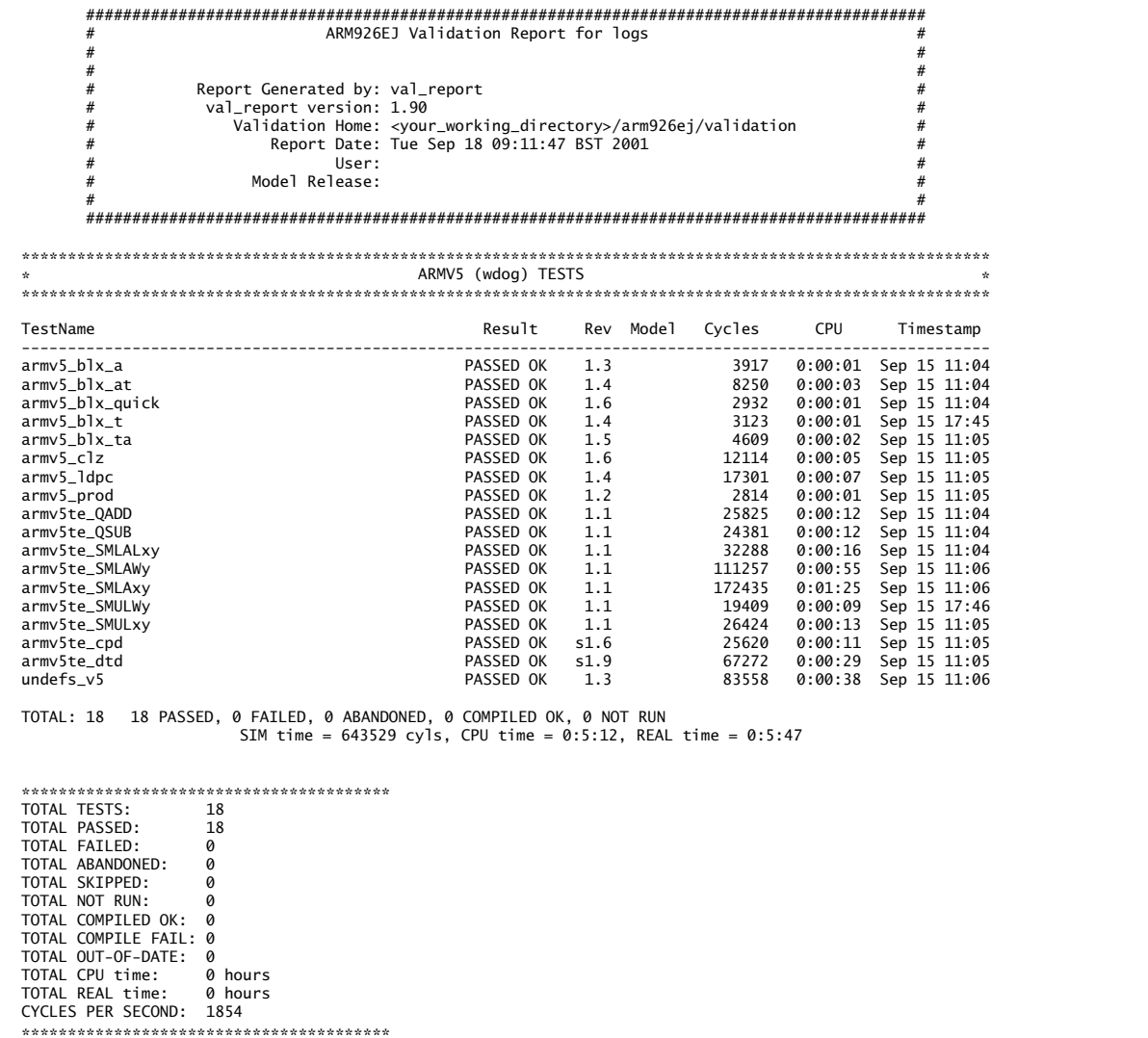


Figure 3-2 Validation report, normal status

```
#####
#                               ARM926EJ Validation Report for Logs                               #
#                               -----                               #
#                               Report Generated by: val_report                               #
#                               val_report version: 1.90                               #
#                               Validation Home: <your_working_directory>arm926ej/validation    #
#                               Report Date: Tue Sep 18 09:11:40 BST 2001                    #
#                               User:                                                       #
#                               Model Release:                                              #
#                               #####                                                       #
#####
*****
*                               ARMV5 (wdog) TESTS                                         *
*****
TestName      Result      Rev  Model Err Warn  Cycles  Size  Sim  Host      Real      CPU  cov  Timestamp
-----
armv5_b1x_a    PASSED OK    1.3      0    0      3917    4   vcs  cam-linux5 0:00:04 0:00:01 - Sep 15 11:04
armv5_b1x_at   PASSED OK    1.4      0    0      8250    5   vcs  cam-linux4 0:00:06 0:00:03 - Sep 15 11:04
armv5_b1x_quick PASSED OK    1.6      0    0      2932    4   vcs  cam-linux4 0:00:03 0:00:01 - Sep 15 11:04
armv5_b1x_t    PASSED OK    1.4      0    0      3123    5   vcs  cam-linux2 0:00:04 0:00:01 - Sep 15 17:45
armv5_b1x_ta   PASSED OK    1.5      0    0      4609    6   vcs  cam-linux5 0:00:03 0:00:02 - Sep 15 11:05
armv5_clz      PASSED OK    1.6      0    0     12114    4   vcs  cam-linux5 0:00:07 0:00:05 - Sep 15 11:05
armv5_ldpc     PASSED OK    1.4      0    0     17301    7   vcs  cam-linux5 0:00:08 0:00:07 - Sep 15 11:05
armv5_prod     PASSED OK    1.2      0    0      2814    4   vcs  cam-linux5 0:00:02 0:00:01 - Sep 15 11:05
armv5te_QADD   PASSED OK    1.1      0    0     25825   30   vcs  cam-linux4 0:00:14 0:00:12 - Sep 15 11:04
armv5te_QSUB   PASSED OK    1.1      0    0     24381   29   vcs  cam-linux4 0:00:13 0:00:12 - Sep 15 11:04
armv5te_SMLALxy PASSED OK    1.1      0    0     32288   46   vcs  cam-linux4 0:00:18 0:00:16 - Sep 15 11:04
armv5te_SMLAWy PASSED OK    1.1      0    0     111257  131   vcs  cam-linux5 0:01:01 0:00:55 - Sep 15 11:06
armv5te_SMLAxy PASSED OK    1.1      0    0     172435  199   vcs  cam-linux4 0:01:28 0:01:25 - Sep 15 11:06
armv5te_SMULWy PASSED OK    1.1      0    0     19409   23   vcs  cam-linux2 0:00:13 0:00:09 - Sep 15 17:46
armv5te_SMULxy PASSED OK    1.1      0    0     26424   31   vcs  cam-linux4 0:00:15 0:00:13 - Sep 15 11:05
armv5te_cpd    PASSED OK    s1.6     0    0     25620   12   vcs  cam-linux4 0:00:13 0:00:11 - Sep 15 11:05
armv5te_dtd    PASSED OK    s1.9     0    0     67272   26   vcs  cam-linux4 0:00:31 0:00:29 - Sep 15 11:05
undefs_v5     PASSED OK    1.3      0    0     83558   54   vcs  cam-linux5 0:00:44 0:00:38 - Sep 15 11:06

TOTAL: 18 18 PASSED, 0 FAILED, 0 ABANDONED, 0 COMPILED OK, 0 NOT RUN
SIM time = 643529 cyls, CPU time = 0:5:12, REAL time = 0:5:47

*****
TOTAL TESTS:      18
TOTAL PASSED:     18
TOTAL FAILED:     0
TOTAL ABANDONED:  0
TOTAL SKIPPED:    0
TOTAL NOT RUN:    0
TOTAL COMPILED OK: 0
TOTAL COMPILE FAIL: 0
TOTAL OUT-OF-DATE: 0
TOTAL CPU time:   0 hours
TOTAL REAL time:  0 hours
CYCLES PER SECOND: 1854
*****
```

Figure 3-3 Validation report, full status

3.7 Debugging a single validation test

If a validation test has failed you can run it interactively so that it can be debugged using the features available in your chosen simulator.

Select and run the desired test by using the filtering options of `val_report`:

1. Identify the test by noting its name, suite, and configurations by typing:

```
val_report -nogaudy -npass
```

Where:

`-nogaudy` prints the name of the log file being parsed and its basic status

`-npass` selects only those tests that have not passed, that is all failing tests.

An example output from the command is:

```
armv4/logs/T1-32.wdog.wb.icacheon.dcacheon.log      FAILED  0:00:12   Sep 6 09:55
```

Where:

`armv4` is the suite

`T1-32` is the test name

`wdog.wb.icacheon.dcacheon`
are the configurations.

2. Run the test interactively using the following `val_report` syntax:

```
val_report <suite> -test <name> -config <configs> -vopt "-I" -s -exec
```

Where:

`-vopt "-I"` supplies to validation script the run interactively option

`-s` creates batch job scripts for each matching test

`-exec` executes each batch job script created.

The `-test <name>` and `-config <configs>` filter options match using Perl regular expressions. So, to obtain a precise match it might be necessary to use the `^` and `$` symbols to match from the beginning to the end of the string respectively.

For more information on the use of this script see *val_report script usage* on page 3-22.

Using the example output from step 1:

```
val_report armv4 -test T1-32 -config '^wdog.wb.icacheon.dcacheon$' -vopt "-I" -s -exec
```

3.8 Validation script usage

The validation script first builds the executable image of the test being run (if applicable), and then launches the simulation. If the output of the simulation is being logged to the screen (the default), messages are echoed to the console during the simulation:

- If the test passes, the message `** TEST PASSED OK **` is printed.
- If the test fails, diagnostic messages precede the message `** TEST FAILED **`.

The usage of the validation script is as follows:

```
validation <options> <testname>
```

Where:

- <options>
- *Simulation options*
 - *Run options* on page 3-20
 - *Build options* on page 3-20
 - *Debug option* on page 3-21.

<testname> The name of the test that might have caused all, multiple, or single validation test suites to fail. If an entry in the `source_list` file has debug option `-bin2bst` next to the <testname>, the test contains debug sequences and you must run it with the `-bin2bst` option on the command line.

Note

Calling `validation` with no arguments or switch options causes the full usage to be displayed.

3.8.1 Simulation options

The validation script can use various simulators. For RTL validation the supported options are:

- `-mti` Selects the MTI ModelSim simulator.
- `-vcs` Selects the VCS simulator.
- `-nc` Selects the NC-Verilog simulator.
- `-xl` Selects the Verilog-XL simulator.

Note

You can use `$DefaultSimulator` in the `validation.cfg` file to specify which simulator is chosen, if no simulator options are supplied on the command line.

3.8.2 Run options

The validation script run options are:

- I Specifies that the simulation is run interactively. You use this to invoke the *Graphical User Interface* (GUI) of the specified or default simulator tool.
- v Specifies that the test runs in batch mode, that is non-interactively.
- log Instructs all console output to be redirected to a log file that, when simulation is complete, is copied to the logs directory in the suite where the test resides. val_report uses these log files to generate RTL validation status reports.
- eis Specifies that an *Executed Instruction Stream* (EIS) is generated as part of the simulation output.

3.8.3 Build options

The validation script build options are:

- gbin Specifies that the pre-built golden binary image for the test is used. This option is used when the binary-only build of the validation suite has been delivered. (This deliverable does not contain the source code for the validation suite and supplies only pre-built binaries.)
- config Specifies additional configs that are applied to the test. You can specify multiple configs using -config for each configuration, or as a dot (.) separated list. See for details *RTL validation suite configurations* on page 3-5.
- list Specifies that an assembler list file is generated as part of the simulation output.
- bigend Compiles and simulate in big-endian format.
- run Specifies additional test initialization functions to call.
- exit Specifies additional test finalization functions to call.
- src Specifies additional source files to compile with. Used in conjunction with the -run and -exit options.
- PD Predefines assembler variables to configure the test.
- [no]list Specifies if an assembler list file is generated during test compilation.

- lobj Specifies that test object files are placed in a local obj directory in the TempRunDir.
- lbin Specifies that test binaries are placed in a local bin directory in the TempRunDir.
- hivecs Loads the test image at 0xFFFF0000. The processor boots from this location (**VINITHI** is 1).
- scatter <options>
 Specifies that a scatter file is created based on the following options:
 - -code <addr> defines a code region at <addr>
 - -data <addr> defines a data region at <addr>
 - -boot <obj> comma separated list of boot objects.
- etm Specifies that the ETM is used and its output trace is verified.
- cd Run in the current directory, that is do not create a TempRunDir.
- [no]clean Specifies if TempRunDir is retained for debug purposes.
- [no]F Specifies if test recompilation is forced.

3.8.4 Debug option

The validation script debug options are:

- bin2bst Specifies that *Boundary Scan Instructions* (BSI) are extracted from the compiled test. You use this option when the test exercises the debug functionality of the core using the *Boundary Scan Testbox* (BST).

———— **Note** ————

val_report automatically adds the -bin2bst option if necessary. However, you can safely use the -bin2bst option for all tests, but a warning is given for tests that do not require it.

—————

3.9 val_report script usage

The `val_report` script parses the log files of the available tests and reports their status. If requested, `val_report` then submits for execution the tests that meet the selected filter options. The usage of the `val_report` script is:

```
val_report [logs|glogs] [<suitename(s)>] [<options>]
```

Where:

`logs` Specifies that the `logs` directories contain the log files to use in the generation of the report.

————— Note —————

`logs` is assumed by default, therefore is not required by `val_report`.

`glogs` Specifies that the `glogs` directories contain the log files to use in the generation of the report. The log files in these directories are the golden log files supplied with the macrocell installation.

`<suitename(s)>`

Limits the report to a particular suite, or suites, listed. If `suitename` is not specified, a report is generated for all suites.

`<options>`

- *Filter options*
- *Job submission options* on page 3-23
- *Output options* on page 3-24.

3.9.1 Filter options

The filter options are:

`-pass` Reports passing tests.

`-npass` Reports tests that have not passed.

`-fail` Reports tests that have failed.

`-abandoned` Reports tests that were abandoned during simulation.

`-notrun` Reports tests that have not been run.

`-compfail` Reports tests that failed to compile.

`-compok` Reports tests that compiled successfully.

`-config <regular expression>`

Reports only those tests that have been run with the matching configuration `<regular expression>`.

`-test <regular expression>`

Reports tests only with names that match the `<regular expression>`.

-xconfig <regular expression>

Reports only those tests that do not match the configuration <regular expression>.

-xtest <regular expression>

Reports tests that do not match the testname <regular expression>.

3.9.2 Job submission options

The job submission options are:

- s Creates jobs scripts for the selected tests and store them in the jobs directory.
- lsf Makes jobs scripts suitable for LSF submission. Overrides \$lsfDefaultQueue, \$lsfshortQueue, and \$lsfQuickQueue parameters in validation.cfg.
- exec Executes each job script in turn. Must be used in conjunction with -s option.
- sub Creates and executes LSF jobs scripts. Same as using -s, -lsf, and -exec
- c Job scripts only compile tests.
- queue <queue>
 Uses <queue> for LSF job submission. Overrides \$lsfDefaultQueue, \$lsfShortQueue, and \$lsfQuickQueue in validation.cfg.
- vopt <options>
 Supplies additional validation command-line options, see *Run options* on page 3-20.

Note

These options must be quoted. For example:

-vopt "-I"

- mti Selects the MTI ModelSim simulator.
- vcs Selects the VCS simulator.
- nc Selects the NC-Verilog simulator.
- xl Selects the Verilog-XL simulator.

Note

\$DefaultSimulator is chosen if a simulator switch is not supplied.

3.9.3 Output options

The output options are:

- full Outputs with full test status information.
- [no]gaudy Specifies that a status report is produced for each suite. -nogaudy is used to list test log filenames for easier identification of test name, suite and config options.
- o [<file>] Outputs report to file or to reports/<project>.rep by default.
- p [<printer>] Prints report to default printer or <printer> if specified.
- m [<recipient>] Mails report to recipients. If no recipient is specified the report is mailed to the user, otherwise multiple -m [<recipient>] options can be used to mail a list of recipients.
- h[e]lp Prints full usage information.

3.10 Script file generation

If you run `val_report` with the `-s` option, it generates script files to run each of the tests reported. `val_report` generates a separate script file for each combination of suite used and configuration used. The script files are located in the `validation/jobs` directory. You configure the `val_report` script using the `val_report.cfg` file that describes the configurations to run, on what suite to run them, and how to construct the script file.

The `val_report.cfg` format is flexible so you can optimize the script files for your particular system. For example, if your system has a compute cluster, you can generate script files that run multiple validation tests concurrently. An example fragment of a `val_report.cfg` file is:

```
val_jobs_header:
#!/bin/tcsh -f
set standard_options = "-lobj -log"
END

armv4_val_cmd=validation $standard_options
armv4_val_configs:
native                -simopts +noeis
wdog.itcmon.randomTCM -simopts +noeis -PD 'WATCHDOG_COUNT SETA 0x04000000'
END
```

The `val_jobs_header` entry is a standard header written to the start of every script file. In the example, the header:

1. Declares that the script is a csh script.
2. Sets an environment variable with some standard validation options to apply.

The next entry in the example is an entry for the `armv4` suite. The `armv4_val_cmd` entry specifies the validation command line used for each test. The `armv4_val_configs` entry specifies the configurations that are applied to the tests. You can specify multiple configurations by using `.` (period) to join configuration names. The `native` configuration is a special designation used to indicate that the test runs without additional configurations.

If you run `val_report` with the example configuration file using:

```
val_report logs armv4 -s
```

Two job files are generated:

- `validation/jobs/armv4.job`
- `validation/jobs/armv4.wdog.itcmon.randomTCM.job`

An example fragment of the script file created by `val_report`, with LSF enabled, from the example configuration file is:

```
jobs/armv4.wdog.itcmon.randomTCM.job

#!/bin/csh -f
set standard_options = "-lobj -log"
bsub -F 100000 -o /dev/null -R "solaris" -q short -J "ARM926EJ.armv4" \
    validation $standard_options -simopts +noeis \
    -PD 'WATCHDOG_COUNT SETA 0x04000000' \
    -config wdog.itcmon.randomTCM -vcs T1-32 -v
bsub -F 100000 -o /dev/null -R "solaris" -q short -J "ARM926EJ.armv4" \
    validation $standard_options -simopts +noeis \
    -PD 'WATCHDOG_COUNT SETA 0x04000000' \
    -config wdog.itcmon.randomTCM -vcs T2-32 -v
```

An example fragment of the script file created by `val_report`, with LSF disabled, from the example configuration file is:

```
jobs/armv4.job

#!/bin/csh -f
set standard_options = "-lobj -log"
validation $standard_options -simopts +noeis -vcs T1-32 -v
validation $standard_options -simopts +noeis -vcs T2-32 -v
```

Chapter 4

Floorplanning

This chapter describes how to create a floorplan for the macrocell. It contains the following sections:

- *About floorplanning* on page 4-2
- *Supported flow* on page 4-3
- *Guidelines* on page 4-4
- *Floorplanning* on page 4-5
- *Placement* on page 4-7
- *Floorplan example* on page 4-8.

4.1 About floorplanning

The macrocell is small enough to be placed and routed flat. Placing the design as a sea of gates means that you can perform timing optimizations on the whole design simultaneously and achieve timing closure quickly.

During floor planning, you must consider:

- routing
- power rails
- clock trees and high fanout nets
- any extra cells placed to meet timing requirements.

4.2 Supported flow

Figure 4-1 shows the supported flow for macrocell floorplanning. The flow takes the netlist output from Design Compiler and produces a floorplan to use as the input for Physical Compiler.

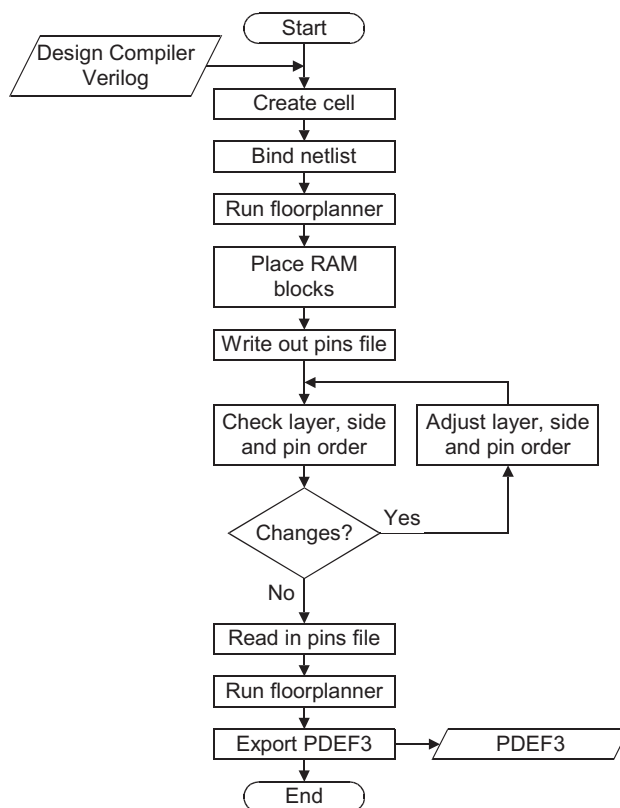


Figure 4-1 Floorplanning flow

4.3 Guidelines

Follow these guidelines when you create the floorplan:

- Make pins at least twice the size of the routing width because this reduces congestion around the pins both inside and outside the design.
- Bunch all pins of the same signal type together. Sorting the pins alphabetically helps to keep the same signal types together.
- Orient the memory cells so that the pins face the preferred routing direction.
- Place pins on the boundary that faces the standard cell row, rather than in positions blocked by RAMs.
- To decide what standard cell utilization to use, balance:
 - small area with low metal fill, to achieve good timing performance
 - large area with more metal fill, to achieve good cross talk performance and lower congestion.
- Keep the standard cell area as square as possible because this gives the shortest timing paths.
- Place memory cells at the edge of the placeable area.
- Keep the distance between the memory cell pins and the standard cell area short.
- Make memory cell stacks shallow to avoid long transition delays.
- Add placement and routing blockage, for example for power rails, to the floorplan before you generate the PDEF3 file.
- Use a power grid because this gives the best power distribution.
- For designs with I/O antenna diodes, add placement blockages next to the pin that each diode connects to. This reserves area for the diode so that the layout is not disturbed by the placement of the I/O antenna cells.
- Use two, coincident, pins for each I/O, where possible, one for the preferred horizontal and one for the preferred vertical layer. This eases rotation of the macrocell during integration.

4.4 Floorplanning

To produce your floorplan:

1. Select a suitable aspect ratio for the floorplan.

The standard cell region of the macrocell has a high number of critical paths that cross the placement area. To minimize routing and maximize performance, the standard cell region must be compact and with an aspect ratio that is not detrimental to routing. A sensible aspect ratio is between 0.6 and 1.4. Adverse aspect ratios can lead to long paths and areas of high congestion.

2. Place the macrocell pins.

Pin placement is very important because it influences the placement of standard cells within the macrocell. Poor pin placement can result in poor I/O timing, poor internal path timing, or both. You can use the following information to determine your pin placement:

- The `verilog/ARM926EJS/Pin_to_Block.txt` file describes which pins connect to which block. Place pins adjacent to the blocks that they are connected to, to minimize pin routing.
- *Floorplan example* on page 4-8 gives an example of pin placement.
- *RAM interface guidelines* on page 4-6 gives recommendations that enable you to maximize the performance of the RAM interfaces.

3. Design to reduce electromigration.

Ideally all RAMs must have a power ring around them with a width that restricts current density to reduce electromigration to within permissible limits. If this is not possible because of area constraints then, at the very least, the power for the RAMs must have a strap located in the placeable area between the RAMs and standard cells. The standard cell straps must not be directly connected to the RAM power rings unless the power rings have been designed to incorporate the extra load of the standard cells.

————— Note —————

For the current version of the PrimeTime tool (2003.03-1), to generate an accurate extracted timing model for synthesis, requires that you must place the I/O cells close to the I/O pins. Using `extract_model` with the `-library_cell` option, use lumped capacitance instead of modelling it with SPEF parasitics. Minimizing the I/O net loading reduces inaccuracy of the generated timing model. This complicates your layout flow but is necessary if you require an accurate extracted timing model. It is possible that this requirement might change for later versions of the tools so you must check against the version of PrimeTime that you are using. You can also minimize the I/O net load by setting synthesis constraints to limit fanout or maximum transition time.

4.4.1 RAM interface guidelines

Paths to and from all of the RAM blocks are critical paths in the macrocell. It is essential that you minimize the routing to and from the RAM blocks.

The following information is applicable to floorplanning the macrocell:

- The ports on the RAM must be as close to the standard cell region as possible.
- If the RAM outputs have a high drive strength, longer routing might be tolerated on the outputs. This influences the orientation of the RAM blocks if the input and output ports are not on the same side of the RAM.
- You must organize the RAM blocks to minimize the routing of common signals, if RAM blocks share common signals.
- The address and chip select are the most critical RAM control signals. Minimizing the routing of these signals must take priority over the other RAM signals (write data and write enable).

4.5 Placement

The macrocell is small enough that you can place and route it flat. A timing driven placement achieves good results, with good pin positioning.

If your layout does not achieve the required performance because of long timing paths, or is unroutable because of areas of high congestion, you might have to revisit your floorplan and:

- move the pins on the macrocell to try and force better positioning of the standard cells within the macrocell
- modify the floorplan aspect ratio.

4.6 Floorplan example

Figure 4-2 shows a floorplan based on the information in *Floorplanning* on page 4-5 and *Placement* on page 4-7

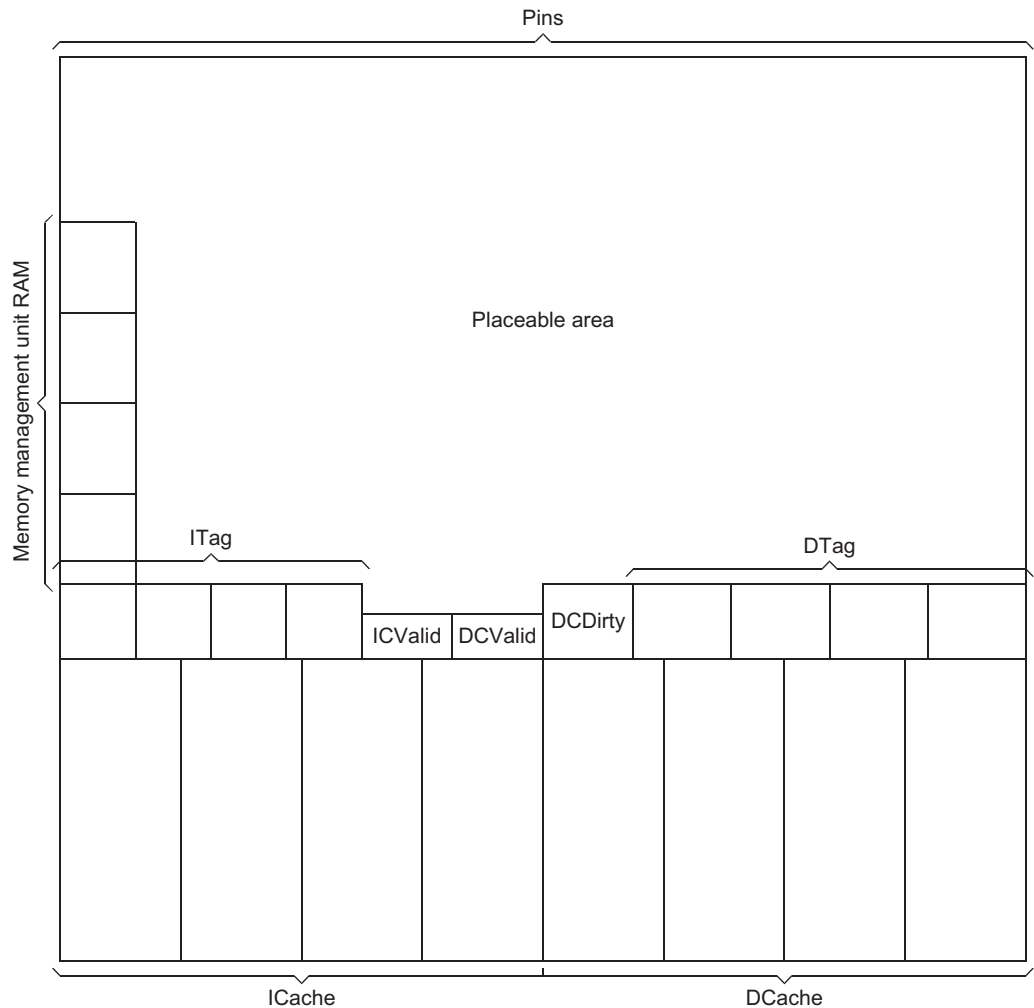


Figure 4-2 Floorplan example

Chapter 5

Synthesis

This chapter describes how to synthesize the macrocell. It contains the following sections:

- *About synthesis* on page 5-2
- *Supported synthesis flow* on page 5-4
- *Synthesis scripts* on page 5-6
- *Setting up synthesis* on page 5-9
- *Running synthesis* on page 5-12
- *Synthesis process* on page 5-15
- *Floorplanning and SoCTest* on page 5-32
- *Reports* on page 5-34.

5.1 About synthesis

This chapter provides information that enables you to create a synthesis flow as quickly as possible. It contains information on the files and scripts supplied, and how they are used in the synthesis flow. You can examine the scripts if you require more detailed information about how they work.

Note

There is no restriction if you want to:

- modify the scripts
 - update the scripts for any synthesis enhancements not available when the scripts were released.
-

The synthesis strategy is:

- to produce a complete netlist in a target technology that is functionally equivalent to the RTL description
- to satisfy all performance, area, and power goals imposed.

This section provides information on:

- *Synthesis directory structure*
- *Synopsys tool requirements* on page 5-3.

5.1.1 Synthesis directory structure

Figure 5-1 shows the synthesis directory structure.

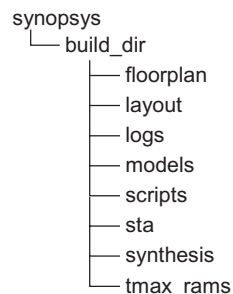


Figure 5-1 Synthesis directory structure

The purpose of each subdirectory shown in Figure 5-1 on page 5-2 is:

<code>floorplan</code>	Contains the floorplan for physical synthesis.
<code>layout</code>	Contains the layout data from third-party physical design tools.
<code>logs</code>	Contains the transcripts and log files for all Synopsys tools used with the macrocell.
<code>models</code>	Contains test models for use with TetraMax and is where ATPG patterns are written. Also contains the extracted timing model created by the STA script.
<code>scripts</code>	Contains the Synopsys tool scripts for use with the macrocell.
<code>sta</code>	Contains reports created as part of the PrimeTime flow.
<code>synthesis</code>	Contains the data generated during synthesis.
<code>tmax_rams</code>	Contains RAM models used by TetraMax.

5.1.2 Synopsys tool requirements

The methodology and scripts have been developed using the 2003.03-1 production release. The methodology is portable across all operating systems supported by the release. If you use the Solaris platform you might have to use the 64-bit synthesis release because synthesis of the macrocell requires significant memory resource.

The following licenses are required to implement the macrocell:

- HDL-Compiler for Verilog
- DC-Ultra
- Design-Compiler
- DesignWare-Foundation
- DFT-Compiler or Test-Compiler
- Power-Compiler
- PhysOpt
- SoCTest.

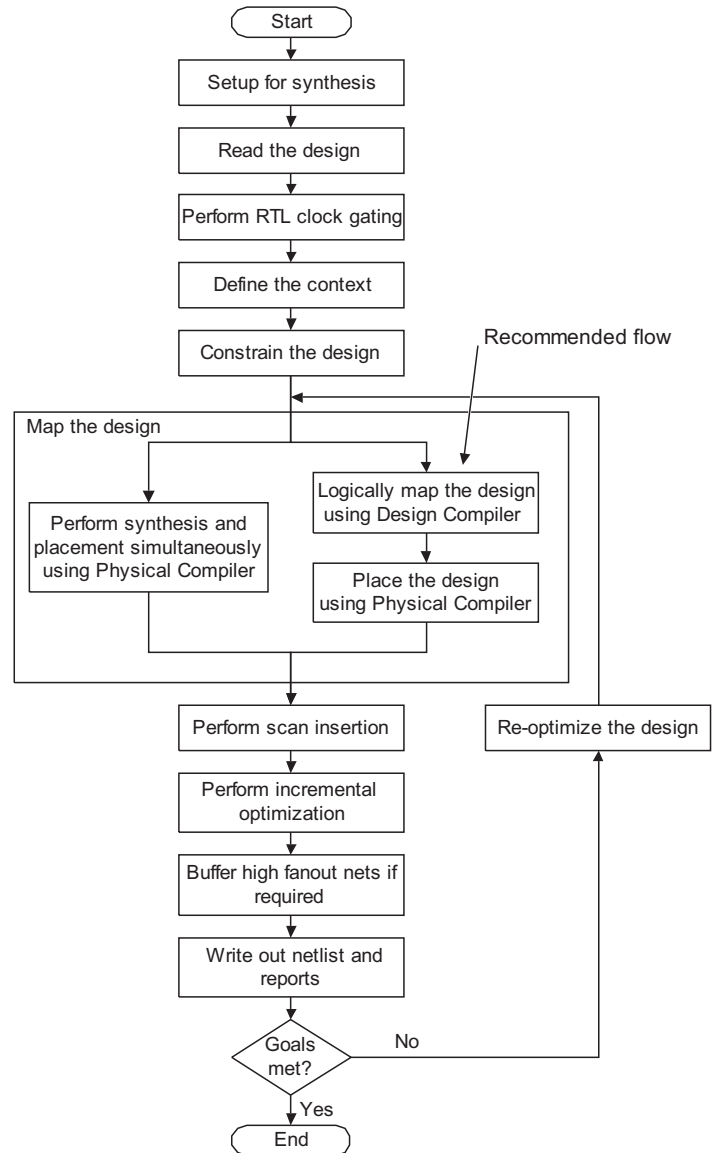
5.2 Supported synthesis flow

This section provides information on the macrocell synthesis flow. The synthesis flow enables you to:

1. Set up the environment for Design Compiler and Physical Compiler.
2. Control the analysis and elaboration of the macrocell.
3. Identify the operating environment for the macrocell including port characteristics, operating conditions, and interconnect models.
4. Establish specific performance goals for the design. Define clocks and interface timing. Define all required design rules that the resulting implementation must satisfy.
5. Map the design using:
 - Design Compiler for RTL to gates. Place the design using Physical Compiler. This is the recommended method of mapping the design because it provides higher performance.
 - Physical Compiler for RTL-to-placed-gates.
6. If you use Design Compiler to map the design, use Physical Compiler to place it.
7. Insert scan chain(s) into the macrocell using physical information to order the scan chain elements optimally.
8. Perform post synthesis analysis for timing, design rules, testability, and *Quality of Results* (QoR). Save the design and all associated derivative files including constraints (*Synopsys Design Constraints* (SDC)) and testability information.

Figure 5-2 on page 5-5 shows the macrocell synthesis flow.

You can use a logical synthesis flow using Design Compiler to create a netlist and an SDC file as inputs to the floorplanner. The floorplan must identify the location of primary inputs and outputs, the placement of RAM blocks, specific blockages, and possibly power planning details. See Chapter 4 *Floorplanning*. Use a physical synthesis flow, using Physical Compiler on this floorplan, to perform either RTL-to-placed-gates, or gates-to-placed-gates synthesis.

**Figure 5-2 Supported synthesis flow**

5.3 Synthesis scripts

Support for macrocell synthesis is provided by .tcl scripts:

- *arm926ejs_verilog.tcl*
- *arm926ejs_compile.tcl*
- *arm926ejs_physopt.tcl* on page 5-7
- *arm926ejs_constraints.tcl* on page 5-7
- *arm926ejs_proc.tcl* on page 5-7
- *arm926ejs_config.tcl* on page 5-7
- *options* on page 5-8
- *arm926ejs_tsmc18.tcl* on page 5-8.

All the files are located in your installed `./build_dir/scripts` directory except the options file, which is in the `./synopsys/build_dir` directory. Figure 5-3 shows the association of the synthesis scripts.

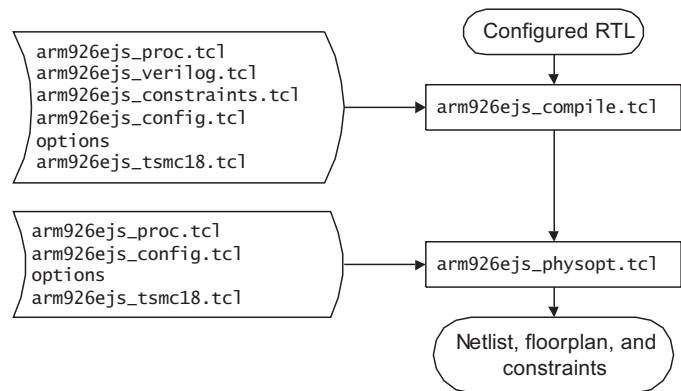


Figure 5-3 Synthesis script association

A Makefile in the `synopsys/build_dir` directory controls the synthesis flow, see *Makefile* on page 5-12.

5.3.1 arm926ejs_verilog.tcl

This script identifies the names and locations of the components of the macrocell RTL.

5.3.2 arm926ejs_compile.tcl

This is the primary compile script for the macrocell. This script manages all aspects of implementing the design in a target technology and manages the process from RTL to gates or placed-gates. A floorplan of the macrocell in PDEF format is required if you use Physical Compiler. This script is technology independent.

5.3.3 arm926ejs_physopt.tcl

To use this script, the database produced by `arm926ejs_compile.tcl` is required.

If you have used an RTL-to-placed-gates flow, only scan insertion occurs in this script. If you have used a gates-to-placed-gates flow, Physopt places the design before scan insertion.

For scan insertion, DFT Compiler, integrated within Physical Compiler, constructs and optimizes the scan chains within the design. For details of scan chains, see *Scan chain stitching* on page 5-28 and *Scan chain stitching with SoCTest* on page 5-28.

Finally, Physopt exports the following industry-standard files for detailed routing in tools such as Synopsys Astro or Cadence Silicon Ensemble:

- Verilog netlist
- floorplan in *Physical Design Exchange Format* (PDEF)
- *Synopsys Design Constraints* (SDC).

5.3.4 arm926ejs_constraints.tcl

This script gives I/O default constraints. You can modify these during implementation to suit the target design context. You must ensure that any modifications to the timing constraints are consistent with the timing budget for the target design.

5.3.5 arm926ejs_proc.tcl

This script holds a number of tcl procedures that the `compile` and `physopt` scripts call. These procedures include:

- writing out an SDC file for floorplanning and layout tools to use
- generating the test ports.

5.3.6 arm926ejs_config.tcl

This script:

- sources the options file
- validates the options
- sources the technology script.

Both the `compile` and `physopt` scripts use the `config` script.

5.3.7 options

The options file located in your installed synopsys/build_dir directory enables you to control the synthesis flow and set technology-specific options. For details on the options and how to use them see *Setting up the control and technology-specific options* on page 5-10. The options script is sourced by the config script.

5.3.8 arm926ejs_tsmc18.tcl

This is an example technology script that holds the paths to the target libraries, link libraries, and synthetic libraries. The script is sourced by the config script. This is required by Design Compiler and Physical Compiler during synthesis of the macrocell to target the TSMC18G process. You can also include a list of cells that you do not want to use in synthesis in the script. For example, any cells with poor timing characteristics. For details see *Setting up the technology libraries* on page 5-9.

5.4 Setting up synthesis

You must set up the technology libraries and synthesis options to your requirements before running the synthesis process.

5.4.1 Setting up the technology libraries

You must create a script for your target technology library. An example script is included in the `synopsys/build_dir/scripts` directory called `arm926ejs_tsmc18.tcl`. You can copy this script and save it to a new name, for example `arm926ejs_<rm_technology>.tcl`, where `<rm_technology>` is defined in the options script.

The following code in `arm926ejs_<rm_technology>.tcl` assumes that each RAM lives in a sub-directory of the search path:

```
foreach ram [list RA1SH_1024x32Bwm4 RA1SH_128x22 RA1SH_128x8bw RA1SH_256x22
RA1SH_32x24 RA1SH_32x26 RA1SH_32x30 ] {
    set search_path [concat $search_path
/chips/arm926ej-s/artisan/sp_ram_generator_2002q2v0_output/${ram} ]
}
```

If your RAMs exist in the same directory, remove the foreach loop to set up a single search path.

To set up your `arm926ejs_<rm_technology>.tcl` script to represent your target library you must change the parameters listed in Table 5-1.

Table 5-1 Synthesis technology parameters

Parameter	Description
<code>synthetic_library</code>	The technology library that stores the DesignWare libraries.
<code>target_library</code>	The technology library used to build the macrocell
<code>symbol_library</code>	The database containing the schematic symbols to be used by Design Analyzer
<code>physical_library</code>	The physical libraries used by Physical Compiler
<code>link_path</code>	Path to the technology libraries used during the linking operation
<code>search_path</code>	Search path to the directories where the standard cell, memories, and DesignWare libraries are located
<code>rm_loading_pin</code>	The pin of the standard cell used to load the outputs of the macrocell
<code>rm_driving_cell</code>	Name of the standard cell that is used for driving the macrocell inputs
<code>rm_driving_pin</code>	Name of the output pin for <code>rm_driving_cell</code>
<code>rm_max_transition</code>	The maximum transition time for the design. This is technology dependent.

Note

It is recommended that you use the naming conventions `{rm_operating_condition}.db` and `{RAM}_{rm_operating_condition}` for the standard cell and RAM libraries. This enables the `rm_operating_condition` switch to be set in the options script to speed up the library switching process for STA.

5.4.2 Setting up the control and technology-specific options

The options file provides overall control of the synthesis flow and is located in your installed `synopsys/build_dir` directory. Table 5-2 gives a description of synthesis options and their default settings. You can modify these options to control the flow as appropriate.

Table 5-2 Options

Options	Default settings	Description
<code>rm_technology</code>	<code>tsmc18</code>	The technology. The technology file must have the same suffix as this setting. For example <code>arm926ejs_tsmc18.tcl</code> .
<code>rm_vhdl_rtl</code>	<code>0</code>	When set, VHDL RTL is used.
<code>rm_arch_clk_gating</code>	<code>0</code>	For VHDL only. For more information on this option contact ARM Limited
<code>rm_operating_condition</code>	<code>slow</code>	The process, voltage, and temperature corner that the design is synthesized to. It is strongly recommended that you synthesize the design at the worst case corner.
<code>rm_rtl2pg</code>	<code>0</code>	When set, this variable instructs Physical Compiler to operate in the RTL-to-placed-gates mode using the <code>compile_physical</code> command.
<code>rm_num_scan_chains</code>	<code>4</code>	The number of scan chains in the design.
<code>rm_clock_gating</code>	<code>1</code>	When set, this variable instructs Design Compiler or Physical Compiler to perform RTL clock-gating on the design.
<code>rm_ungroup</code>	<code>0</code>	When set, the design is completely ungrouped and flattened.
<code>rm_num_cpu</code>	<code>1</code>	The number of CPUs that Physical Compiler uses to perform coarse placement. Each CPU uses a Physical Compiler licence.
<code>rm_create_buffer_tree</code>	<code>1</code>	When set, all high fanout nets (not clocks) are replaced with buffer trees.
<code>rm_compile_ultra</code>	<code>1</code>	When set, Design Compiler synthesizes using the <code>compile_ultra</code> switch.
<code>rm_use_presto</code>	<code>1</code>	When set, Presto HDL compiler is used.

Table 5-2 Options (continued)

Options	Default settings	Description
rm_create_test_model	0	When set, this variable uses Synopsys SoCTest to create a test wrapper and associated test model.
rm_num_wrapper_chains	2	The number of wrappers that SoCTest inserts.
rm_wrap_in	WRP_SI	The name of the wrapper input.
rm_wrap_out	WRP_SO	The name of the wrapper output.
rm_use_rpp	0	When set, the <i>RTL Performance Prototyping</i> (RPP) option in Physical Compiler is used for design exploration.
rm_rpp_util	0.80	The target floorplan utilization set to 80% for RPP design exploration.
rm_rpp_ratio	0.66	The target floorplan aspect ratio set to 0.66 for RPP design exploration.
rm_clock_latency	2.0	Expected clock insertion delay. Set this to what you expect the clock tree insertion to be.
rm_clock_period	4.0	The clock period used for synthesis.
rm_ahb_clock_period	4.0	The AHB clock period used for synthesis.
rm_io_clock_period	4.0	The clock period for all I/O ports (except AHB) of the design.
rm_icg_name	{integrated: TLATNTSCAX8}	The name of the integrated clock-gating cell in the library. You must ensure that it has sufficient drive strength to drive up to 32 registers. If integrated is not used then you must specify {}.
rm_clock_uncertainty	0.2	The arrival time uncertainty of the clock at the registers within the design. This is technology dependent and you can set this to what you expect the clock skew to be for your technology.
rm_icg_latency	0.4	Expected integrated clock gate insertion delay minus the propagation delay through a clock gate.
rm_critical_range	0.1	The time range that synthesis optimizes over. This is technology dependent. Set an appropriate critical range to improve the overall QoR.
rm_scan_in	SI	The name of the scan-in port.
rm_scan_out	SO	The name of the scan-out port.
rm_enable_si	0	When set, enables crosstalk analysis in PrimeTime-SI.
rm_extract_model	1	When set, enables Timing Model Extraction.
rm_fm_postlayout	0	When set, reads the placed-and-routed netlist for equivalence checking.

5.5 Running synthesis

Before running synthesis you must ensure that:

- RAM integration is complete. See Chapter 2 *RAM Integration*.
- The options and technology scripts are set up to your requirements.

A summary of the type of operation, the floorplan and library design flow requirements, and the `rm_rt12pg` and `rm_use_rpp` settings in the options file is shown in Table 5-3. For details of setting up other options and technology scripts see *Setting up synthesis* on page 5-9.

Table 5-3 Summary of operation, option settings and design flow requirements

Operation	Option setting		Design flow requirements		Comment
	<code>rm_rt12pg</code>	<code>rm_use_rpp</code>	Physical library data	Floorplan	
compile	0	0	No	No	Logical compile only. Default settings.
compile	x	1	Yes	No	RPP only. Used for design exploration.
compile	1	0	Yes	Yes	RTL to placed gates
physopt	0	0	Yes	Yes	For placement, scan insertion, high fanout net synthesis. Default settings.
physopt	1	0	Yes	Yes	Scan insertion, high fanout net synthesis.

———— **Note** ————

To improve the QoR the `rm_compile_ultra` option default setting is 1. This enables you to perform a two-pass high-effort compile, that uses delay driven auto-ungrouping. A DC-Ultra license is required.

5.5.1 Makefile

The synthesis flow is controlled by a Makefile in the `synopsys/build_dir` directory. You can start all stages of the flow by typing `make` followed by the target. For example, you can launch the compile stage with the command:

```
make compile
```

For a full list of targets type:

```
make help
```

Before you use the Makefile for the first time, you must make two modifications to it:

- setup the installation details for each tool
- add the paths to your technology files.

The meanings of the Makefile targets are:

compile	Run the arm926ejs_compile.tcl script.
physopt	Run the arm926ejs_physopt.tcl script.
prep	Reformat the PDEF3 into an .scm file that Astro can read and parse the SDC file to remove unsupported commands for Astro.
import	Run the arm926ejs_import.scm script.
cts	Run the arm926ejs_cts.scm script.
route	Run the arm926ejs_route.scm script.
opt	Run the arm926ejs_opt.scm script.
export	Run the arm926ejs_export.scm script.
fm	Run the arm926ejs_fm.tcl script.
tme	Run the arm926ejs_sta.tcl and arm926ejs_tmequal.tcl scripts.
tmax	Run the arm926ejs_tmax.tcl script.
help	List the targets available in the Makefile.
clean	Clean the implementation directory.
front	Run the front-end commands from the Makefile: • compile • physopt.
back	Run the back-end commands from the Makefile: • prep • import • cts • route • opt • export.

models	Run the tme commands for the Makefile: • tme.
all	Run the front-end, back-end, and models commands from the Makefile: • front • back • models.

5.5.2 Synthesis procedure

To run synthesis:

1. Navigate to your installed synopsys directory:

```
cd synopsys/build_dir
```

This is where the options and Makefile files are located and the make commands are designed to be run from.
2. At the prompt type the command:

```
make compile
```

make compile synthesizes a scan-ready design as either a logical compile only using DC-Expert/DC-Ultra, or as placed gates using Physical Compiler.
3. At the prompt type the command:

```
make physopt
```

make physopt reads in the ARM926EJS-compile.db design produced by make compile and uses Physical Compiler to perform:

 - placement (if necessary)
 - scan chain insertion
 - wrapper chain insertion
 - high-fanout net synthesis (excluding the clock tree).

This creates a fully scan-inserted, and wrapped, placed-gates design.

————— **Note** —————

You must edit the Makefile so that it gives the correct paths to your installation.

5.6 Synthesis process

This section provides detailed information on the implementation flow in the supplied synthesis scripts:

- *Reading the design*
- *Constraining the design* on page 5-20
- *Defining the design context* on page 5-21
- *Compiling the design* on page 5-24
- *Scan and test wrapper insertion* on page 5-27
- *High fanout net buffering* on page 5-30
- *Saving the design* on page 5-30

See Synopsys documentation if you require detailed information on the switches used in the implementation scripts.

5.6.1 Reading the design

The macrocell is supplied in Verilog RTL format that is the starting point in creating the gate-level macrocell. This section provides information on:

- *RTL analysis*
- *Clock-gating with Power Compiler* on page 5-16
- *Elaboration* on page 5-17
- *Uniquifying the hierarchy* on page 5-17
- *Test-ready compile* on page 5-18.

RTL analysis

You must set the RTL analysis control variables before you can process the RTL at the elaboration stage of the synthesis flow.

The RTL is parameterized to support various configurations of the macrocell. You can manage parameterization using directives, including 'define and 'ifdef. To process an HDL description with these directives, set the Verilog preprocessor within HDL Compiler in the arm926ejs_compile.tcl script:

```
set hdlin_enable_vpp true
```

The macrocell uses only edge-triggered flip-flops to implement sequenced standard cell elements. Latches are not to be inferred in the design except those that are required to implement architectural clock gating:

```
set hdlin_check_no_latch true
```

Note

If you implement clock-gating with Power Compiler, and there is not an integrated cell in the target library, then latches are intentionally used in the clock-gating structure and are not flagged by this variable.

You can identify the design library location for the intermediate files with the variable `define_design_lib`, for example:

```
define_design_lib work -path ./synthesis/work
```

Use the `analyze` command to translate the Verilog, or VHDL, RTL into an intermediate format. The variable `$rtl_image` identifies and orders all the RTL components.

If you use VHDL, rather than Verilog, set the variable `rm_vhdl_rtl` in the options file. The compile script uses an `if` statement to switch between Verilog and VHDL:

```
if { $rm_vhdl_rtl } {
    source -echo -verbose ./scripts/arm926ejs_vhdl.tcl
    analyze -f vhdl $rtl_image
} else {
    source -echo -verbose ./scripts/arm926ejs_verilog.tcl
    analyze -f verilog $rtl_image
}
```

Clock-gating with Power Compiler

When the `rm_clock_gating` variable is set to 1 in the options script, Power Compiler automatically integrates clock-gating logic, ensuring that register banks are only clocked when necessary.

Use the `set_clock_gating_style` command, defined in the `arm926ejs_compile.tcl` script, to specify four primary aspects of clock-gating to Power Compiler:

- the conditions necessary to implement clock-gating
- the nature of the clock-gating circuitry
- the additional circuitry required to improve testability
- the maximum fanout of the clock-gating circuitry.

By default, the `arm926ejs_compile.tcl` script is set up to use an integrated clock-gating cell. An integrated clock-gating cell reduces the probability of glitches and simplifies the implementation flow.

Propagating constraints

The setup and hold constraints on clock-gating structures propagate up through the hierarchy of the current design, for example:

```
if { $rm_clock_gating } {
    propagate_constraints -gate_clock -output ./synthesis/reports/ARM926EJS.propagate_constraints
}
```

The `-gate_clock` switch forces all setup and hold constraints set on the clock-gating elements at lower levels in the design hierarchy up to the top-level.

Elaboration

The `elaborate` command elaborates the design. The `elaborate` command behaves differently depending on:

- if you use Verilog or VHDL
- if you use Power Compiler to provide clock gating or not.

For example:

```
if { $rm_vhdl_rtl } {
    if { $rm_clock_gating } {
        elaborate -architecture "synth" -gate_clock ARM926EJS
    } else {
        elaborate -architecture "synth" ARM926EJS
    }
} else {
    if { $rm_clock_gating } {
        elaborate -architecture verilog -gate_clock ARM926EJS
    } else {
        elaborate -architecture verilog ARM926EJS
    }
}
```

If clock-gating is required you must define the style of clock-gating before executing the `elaborate` command. See *Clock-gating with Power Compiler* on page 5-16.

Uniquifying the hierarchy

The macrocell contains multiple instantiated blocks. The compiler uniquifies these before linking the design. In a top-down approach the compiler compiles each of these unique instantiations of the design individually based on the context where it is located. The design is then linked and checked, for example:

```
current_design ARM926EJS
uniquify
link
```

Test-ready compile

Use *Design For Test* (DFT) Compiler to combine the logic and test synthesis phases of the design flow into a single test-ready compile step. Test-ready compile integrates logic optimization and scan replacement.

If your library contains scannable flip-flops, test ready compile inserts scannable flip-flops into the design during compilation instead of normal flip-flops. The scan enable control is tied off and the output of the flip-flop is connected to the scan input.

If your library does not contain scannable flip-flops, test ready compile implements D-type flip-flops with synthesized scan logic to produce the required functionality, tying it off as for the scannable flip-flops. The optimization cost function considers the impact of the scan cells, and the additional loading due to scan cell routing.

You must define the following before initiating a test-ready compile:

- scan methodology and style
- all test-specific ports.

You define these in `arm926ejs_compile.tcl`. For example:

```
set_scan_configuration -methodology full_scan -style multiplexed_flip_flop
-methodology All sequential cells are replaced with their scan equivalent unless they
              violate scan design rules.
-style       The multiplexed flip-flop scan style is used by compile. The multiplexed
              flip-flop scan style uses a multiplexed data input to provide serial shift
              capability. The scan enable signal, that is the select to the multiplexor,
              controls the selection of functional mode or scan shift mode.
```

You must identify the scan enable port on the design in `arm926ejs_compile.tcl`:

```
set_scan_signal test_scan_enable -port SCANENABLE
```

Apply any attributes to primary ports in the design that have to be held at constant values during scan shift, for example:

```
set_test_hold 1 TESTMODE
```

The test ports for the macrocell are not defined in the RTL but are added during synthesis. The `tcl` procedure `rm_create_test_ports` controls the creation of the wrapper and scan ports:

```
if { $rm_create_test_model } {
  create_port WRP_CLOCK -direction in
  create_port WRP_SHIFT -direction in
  create_port WRP_IF -direction in
  create_port WRP_OF -direction in
```



```

create_port WRP_SAFE -direction in

if { $rm_num_wrapper_chains > 1 } {

    for { set i 1 } { $i <= $rm_num_wrapper_chains } { incr i } {
        set port_name [ format "%s%s" ${rm_wrap_in} $i ]
        create_port $port_name -direction in
        set port_name [ format "%s%s" ${rm_wrap_out} $i ]
        create_port $port_name -direction out
    }

} else {

    create_port $rm_wrap_in -direction in
    create_port $rm_wrap_out -direction out
}

}

for { set i 1 } { $i <= $rm_num_scan_chains } { incr i } {
    set port_name [ format "%s%s" ${rm_scan_in} $i ]
    create_port $port_name -direction in
    set_scan_signal test_scan_in -port $port_name
    set port_name [ format "%s%s" ${rm_scan_out} $i ]
    create_port $port_name -direction out
    set_scan_signal test_scan_out -port $port_name
}

```

To implement a successful scan strategy the design has to meet all the required scan design rules. It is recommended that you analyze the design against the scan design rules early in the synthesis flow. You can use a fast map-only compile followed by a scan *Design Rule Check* (DRC) to identify scan design violations and correct them before the full constraint driven compile.

Executing a test-ready compile instructs Design Compiler and/or Physical Compiler to take into consideration the impact of scan insertion on the constraints in the design. DFT Compiler replaces every sequential cell with a scan equivalent during optimization.

If scan design rules are violated by the scan cell, then the cell is switched back to a non-scan equivalent later in the test synthesis process. A full analysis of the design for test rule violations, using `check_dft`, is performed after the test-ready compile and before the scan insertion process (see *Design Rule Check* on page 5-28).

————— Note —————

Because the design no longer uses the `INTEST` and `EXTEST` wrapper, the signals **INTEST** and **EXTEST** are tied to zero by the `set_logic_zero` command in the `arm926ejs_compile.tcl` script.

5.6.2 Constraining the design

Before you can compile, you must apply both design rule and optimization constraints. This section contains information on:

- *Clock*
- *Clock-gating constraints*
- *Port timing constraints* on page 5-21
- *Design rule constraints* on page 5-21.

Clock

CLK is the only clock input into the processor.

By default, the clocks are defined to have:

- a 50% duty cycle
- a 4.0ns clock period, set up in the options script
- a clock uncertainty of 0.20ns, set up in the options script
- a clock transition time of 0.

For example, in the `arm926ejs_compile.tcl` script:

```
create_clock -name CLK -period $rm_clock_period [ get_ports {CLK} ]
    set_clock_uncertainty $rm_clock_uncertainty [ get_clocks {CLK} ]
    set_clock_transition 0 [ get_clocks {CLK} ]
    set_clock_latency $rm_clock_latency [ get_clocks {CLK} ]
```

In addition, the script creates a duplicate of this clock to constrain the I/O. This clock is a virtual clock named **VCLK**:

```
create_clock -name VCLK -period $rm_clock_period
    set_clock_uncertainty $rm_clock_uncertainty [ get_clocks {VCLK} ]
    set_clock_latency -min [expr $rm_clock_latency - $rm_clock_uncertainty] [get_clocks {VCLK} ]
    set_clock_latency -max $rm_clock_latency [get_clocks {VCLK} ]
```

Clock-gating constraints

A clock latency value is specified that represents the delay to the clock ports of the integrated clock-gating cell. This takes into consideration the potential delay between an integrated clock-gating cell and the destination registers:

```
if { $rm_clock_gating } {
    set latches [all_registers -clock CLK -level -clock_pins]
    set cg_latches [filter_collection $latches "full_name =~ */clk_gate_*/latch/*"]
    set_clock_latency $rm_icg_latency $cg_latches
}
```

If you specify and apply this latency in conjunction with specifying an overall clock latency, you constrain the enable signal to the integrated clock-gating cell. This prevents the clock gate enable signals from becoming a critical path after CTS.

Port timing constraints

The `set_input_delay` and `set_output_delay` commands place timing constraints on the input and output ports of the macrocell. You apply these from the `arm926ejs_constraints.tcl` file that is included by the `arm926ejs_compile.tcl` script.

You specify the input and output timing for the macrocell as a function of the target clock frequency.

The `set_input_delay` and `set_output_delay` commands take the form of an expression. For example, if the maximum output timing constraint has an external output delay of 40% of the `$rm_clk_period`, the expression is:

```
set cycle40      [expr ${rm_io_clock_period} - ((1 - 0.40) * ${rm_clock_period})]
```

For more information on the input and output port parameters see the *ARM926EJ-S Technical Reference Manual*.

Design rule constraints

No specific restrictions on maximum fanout are included in the scripts for the macrocell. The exception to this is for clock gates, where the fanout of the clock-gating cells is constrained to be no greater than 32. Technology-specific constraints are described in Table 5-2 on page 5-10.

5.6.3 Defining the design context

Because the implementation of the macrocell produces a hardened macrocell, you cannot normally:

- define accurate input drive characteristics and output load characteristics for all technologies
- adjust the size of the boundary cells to accommodate the context of the placed macrocell.

Therefore, you must define the context that the macrocell is to operate in. This section provides information on:

- *Clock and high fanout nets* on page 5-22
- *Port characteristics* on page 5-22.

Clock and high fanout nets

As part of the macrocell synthesis, you define clock and high fan-out nets using the `set_ideal_net` variable:

```
set_ideal_net [list HRESETn DBGnTRST DBGTKEN SCANENABLE]
```

The drivers on these nets have infinite drive and their edge rates do not degrade along the length of the net.

Unspecified nets in the design have their characteristics derived from one of:

- driving cell that forms part of the design
- driving cell applied to the input ports
- capacitive load applied to the output ports.

Port characteristics

You must make a realistic estimation of the range of operating contexts for the macrocell in the target SoC. You must adjust the `arm926ejs_<rm_technology>.tcl` and `arm926ejs_compile.tcl` scripts to the appropriate cell and pin names for the particular technology being targeted.

The primary input ports, excluding clocks, of the macrocell are driven by buffers. You define the buffer in the `arm926ejs_tsmc18.tcl` script, for example:

```
set rm_driving_cell BUF4
set rm_driving_pin y
```

You apply the drive characteristics of the selected cell to each input port of the design, using the `set_driving_cell` command in the `arm926ejs_compile.tcl` script, for example:

```
set_driving_cell -cell $rm_driving_cell -pin $rm_driving_pin $inputs
```

The output ports of the macrocell have a capacitive load of four times BUF4. The `arm926ejs_tsmc18.tcl` script defines the load, for example:

```
set rm_loading_pin slow_cg/BUF4/A
```

The `set_load` command in the `arm926ejs_compile.tcl` script applies the load characteristics of the selected cell to each output port of the design, for example:

```
set_load [expr [load_of $rm_loading_pin ] * 4 ] $outputs
```

Interconnect modeling

If you use a gates-to-placed-gates flow, then Design Compiler performs an initial mapping to gates based on a statistical representation of the interconnect called a wire load model. The choice of wire load model for this initial synthesis is important because an over-buffered design is harder to optimize in PhysOpt.

The Reference Methodology enables automatic wire load model selection during synthesis and specifies a segmented wire load mode. This approach has provided the best results with the specific combination of technology and library targeted:

```
set auto_wire_load_selection true
set_wire_load_mode segmented
```

The `auto_wire_load_selection` variable is set to `true` so that Design Compiler can automatically pick the appropriate wire load model from the technology library for the design being compiled. If a particular wire load model is required, then you must set this variable to `false` and select the wire load model explicitly by name:

```
set auto_wire_load_selection false
set_wire_load_model -name <the_chosen_one> -library slow
```

The approach you take is a technology and library-dependent decision. The default approach is to use automatic wire load model selection and a segmented wire load model.

Operating conditions

The synthesis of the macrocell must take place with worst-case operating conditions for:

- process
- temperature
- voltage.

It is recommended that:

- You perform hold-fixing. You can use Physical Compiler post-route optimization or use the layout tool.
- You use the slow library in synthesizing the macrocell.

5.6.4 Compiling the design

The compile strategy does not require extensive control. The recommended compile strategy for the macrocell is a top-down test-ready compile, because better results are obtained when the synthesis tool can optimize all timing paths within the design. During design evaluations the best results for compiling the macrocell are produced when:

- the `rm_rt12pg` and `rm_rpp` options are set to 0
- the `rm_compile_ultra` option is set to 1.

Note

The implementation of the macrocell is technology-dependent.

This section provides information on specific technologies and libraries, and improvements to the *Quality of Result* (QoR) of the design:

- *DesignWare*
- *DC Ultra optimization* on page 5-25
- *Logical compile* on page 5-25
- *Physical compile* on page 5-25.

DesignWare

DesignWare is used directly in the implementation of the processor core. HDL Compiler can also directly infer DesignWare elements automatically from the RTL description. The selection of implementations of the DesignWare components is based on the:

- constraints applied to the design
- nature of the inference within the code.

Where the DesignWare component is inferred from the RTL, compile can select several possible implementations based on the constraints and on the value of certain switches using the `set_resource_implementation` command. The default setting is:

```
set_resource_implementation use_fastest
```

The `set_resource_implementation` command directs the compiler to select implementations of DesignWare components to meet the performance targets for the processor. It is recommended that you use the `set_resource_implementation` command where high-speed implementation of the processor is required.

If you are creating an implementation of the processor for a SoC application where the performance of the core is not critical, then you can use the `constraint_driven` command.

DC Ultra optimization

If you target the macrocell at a high-performance application, where the maximum operating frequency is a critical parameter, access the Ultra Optimization algorithms in Design Compiler and Physical Compiler using the command:

```
set_ultra_optimization true -force
```

————— Note —————

To use DC-Ultra optimization, you must have a Design Compiler-Ultra license.

Some libraries might run faster if you add the `compile_new_optimization` to the `arm926ejs_compile.tcl` script. This variable is not part of the Reference Methodology.

Logical compile

Use a logical compile:

- when a fast map for the design is required, for example, as an input to a floorplanner
- where there is little or no physical information about the design.

To perform the logical compile step using Design Compiler, you can use one of two commands:

<code>compile</code>	For all Synopsys synthesis releases: <code>compile -map_effort high -area_effort medium -scan</code>
<code>compile_ultra</code>	Combines all logical optimizations: <code>compile_ultra -scan</code>

Physical compile

To achieve a legally placed gate-level design, Physical Compiler simultaneously performs:

- logic-level synthesis
- optimization
- gate-level placement.

You can use Physical Compiler in three modes:

- RTL-to-placed-gates by setting `rm_rt12pg` to 1
- gates-to-placed-gates by setting `rm_rt12pg` to 0
- RPP by setting `rm_use_rpp` to 1.

RTL-to-placed-gates

In this mode the source RTL maps directly to placed gates using a floorplan in *Physical Design Exchange Format* (PDEF) version 3.0 format. To do RTL-to-placed-gates synthesis, use the `compile_physical` command in the `arm926ejs_compile.tcl` script:

```
compile_physical -timing_driven_congestion -scan -map_effort high -area recovery
```

Gates-to-placed-gates

In this mode you place the gate-level logical design, created using Design Compiler, using a floorplan supplied in PDEF version 3.0 format. Use the `physopt` command in the `arm926ejs_physopt.tcl` script to perform gates-to-placed-gates synthesis:

```
physopt -timing_driven_congestion -area_recovery -effort high -num_cpus $rm_num_cpu
```

RTL Performance Prototyping (RPP)

In this mode the RTL is mapped directly to placed gates without a supplied floorplan. In the 2003.03 synthesis release and later this mode performs memory and I/O port placement as well as gate-level placement. Use this mode only to explore the design and to develop a floorplan for the RTL-to-placed-gates or the gates-to-placed-gates synthesis modes. To perform RPP use the `-mpc` switch with the `compile_physical` command in the `arm926ejs_compile.tcl` script:

```
compile_physical -scan -mpc
```

For the RTL-to-placed gates and gates-to-placed-gates synthesis modes a floorplan must have the following attributes:

- the location of the primary input and output ports, including any interface ports added for scan
- placements of the RAM blocks
- specific blockages
- power planning details.

Use the command `read_pdef` to read the floorplan information into Physical Compiler after the design is read in and linked. For example:

```
read_pdef -allow_physical_ports ./floorplan/data/ARM926EJS.pdef
```


5.6.5 Scan and test wrapper insertion

Scan insertion for the macrocell uses DFT Compiler to stitch together the existing virtual scan chains created as part of the test-ready compile. The scan insertion flow automatically creates the necessary **SI**, **SO**, and wrapper ports on the macrocell, see *Test-ready compile* on page 5-18. This section provides information on:

- *Scan configuration*
- *Test clocks*
- *Design Rule Check* on page 5-28
- *Scan chain stitching with SoCTest* on page 5-28.

Scan configuration

You must define all the specific scan configurations for the design to enable the scan insertion process. You define the specific scan configuration attributes with the `set_scan_configuration` variable.

```
set_scan_configuration -methodology full_scan -style multiplexed_flip_flop -replace false \
  -chain_count $rm_num_scan_chains -dedicated_scan_ports true -add_lockup true \
  -clock_mixing mix_clocks
```

The scan configuration specifies a full-scan multiplexed flip-flop approach with a number of scan chains specified by `rm_num_scan_chains` in the options file.

Because the macrocell uses a single clock domain throughout, the minimum number of scan chains that you can insert within the device is one. This number excludes the SoCTest wrapper chain (see *Test wrapper insertion* on page 9-5).

If you obscure the macrocell at a higher level, the macrocell scan elements are not visible to test tools at that level and so it is not possible to integrate the macrocell scan chains into other scan chains. Whenever you require an obscured ARM926EJ-S macrocell, you must maintain the ARM926EJ-S scan chains as unique chains with dedicated scan ports at the top-level design.

Test clocks

When you have defined the scan configuration, you must set the test clocks and specific timing variables. `arm926ejs_physopt.tcl` defines:

```
set test_default_delay 0
set test_default_bidir_delay 0
set test_default_strobe 40
set test_default_period 100
```

You must also define the *Test Protocol File* (TPF) created after scan insertion:

```
set test_stil_netlist_format verilog
```

The `create_test_clock` variable defines the test clocks:

```
create_test_clock [ get_ports {CLK} ]-period 100 -waveform {45 55}
```

All the test clocks have:

- a clock period of 100ns
- rising edges 45ns into the clock period
- falling edges 55ns into the clock period.

Design Rule Check

You must perform a *Design For Test (DFT) Design Rule Check (DRC)* to ensure that the design satisfies all test rules for the configuration selected. Use the `check_dft` command to perform the DRC. A report file maintains the output of the DRC and you must analyze this report file thoroughly. Errors and warnings found during `check_dft` can significantly reduce the overall test coverage of the design:

```
check_dft > ./synthesis/reports/ARM926EJS.dft
```

The `preview_dft` command previews the scan chains and enables you to change the scan chain configuration, if necessary. The output from `preview_dft` attaches to the DFT DRC report:

```
preview_dft -physical >> ./synthesis/reports/ARM926EJS.dft
```

Scan chain stitching

You physically stitch and reorder scan chains using the `insert_dft` command. To have the scan chains ordered based on the physical location of the cells, you must use the `-physical` switch with the `insert_dft` command, for example:

```
insert_dft -physical -map_effort medium
physopt -incr -effort high
```

Scan chain stitching with SoCTest

If you set the `rm_create_test_model` switch in the options file, Synopsys SoCTest creates a test wrapper around the macrocell. Use the `arm926ejs_physopt.tcl` script to set up SoCTest, see *arm926ejs_physopt.tcl* on page 5-7. For details of the variables in the options file, see Table 5-2 on page 5-10.

———— Note ————

You must have a SoCTest license when `rm_create_test_model` is set to use Synopsys SoCTest.

To enable SoCTest you must define these configuration settings in DFT Compiler:

```
set test_use_test_models true
set test_use_ctl2prot true
set_dft_configuration -core_wrapper
```

These commands enable:

- generation of the test models
- conversion of CTL to protocol for DRC
- core wrapping functionality.

Set the type of wrapper cells to use to wrap the input and output ports with the command:

```
set_core_wrapper_configuration -dedicated_wrapper_cell WC_D1 -chain_count $rm_num_wrapper_chains
```

The `-chain_count` switch defines the number of wrapper chains used to wrap the core. Set up this variable, and the naming convention for the wrapper input and output ports, in the options file.

With the 2003.03 release of the Synopsys tools, SoCTest is physically aware. Therefore you do not have to supply port listings for each wrapper chain. SoCTest adds ports to each wrapper based on the location of the ports in the floorplan.

You must name each wrapper that SoCTest inserts. You can use SoCTest to associate scan in and scan out ports with the named wrappers with these commands:

```
set_core_wrapper_path $wrapper_name []
set_scan_signal test_scan_in -port $port_name -chain $wrapper_name
set_scan_signal test_scan_out -port $port_name -chain $wrapper_name
```

The empty square brackets instruct SoCTest to add ports to each wrapper based on the physical location of the ports. The two `set_scan_signal` commands assign the wrapper ports to a specific wrapper using the name of the wrapper and the `-chain` switch.

A for loop gives support for multiple wrappers and the `rm_num_wrapper_chains` variable stores the number of wrappers created. Set the `rm_num_wrapper_chains` variable in the options file, see *Options* on page 5-10.

The `set_dft_signal` command, in the compile script, assigns the wrapper signals common to all wrappers to their corresponding ports.

The script inserts the wrapper and scan chains with this command:

```
insert_dft -physical -ignore -dont
```

To uniquify and physically add the wrapper cell to the floorplan:

```
uniquify
physopt -incr -effort high
```

On completion of scan insertion, a DRC produces a report:

```
check_dft>>./synthesis/reports/ARM926EJS.dft
```

5.6.6 High fanout net buffering

You can set the `rm_create_buffer_tree` switch in the options file. This enables Physical Compiler to synthesize buffer trees on high fanout nets. From the `arm926ejs.physopt.tcl` file:

- You must increase the `steiner_route_fanout_limit` and the `high_fanout_net_threshold` so that you can create buffer trees, for example:


```
set steiner_route_fanout_limit 40000
set high_fanout_net_threshold 40000
```
- You must remove the high fanout nets set to ideal during synthesis, to create a buffer tree, for example:


```
foreach hfn [list HRESETn DBGnTRST DBGTKEN SCANENABLE ] {
  remove_ideal_net [ get_net $hfn ]
  set_dont_touch [ get_net $hfn ] false
  create_buffer_tree -from [ find port $hfn ]
```

5.6.7 Saving the design

For interfacing to other tools you must save the completed design in:

- Synopsys `.db` format
- *Synopsys Design Constraints* (SDC) format
- Verilog netlist `.v` format.

In all cases it is recommended that you save the full hierarchical database. This section provides information on:

- *Changing names*
- *Exporting the design* on page 5-31

Table 5-4 on page 5-34 shows the reports that synthesis process outputs.

Changing names

To change any names in the design that do not satisfy the name rules defined in the technology script, use the `change_names` command in Design Compiler:

```
change_names -rules verilog -hierarchy -verbose
```

You can, optionally save the output from the `change_names` command to a file for further analysis.

Exporting the design

To write Verilog netlists out of Physical Compiler use these two variables:

- `set verilayout_higher_designs_first true`
- `set verilayout_no_tri true`

With the `verilayout_no_tri` variable set to true tristates do not occur in the final gate-level netlist because they can cause problems in a number of third-party EDA tools.

Third-party EDA tools require timing constraints for the design in *Synopsys Design Constraints* (SDC) format.

The design is ready for exporting:

```
write -f db -hierarchy -o ./synthesis/data/ARM926EJS.db
write -f verilog -hierarchy -o ./synthesis/data/ARM926EJS.v
write_sdc ./synthesis/data/ARM926EJS.sdc
write_pdef -v3.0 -output ./synthesis/data/ARM926EJS.pdef
```

5.7 Floorplanning and SoCTest

When you use SoCTest, as part of DFT Compiler, you might encounter congestion problems related to your floorplan design. Congestion occurs because the physopt script does not insert the wrapper cells until the `insert_dft` stage. You cannot fix this congestion at the incremental physopt stage because Physical Compiler can only move cells a small maximum displacement from their starting point for an incremental compile.

To get around this congestion problem, constrain the floorplan to create a keepout region that covers the area close to the pins. For example, with Physical Compiler:

1. Compile the design and define the keepout area:
 - a. Run the compile and physopt scripts with low effort compile and physopt options and with reduced timing targets. This normally completes quickly.
 - b. Load the db file using the Physical Compiler GUI.
 - c. Select **Show Congestion**, on the **Layout** menu, to show the congestion map for the current design.
 - d. Find the lower left and upper right bounds of the keepout region that you require. Make sure that the box you define encapsulates any congestion greater than 100% in the region of the pins on the floorplan.
2. Edit the scripts:
 - for a gates-to-placed-gates flow edit physopt script
 - for an RTL-to-placed-gates flow edit both the physopt and compile scripts.

Change the appropriate scripts after the `read_pdef` command, for example:

```
read_pdef -allow_physical_ports ./floorplan/data/ARM926EJS.pdef
create_placement_keepout -name "port_keepout" -coordinate { your_coords } -type hard
physopt -timing_driven_congestion -effort high -area_recovery -num_cpus $rm_num_cpu
remove_placement_keepout -name "port_keepout"
report_placement_keepout [get_placement_keepout * ]
```

This change inserts a placement keepout, before the physopt runs, and then removes it, when physopt completes, so there is enough space for the `insert_dft` stage to place the wrapper cells.

3. Use a congestion based incremental physopt stage, if necessary, to remove any remaining congestion. This last step removes any congestion, around the bounds of the removed keepout region, that remains after the `insert_dft` and incremental physopt stages complete. For example:

```
insert_dft -physical -ignore -dont
uniquify
physopt -incr -effort high
set_congestion_options -horizontal 0.7 -vertical 0.7 -max_util 0.95 -coord { your_coords }
```

physopt -incr -effort medium -congestion

Because this flow depends on your floorplan you must update it, if necessary,
when you change your floorplan.

5.8 Reports

Table 5-4 shows the reports that are output from the synthesis process. These reports are written to the synthesis/reports directory.

Table 5-4 Synthesis reports

report	Description
ARM926EJS-compile.options	An output of the compile options used during synthesis
ARM926EJS-compile.hier	An output of report_hierarchy after the initial compile
ARM926EJS-compile.resources	A list of all DesignWare resources after the initial compile
ARM926EJS-compile.timing	A complete timing report of the top 200 violators after the initial compile
ARM926EJS-compile.constraint	A list of all violations after the initial compile
ARM926EJS-compile.clocks	An output of the clock attributes after the initial compile
ARM926EJS-compile.loops	An output of the loops in the design
ARM926EJS-compile.port	An output of the port attributes after the initial compile
ARM926EJS.dft	An output of check_dft after insert_dft has run
ARM926EJS.area	A report of the combinatorial and non-combinatorial area of the design
ARM926EJS.design	A summary of the design
ARM926EJS.check	An output of check_design before compilation
ARM926EJS.hier	An output of report_hierarchy after compilation
ARM926EJS.latches	A list of latches in the design after compilation
ARM926EJS.port	An output of report_port after compilation
ARM926EJS.scanpath	A list of the complete scan chains in the design
ARM926EJS.constraint	A list of all violations in the design
ARM926EJS.propagate_constraints	A list of all violations propagated in the design
ARM926EJS.timing	A complete timing report of the top 200 violators in the design
ARM926EJS.gating	A list of all the clock gates and their location in the design
ARM926EJS.resources	A list of all DesignWare resources in the design
ARM926EJS.congestion	A report of congestion statistics
ARM926EJS.scantest	A list of scan configuration settings
ARM926EJS-scan.timing	Complete timing reports of the top 32 violators in the design before high fanout net insertion

Chapter 6

CTS and Routing

This chapter describes how to do CTS and how to place and route the macrocell using Synopsys Astro tools. It contains the following sections:

- *About CTS and routing* on page 6-2
- *Supported CTS and routing flow* on page 6-3
- *CTS and routing scripts* on page 6-4
- *Set up for Astro layout* on page 6-7
- *CTS and clock skew* on page 6-8
- *Routing* on page 6-9
- *Post-routing* on page 6-10
- *Static Timing Analysis* on page 6-13.

6.1 About CTS and routing

The macrocell is a very tightly tuned design with a large density of paths in the critical range of the design. A good floorplan and placement of the macrocell is crucial to ensure that there is no degradation of performance of the macrocell. For details of floorplanning, see Chapter 4 *Floorplanning*.

This chapter provides layout guidelines and describes considerations for the clock and high fanout nets.

Five scripts control the CTS and routing flow:

- *arm926ejs_import.scm* on page 6-4
- *arm926ejs_cts.scm* on page 6-5
- *arm926ejs_route.scm* on page 6-5
- *arm926ejs_opt.scm* on page 6-5
- *arm926ejs_export.scm* on page 6-5.

6.2 Supported CTS and routing flow

Figure 6-1 shows a general design flow for producing a layout.

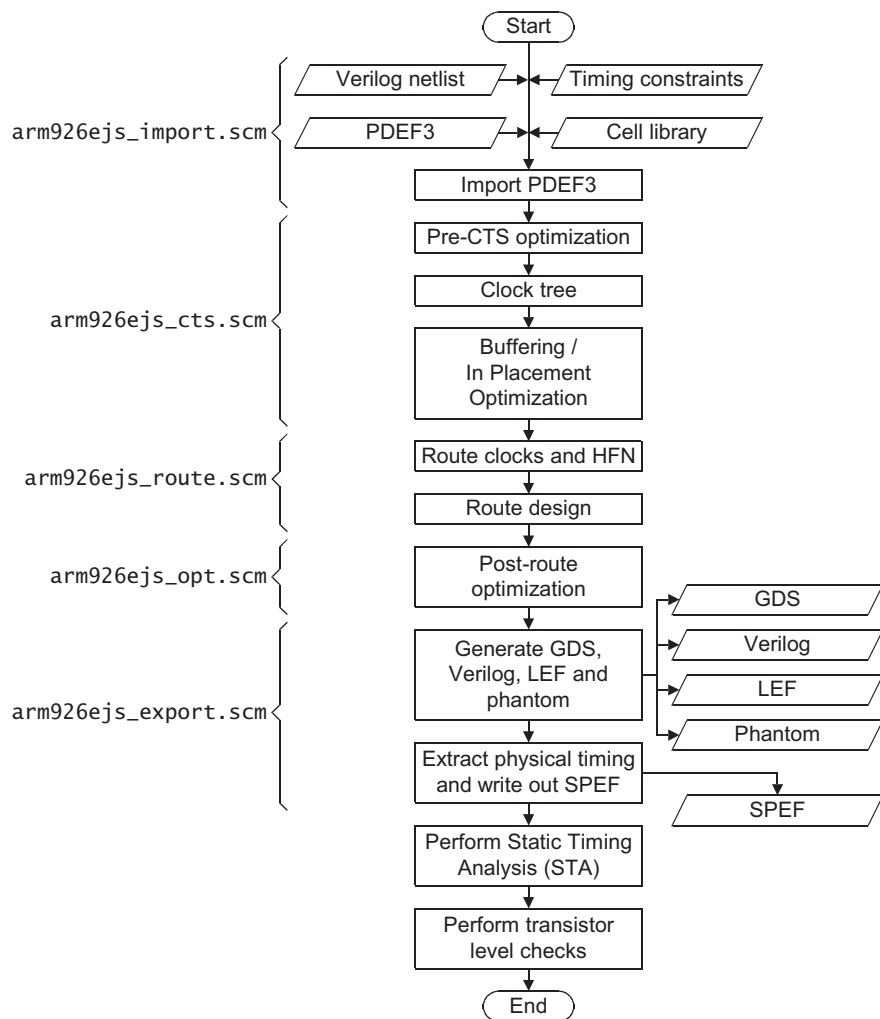


Figure 6-1 CTS and routing flow

You can use the Makefile, located in the /build_dir directory, to run all the Astro flow from preparing the data to generating the output files, see *Makefile* on page 5-12.

6.3 CTS and routing scripts

This section describes the actions of the five scripts that control the CTS and routing process:

- *arm926ejs_import.scm*
- *arm926ejs_cts.scm* on page 6-5
- *arm926ejs_route.scm* on page 6-5
- *arm926ejs_opt.scm* on page 6-5
- *arm926ejs_export.scm* on page 6-5.

The scripts are located in the `build_dir/scripts` directory and you can run them with the Makefile, see *Makefile* on page 5-12.

Make back for example:

- prepares the PDEF3 by running `pdef2scheme`
- modifies the SDC file from synthesis to work with Astro
- runs Astro.

Alternatively, you can make your Astro tool session run all the scripts sequentially by making a Createchip file to load the five scripts in turn, for example:

```
load "../scripts/arm926ejs_import.scm"
load "../scripts/arm926ejs_cts.scm"
load "../scripts/arm926ejs_route.scm"
load "../scripts/arm926ejs_opt.scm"
load "../scripts/arm926ejs_export.scm"
```

To use this method you must edit each of the five scripts:

- comment out the `open library` and `open cell` commands in all but the first script
- comment out the `exit` command in all but the last script.

If you use this method you do not have to start Astro each time to run separate stages individually. This method is especially useful for debugging the Astro flow.

6.3.1 arm926ejs_import.scm

The `arm926ejs_import.scm` script:

- creates the working library
- adds the reference libraries
- adds the standard cell libraries
- loads the Verilog post-synthesis netlist file
- binds the netlist
- sets up the timing
- reads the PDEF3 file.

6.3.2 arm926ejs_cts.scm

The arm926ejs_cts.scm script:

- runs optimization before CTS
- inserts the clocks
- runs post CTS optimization
- optimizes the clock network.

6.3.3 arm926ejs_route.scm

The arm926ejs_route.scm script:

- routes the clocks
- routes the high fanout nets
- routes all the signal nets.

6.3.4 arm926ejs_opt.scm

The arm926ejs_opt.scm script performs the post-route timing optimization.

6.3.5 arm926ejs_export.scm

The arm926ejs_export.scm script:

- extracts SPEF parasitic information using Star-RCXT
- writes the SPEF output file
- writes the Verilog post CTS netlist
- writes the GDS2 output file
- runs Astro *Layout Versus Schematic (LVS)* and *Design Rule Checks (DRC)*.

6.3.6 How to modify scripts for use with SoCTest

If you use a SoCTest wrapper modify the following three scripts:

arm926ejs_tsmc.scm

Set the option `rm_use_wrapper` to 1.

This option affects the following scripts:

- arm926ejs_cts.scm
The net **WRP_CLOCK** is inserted as a clock at the same time as **CLK**
- arm926ejs_route.scm
The net **WRP_CLOCK** is prerouted at the same time as **CLK**

ARM926EJS-astro.sdc

Set **WRP_CLOCK** as a clock net. You can copy the syntax of the `create_clock` command for **CLK**.

For example to create **WRP_CLOCK** at 200MHz, use:

```
create_clock -period 4 -waveform {0 2} [get_ports {WRP_CLOCK}]
```

Remove the `set_input_delay` and `set_driving_cell` commands for **WRP_CLOCK**.

The constraints are generated by the `make_prep` command, see *Makefile* on page 5-12.

arm926ejs_cts.scm

Set **WRP_CLOCK** as a propagated clock.

Under the command `tcl "set_propagated_clock CLK"` in the `Move to propagated clock` section add the following line:

```
tcl "set_propagated_clock WRP_CLOCK"
```

———— Note ————

It might be necessary to balance the latency of **WRP_CLOCK** with **CLK**, especially if you intend to run **WRP_CLOCK** at the same frequency as **CLK**.

6.4 Set up for Astro layout

You must prepare the data for the Astro flow, before you run Astro to produce your layout, by modifying:

- the PDEF3 file produced by Physical Compiler
- the SDC file produced by Physical Compiler
- the arm926ejs_<rm_technology>.scm file
- the arm926ejs_import.scm file.

Use this setup process:

1. Modify the PDEF3 and SDC files using the Makefile:

Make prep

This command:

- reformats the PDEF3 into an .scm file that Astro can read
- removes all unsupported SDC commands for Astro.

For more information on the Makefile, see *Makefile* on page 5-12.

2. Modify the SDC file. If you do not intend to do any work before CTS, remove the set_clock_latency and set_clock_uncertainty commands from the SDC file.
3. Modify the arm926ejs_<rm_technology>.scm file. If you use more than one RAM library:

- include the RAMs in arm926ejs_import.scm under the Add the RAM libraries heading
- ignore the variable rm_rams_dir.

For 0.18μm technology use these values as a starting point:

```
(define rm_high_fanout_count "30")
(define rm_max_transition "0.8")
(define rm_max_capacitance "1")
(define rm_target_utilization "85")
```

4. Modify arm926ejs_import.scm:

If necessary, change the don't use cell list and clock cell names under the Identify cells to be used heading.

6.5 CTS and clock skew

The place and route tool handles CTS better than a synthesis tool. Figure 6-2 shows how the size of the clock buffers tapers the closer they get to the clocked elements.

To achieve the best clock tree skew, the clock signal must arrive at each clocked element at the same time, taking into account the length of the wiring and the placed position of each clocked element. To reduce latency the clock tree must be as shallow as possible and maintain a good transition time through the clock buffers. For the main trunk of the clock tree use a few strong drive strength buffers rather than many smaller ones.

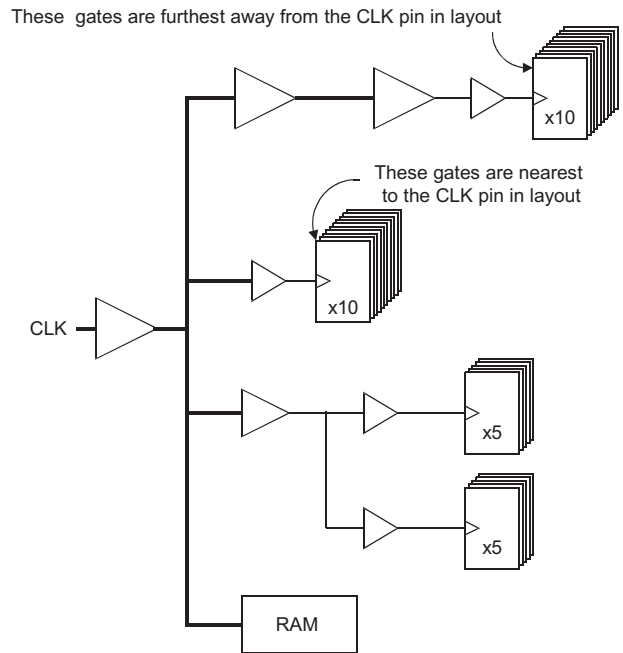


Figure 6-2 Example clock tree

You must include the latency added to **VCLK**, caused by clock insertion. You can do this in one of two ways:

- adding the real latency to **VCLK** in the SDC file and reloading the file
- setting the option “set IO Clock Latency” in the Environment tab of form atTimingSetup.

———— Note ————

By default, Astro automatically buffers high fanout nets when the fanout limit reaches the value set for variable `rm_high_fanout_count`.

6.6 Routing

You must route the sets of signals described in:

- *Clock tree routing*
- *High fanout net routing*
- *Signal routing.*

6.6.1 Clock tree routing

You might have to route at least the first level of the clock tree in wider than minimum metal to reduce the skews across the net, and for electromigration reasons. Figure 6-2 on page 6-8 shows an example clock tree.

A wide clock route can take up two or more routing channels, so you must route this net before routing the rest of the design. Routing the clock first also enables you to minimize the length of the clock net because you can achieve more direct routing.

6.6.2 High fanout net routing

It is important to route the high fanout net signals before all the other signals to minimize wire length. You can combine all the high fanout nets into one routing pass by separating their names with a comma, for example:

1. Change:

```
setFormField "Route Net Group" "Net Name(s)" "SCANENABLE.*"
```

To:

```
setFormField "Route Net Group" "Net Name(s)" "SCANENABLE.*,.*HRESETn.*,.*DBGnTRST.*,.*DBGTCCKEN.*"
```

2. Delete the other three axgRouteGroup commands

6.6.3 Signal routing

Before routing the design it is recommended that you consider:

- shielding the clock route and adding extra space to other signals
- antenna ratio effects
- routing I/O pins in wider than minimum metal, to reduce congestion around pins
- adding more vias because of high switching currents
- areas, such as RAMs, that you might not be permitted to route over.

How much of an effect these factors have on the design depends on the target process technology you are using.

6.7 Post-routing

This section describes:

- *Gate-level DRC and LVS analysis*
- *Electromigration and voltage (IR) Drop*
- *Cross talk analysis* on page 6-11
- *Multiple vias* on page 6-11
- *Antenna rules* on page 6-11
- *Metal fill patterns* on page 6-12
- *Extracting parasitics* on page 6-12.

6.7.1 Gate-level DRC and LVS analysis

When you have completed routing on the design you must check that it is DRC and LVS clean. If the design has violations you must fix them manually or use the automated repair facilities of the tool.

You must ensure that your design is DRC and LVS checked against the latest DRC and LVS decks that the foundry supply.

6.7.2 Electromigration and voltage (IR) Drop

Power supplies and high fanout nets are at risk from electromigration. You must ensure that these nets are not an electromigration risk.

For design considerations for the power supply for the macrocell and the effect that your design has on electromigration, see *Floorplanning* on page 4-5.

————— Note —————

All power ports of the hardened macrocell must be connected to an external power ring. This power ring must supply adequate power to all ports of the macrocell and any additional cells, for example the RAM blocks. Where practical, use more metal for power, clock nets, and other high drive level nets.

Electromigration limits for minimum-width metal tracks depend on the target process, and you must pay attention to any nets that exceed these limits. Clock and high fanout nets are susceptible to electromigration but not all high fanout nets are an electromigration risk.

The minimum width of a track is a function of:

- the equivalent current (I_{eq}) that it carries
- the metal width
- the metal length
- the topology of the track
- the die operating temperature.

An essentially static net with a very high fanout has a low I_{eq} and therefore presents no electromigration risk.

It is recommended that you sign off with a tool, for example Astro-Rail, to analyze electromigration and voltage, IR, drop across the power network.

6.7.3 Cross talk analysis

You must do cross talk analysis on geometries of 0.18 μ m and below. Cross talk violations, caused by long parallel wires of different current drive strengths, can occur in very small process geometries. The stronger driven signal can act as an aggressor to a weaker driven signal and influence its state. The earlier in the flow that you identify signals that might exhibit cross talk problems the better. You can alleviate cross talk by:

- physical separation of gates or nets at placement
- buffer insertion, to boost the weakly driven signals
- upsizing or downsizing gates.

The Astro RM4.1 flow does not support cross talk fixing but STA does take crosstalk effects into account when the option `rm_enable_si` is set.

6.7.4 Multiple vias

It is recommended that you use more than one via on:

- clock nets
- high drive strength signals.

6.7.5 Antenna rules

All signal nets can be subject to antenna failure. The likelihood of an antenna failure depends on the ratio of gate size to the length of metal attached to the gate.

After passing DRC, you must fix antenna rule violations. You can extract the polygon data and use a tool, for example Calibre or Hercules, to check for antenna violations. You can fix these violations using high performance option routing, that makes layer changes or inserts diodes, so that metal run lengths stay within the limits described in the antenna rules for a particular process.

Note

It is recommended that you place antenna diodes on all inputs and outputs to assist with macrocell integration.

Metal layer change

Break the net as close to the source of the antenna violation as possible, in the layer of metal where the violation occurs, and add a jumper to the next layer of metal.

Diode insertion

Insert a diode as close to the gates as possible to sink the *Electrostatic Discharge* (ESD).

6.7.6 Metal fill patterns

You must insert metal fill patterns to meet metal density rules. You must perform fill pattern insertion after detailed routing, and before any post-route optimization, so that this optimization uses the actual post-route capacitances. If you do this as a post-route step, after the final optimizations, you must check the timing and cross talk again.

6.7.7 Extracting parasitics

When you have finished routing the design and it is LVS and DRC clean, you must extract the parasitics for use in post-layout STA.

To use the supplied STA scripts you must extract parasitics in SPEF format. If you require signal integrity analysis as part of the STA flow, turn off couple to ground, on form astStarRCXT, and turn on dump coupling mesh, on form astSPEFout.

Note

If you extract the parasitic data before running post-route optimization you might get better results because the optimization will use real RC data.

6.8 Static Timing Analysis

When the design is placed-and-routed and a first pass of transistor level checks is run, the next stage is to run STA, see Chapter 7 *STA and Timing Model Generation*. The results from STA might indicate that some rework is required for the layout.

Chapter 7

STA and Timing Model Generation

This chapter gives guidelines for timing verification of the netlist and timing model extraction using *Static Timing Analysis* (STA) tools. It contains the following sections:

- *About STA and timing model generation* on page 7-2
- *Supported STA and timing model generation flow* on page 7-3
- *STA and timing model generation scripts* on page 7-4
- *Setting up STA and timing model generation* on page 7-6
- *Running STA and timing model extraction* on page 7-7
- *The STA and timing model generation process* on page 7-8
- *Extracting the library cell timing model* on page 7-13
- *Verifying the extracted timing model* on page 7-14
- *Outputs* on page 7-17.

7.1 About STA and timing model generation

The timing verification method uses STA tools, providing you with a fast and exhaustive means of analyzing the timing performance of the macrocell. When you perform STA in conjunction with formal verification methods, there is no requirement for a large number of resource-expensive gate-level simulations to verify the design.

STA tools enable you to identify the critical path of the design, and also enable you to analyze the distribution of the timing paths. This helps you to identify possible areas for improvement.

You can perform STA after each phase of synthesis, place and route, and in-place optimization to enable early feedback of timing issues.

You can read layout delay or parasitic information into your STA tool to enable accurate post-layout timing analysis. A Synopsys PrimeTime script is provided that enables you to perform STA on the macrocell.

For timing model generation you must ensure that you generate a single timing model view from a physically implemented design. A single model generated from a single source, used in all phases of the SoC design flow, significantly reduces the time required to achieve design timing closure.

It is recommended that you generate the timing model for the macrocell using Synopsys PrimeTime version 2003.03-1 or later, from a post place and route netlist with annotated parasitics extracted from the layout database. However, if necessary, you can generate a design budgeting timing model using a post-synthesis netlist with wire load model estimation of the interconnect effects.

The generated timing model from PrimeTime supports:

- logic synthesis with Design Compiler
- physical synthesis with Physical Compiler
- static timing analysis within PrimeTime
- timing driven place and route with the majority of commercially available place and route tools.

Because the model generated by PrimeTime is represented using industry standard formats, the timing model is portable across design environments for use with other commercially available EDA tools.

7.2 Supported STA and timing model generation flow

Figure 7-1 shows the supported flow for creating a timing model from a gate-level netlist of the macrocell.

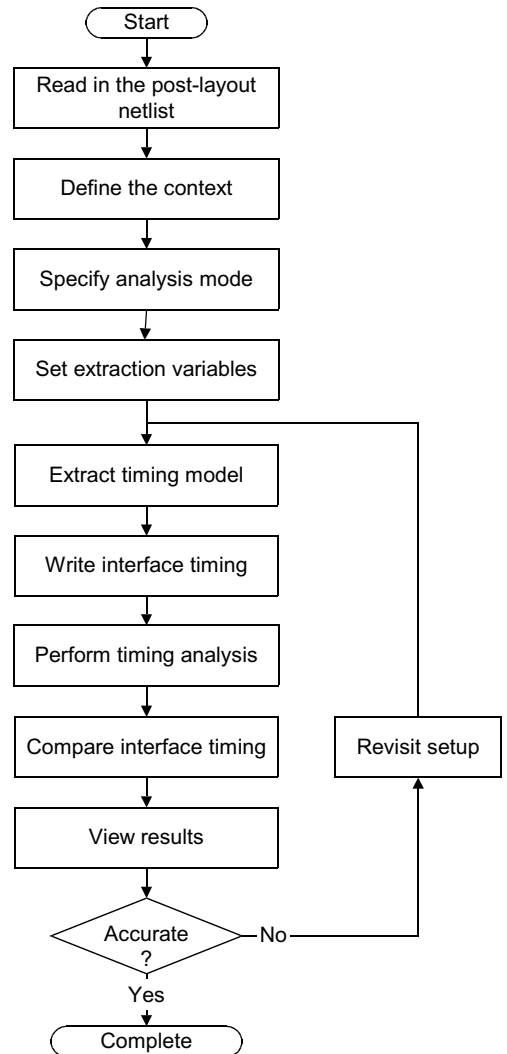


Figure 7-1 Timing model extraction process

7.3 STA and timing model generation scripts

The following scripts support STA and timing model generation for the macrocell:

- *arm926ejs_sta.tcl*
- *arm926ejs_tmequal.tcl*
- *arm926ejs_constraints.tcl*
- *arm926ejs_config.tcl* on page 7-5
- *arm926ejs_tsmc18.tcl* on page 7-5
- *options* on page 7-5.

All the files are located in your installed `./synopsys/build_dir/scripts` directory except the options file, which is located in the `./synopsys/build_dir` directory. Figure 7-2 shows the association of the STA and timing model generation scripts.

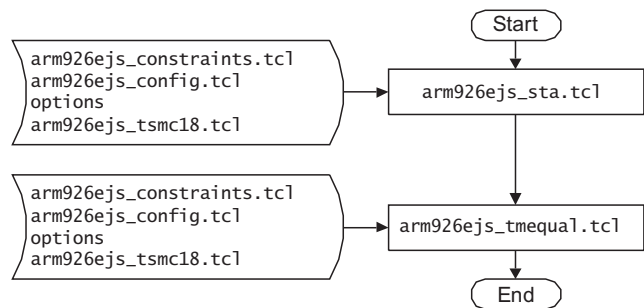


Figure 7-2 STA and timing model generation script association

7.3.1 arm926ejs_sta.tcl

This is the PrimeTime STA and timing model generation script for the macrocell.

7.3.2 arm926ejs_tmequal.tcl

This is the timing model qualification script for the macrocell.

7.3.3 arm926ejs_constraints.tcl

This script gives default I/O constraints. You can modify these constraints during implementation to suit the target design context. You must ensure that any modifications to the timing constraints are consistent with the timing budget for the target design.

7.3.4 arm926ejs_config.tcl

This script:

- sources the options file
- validates the options
- sources the technology script.

Both the sta and tmequal scripts use the config script.

7.3.5 arm926ejs_tsmc18.tcl

This is an example script of the target libraries, link libraries, and synthetic libraries, that STA and timing model generation of the macrocell require.

7.3.6 options

The options file enables you to control the STA and timing model generation flow of the design, and is located in your installed synopsys/build_dir directory. For details on the options and how to use them, see *Setting up the control options for STA and timing model generation* on page 7-6

7.4 Setting up STA and timing model generation

Before you proceed with the STA and generate a timing model of the implemented macrocell you must ensure that:

- The design satisfies all timing constraints at the operating condition of interest.
- The design satisfies all design rules at the operating condition of interest.
- You understand timing exceptions and define them.
- The type of extracted timing model you require is a sign-off quality model.

A complete sign-off quality model might be required by the SoC designer prior to tape out of the macrocell based SoC. This model must be accurate and provide a sign-off quality view of the timing for the physically implemented macrocell. Extract this model from a netlist with the clock tree inserted and extracted layout parasitics.

- You set up the options file to your requirements. See *Setting up the control options for STA and timing model generation*. This file is located in the synopsys/build_dir directory.

7.4.1 Setting up the control options for STA and timing model generation

The options file provides overall control of the STA flow and is located in your installed synopsys/build_dir directory. Table 7-1 describes the STA options and their default settings. You can modify these options to control the flow as appropriate.

Table 7-1 STA options

Synthesis options	Default setting	Description
rm_extract_model	1	Invoke the timing model extraction process
rm_operating_condition	slow	Specify the operating condition required for the extraction process
rm_enable_si	0	Read in the design parasitics for <i>Signal Integrity</i> (SI) analysis

7.5 Running STA and timing model extraction

To run STA and timing model generation, change to your installed synopsys/build_dir directory, then type:

```
make tme
```

This command runs the arm926ejs_sta.tcl and arm926ejs_tmequal.tcl scripts in turn. The build_dir/logs directory stores the logs:

- logs/sta
- logs/tmequal.

7.6 The STA and timing model generation process

This section describes the timing model extraction process for the macrocell design. The PrimeTime STA tool extracts the timing model from post-layout netlist and parasitics. The timing model does not contain any implementation details and you can consider it as a black box model. Because the extracted timing model satisfies the requirement for a protected model it is the preferred method for modeling the timing of the macrocell.

The timing model of the macrocell must have specific attributes for use in the SoC design environment. These attributes are:

Black box Implementation details of the macrocell must remain hidden from the end user. This requirement is for the protection of the intellectual property as well as the protection of the implementation. The timing model represents effectively a leaf cell in the design and therefore any internal node information within the timing model is not represented in a simulation model or physical model and cannot be used.

Context-independent

The extracted timing model must be context-independent and support derating in the SoC environment. PrimeTime uses several different input edge rates and capacitive loads to generate lookup tables within the model that define the characteristics of the inputs and outputs to the design. These lookup tables provide for a context-independent solution.

Accurate Use the timing model, provided for the macrocell, as a sign-off model for the SoC before tape out. It is unlikely that the timing for the SoC is reverified with the macrocell netlist. The accuracy of the timing model must be within a predefined tolerance of the physically implemented netlist. In addition, you must define well, and document, areas where the timing is the most inaccurate.

Industry standard format

The timing model must be in an industry standard form, that commercially available EDA tools can consume, because many stages within the SoC design flow use the model. Extracted timing models are available in Synopsys .db format, Liberty .lib, and STAMP formats.

————— Note —————

The scripts supplied only support the Liberty .lib formats.

7.6.1 Controlling STA and timing model generation

PrimeTime uses several variables to control the extraction of the timing model. Typically these controls trade off accuracy of the timing model with the speed of creation, but others control the nature of the resulting model and its applicability to other phases of the design flow.

PrimeTime uses the following variables to control the extraction of the timing model:

- `extract_model_tolerance`
- `extract_model_min_resolution`
- `extract_model_capacitance_limit`
- `extract_model_clock_transition_limit`
- `extract_model_data_transition_limit`
- `extract_model_status_level`.

Note

The resulting timing model of the macrocell is an abstraction of the original design. Because it is an abstraction, and the netlist is no longer available, this reduces the level of detail available to PrimeTime (or any delay calculator).

It is recommended that you use the PrimeTime defaults for model extraction. If the resulting models are inaccurate, you can adjust the variables. Refer to Synopsys documentation for the use of these variables.

7.6.2 Reading in the design

For a sign-off model, you must use a functional netlist with clock-gating, scan insertion, and clock tree insertion:

```
set svr_keep_unconnected_nets true
read_verilog ./layout/data/ARM926EJS.v
```

You must define the design that you link with the technologies and RAM libraries as appropriate:

```
current_design ARM926EJS
link_design
```

7.6.3 Specifying clocks

You must define the clocks appropriately for the type of timing model you extract.

Because you create a sign-off quality model, the following clocks must use the `set_propagated_clock` define to take into account clock tree propagation:

```
create_clock -name CLK -period $rm_clock_period [get_ports CLK]
set_propagated_clock [get_clocks CLK]
```

```
create_clock -name VCLK -period $rm_clock_period
set_clock_uncertainty $rm_clock_uncertainty VCLK
set_clock_latency $rm_clock_latency VCLK
```

7.6.4 Design context

Place the design into an appropriate context by applying interface constraints and I/O characteristics to the design. Your technology library defines these characteristics.

Although `extract_model` does not use this information, it has to be in place when writing out the interface timing of the design as described in *Verifying the extracted timing model* on page 7-14.

7.6.5 Interconnect

A sign-off model requires detailed parasitics that accurately reflect the resistance and capacitance of each net in the design based on the physically implemented design:

```
read_parasitics -complete_with wlm ./layout/data/ARM926EJS.spef
```

The `read_parasitics` command reports on the annotation process and identifies situations where information is not annotated. If parasitics are not available the net completes using the wire load estimation. A table shows this information after reading in the parasitics.

————— **Note** —————

You must ensure that all nets within the design have correctly annotated parasitic information before extraction.

The STA flow only supports parasitic information provided in SPEF format.

7.6.6 Timing exceptions

PrimeTime uses the clock definitions and the interface timing constraints to determine appropriate start and end points for timing analysis. By default PrimeTime constrains end points for single-cycle timing. You must provide PrimeTime with information on the end points not covered by single-cycle timing constraints and treat these end points as exceptions within the design.

The model extraction process can handle timing exceptions such as false paths and multi-cycle paths. You do not maintain these paths within the extracted model but in a .sdf file created during extraction. When you use the extracted model in a design environment, you annotate the file of timing exceptions to override the single-cycle behavior.

7.6.7 Case analysis

During STA, you must perform case analysis on the following nets to check for hold violations:

- **TESTMODE**
- **SCANENABLE.**

7.6.8 Operating conditions

A timing model represents the timing for a design at a specific operating condition. The extracted timing model has no way of representing timing for multiple operating conditions. Therefore, you must specify the operating condition required before extraction:

```
set_operating_conditions ${rm_operating_condition}
```

7.6.9 Cross talk analysis

You can use PrimeTime-SI, during STA and timing model extraction, to analyze the timing effects of cross-coupled capacitances between nets after layout.

To read in the design parasitics, you must ensure the `rm_enable_si` option is set in the options file. This enables PrimeTime-SI within the flow by setting the `si_enable_analysis` variable to true.

```
set si_enable_analysis true
```

When set this variable modifies the `read_parasitics` command to include a switch that ensures that the cross-coupling capacitance information that is read into PrimeTime is maintained:

```
read_parasitics -keep_capacitive_coupling -complete_with wlm ./layout/data/ARM926EJS.spef
```

Note

To analyze crosstalk effects in the design, STA requires parasitics including coupling information in SPEF format.

7.6.10 Timing the design

To confirm correct application of all the required timing constraints and exceptions to the design, you must update the timing of the design and check the timing using the `check_timing` command:

```
update_timing
redirect ./sta/reports/ARM926EJS.${rm_operating_condition}.checktiming \
{ check_timing -verbose }
```

You must fix potential problems identified by the `check_timing` command before timing model extraction.

7.6.11 Writing out the interface timing

To write out the interface timing of the design use the command:

```
write_interface_timing -timing_type arc \
./sta/reports/ARM926EJS.${rm_operating_condition}.ntiming
```

7.6.12 Writing out SDF and post-layout optimization constraints

To write out the delay calculation and static timing analysis information, required by Physical Compiler, in *Standard Delay Format* (SDF) v3.0 format use the command `write_sdf`:

```
write_sdf -version 3.0 -input_port_nets -output_port_nets -context verilog \
./sta/data/ARM926EJS.${rm_operating_condition}.sdf
```

To ensure that, when pin names are printed, the last pair of square brackets, [and], are not escaped, define a Verilog context.

7.7 Extracting the library cell timing model

The library cell timing model provides a pins-only model of the design. All ports on the original design are maintained in the model and no internal nodes are created. In addition, the delay associated with the boundary net parasitics is accounted for within the extracted model. Because the parasitics on the boundary nets are modeled as part of the timing behavior of the design, some accuracy is lost. Also, the improvements in accuracy offered by using internal nodes is not available with the library timing model. However, the accuracy of the library timing model is very high.

The recommended approach is that you create a library timing model in the Liberty .lib format.

This model contains all the information necessary to link directly with other Synopsys tools in the design flow, without post-processing the timing model. You can also apply this model directly to all commercially available third-party tools that require a timing model and support the Liberty format.

The `-library_cell` option directs PrimeTime to create the library cell timing model. The `-test_design` option directs PrimeTime to create a test fixture for the design that aids in verification of the extracted model.

7.7.1 Liberty .lib format

```
extract_model -library_cell -test_design -format { db lib } -output ARM926EJS
```

This creates two files:

- a .lib file for the target STA corner, for example:
ARM926EJS.slow.lib
- a test .db file for the target STA corner, for example:
ARM926EJS.slow_test.db

7.8 Verifying the extracted timing model

To ensure that any inaccuracies are within the desired range, you must correlate the timing represented in the extracted model with the timing represented in the original netlist.

The process of abstraction of a model, by definition, introduces a degree of inaccuracy between the extracted model and the original netlist. However, it is necessary to quantify this inaccuracy for the extracted model and understand where and why the inaccuracy exists. It might also be possible, on further analysis of the model, to manipulate the design, synthesis approach, or both, to try and minimize some of this inaccuracy.

PrimeTime provides a capability to automatically compare the extracted timing model with the original netlist to ensure that the inaccuracy of the timing in the model is within predefined tolerances.

The approach to validation of the timing model uses interface timing reports. You generate an interface timing report for both the extracted timing model and the original netlist at the same operating conditions.

You compare these two interface-timing reports with specified tolerances on the slack, port capacitance, and transition times. You can generate a report that identifies which ports, if any, have failed the tolerance test:

- Use the `write_interface_timing` command to create an interface timing report for both the model and the netlist.

————— Note —————

A test fixture (created by `extract_model`) must be in place.

- Use the `compare_interface_timing` command to compare the two individual interface-timing reports.

7.8.1 Verification of the library cell timing model

You can use `compare_interface_timing` for each type of model extracted and for each format of model. In the preferred method of extraction you create a library cell timing model in Liberty .lib format and the verification of this type of model is explained here. For information on verifying other forms of extracted model refer to *Synopsys PrimeTime Modelling User Guide*.

After extracting the timing model from the netlist, you must:

1. Read in and link the test design created by `extract_model`, assuming that the `ARM926EJS.slow.lib` model has been compiled into a file called `ARM926EJS.slow.db` using the `arm926ejs_sta.tcl` script. Ensure that you update the `link_library` variable with the location of the extracted timing model `ARM926EJS.slow.db`:

```
set link_library [concat $link_library ARM926EJS.${rm_operating_condition}.lib.db]
read_db ./models/ARM926EJS_test.db
current_design ARM926EJS_test
link
```

2. Apply the appropriate timing constraints to the design, as with the netlist, and include the definition of clocks and interface timing constraints. Set the operating conditions and update the timing. Again, ensure that you cover all end points:

```
create_clock -name CLK -period $rm_clock_period [get_ports CLK ]
set_propagated_clock [ get_clocks CLK ]
```

```
create_clock -name VCLK -period $rm_clock_period
set_clock_uncertainty $rm_clock_uncertainty VCLK
set_clock_latency $rm_clock_latency VCLK
set_load [expr [load_of $rm_loading_pin ] * 4 ] [list [all_outputs]]
set_driving_cell -cell $rm_driving_cell -pin $rm_driving_pin [list \
[all_inputs] ]
source -echo -verbose ./scripts/arm926ejs_constraints.tcl
set_operating_conditions $rm_operating_condition
update_timing
check_timing
```

3. Write the interface timing for the extracted timing model:

```
write_interface_timing -timing_type arc ./sta/reports/ARM926EJS.${rm_operating_condition}.mtiming
```

4. Compare the two interface timing reports produced using the `compare_interface_timing` command. This command can provide reports, on specific tolerances for slack, capacitance and edge rate, that show the results as pass or fail for each port of the design:

```
compare_interface_timing -nos ./sta/reports/ARM926EJS.${rm_operating_condition}.ntiming \
./sta/reports/ARM926EJS.${rm_operating_condition}.mtiming \
-output./sta/ARM926EJS.tme.${rm_operating_condition}.accuracy \
-absolute_tolerance { 0.05 -0.05 } \
-percent_tolerance 20 -sort_by_worst
```

———— **Note** ————

The reports show the paths that have a difference in total slack of greater than ± 50 ps as failing paths.

This report provides the designer with a quick way of identifying interface-timing paths in the design that are deemed inaccurate to the netlist.

In addition, you can specify and receive reports on the tolerance for the capacitance on ports. See the *PrimeTime Modelling User Guide* for more information on verifying extracted timing models.

7.9 Outputs

STA reports are written to the `synopsys/sta/reports` directory. Model timing extraction outputs are written to the following directories:

- `synopsys/build_dir/log`
- `synopsys/build_dir/sta`

Table 7-2 shows the reports produced by the STA script. The reports are located in the `./synopsys/build_dir/sta/reports` directory.

Table 7-2 STA reports

Report	Description
ARM926EJS.\${rm_operating_condition}.checktiming	Display the timing attributes placed on the current design, for example unconstrained endpoints
ARM926EJS.\${rm_operating_condition}.constraints	Display all constraint violators within the design for the particular process corner where STA was used
ARM926EJS.\${rm_operating_condition}.bottleneck	Display design bottlenecks and the number of paths contributing to the bottleneck
ARM926EJS.\${rm_operating_condition}.coverage	Display set up, hold, and minimum pulse width coverage
ARM926EJS.\${rm_operating_condition}.clock_skew	Display clock skew timing information
ARM926EJS.\${rm_operating_condition}.clock_latency	Display clock latency timing information
ARM926EJS.\${rm_operating_condition}.clock_transition	Display clock transition timing information
ARM926EJS.\${rm_operating_condition}.itiming	Display output port timings
ARM926EJS.\${rm_operating_condition}.otiming	Display input port timings
ARM926EJS.\${rm_operating_condition}.rtiming	Display register-to-register timing
ARM926EJS.tme.\${rm_operating_condition}.accuracy	Display results of compare_interface_timing

Chapter 8

Functional Verification of Netlist

There are several possible flows you can use to verify that the functionality of the post-synthesis and post-layout netlists match that of the RTL reference. This chapter provides guidelines for functional verification of the netlist. It contains the following sections:

- *About netlist verification* on page 8-2
- *Verification using equivalence checking tools* on page 8-3
- *Netlist verification preparation* on page 8-11
- *Functional verification in conjunction with the macrocell RTL model* on page 8-12
- *Verification using test vectors* on page 8-15
- *Replaying test vectors in a Verilog environment* on page 8-18.

8.1 About netlist verification

This chapter describes the recommended approach to verify your post-synthesis and post-layout netlists.

ARM Limited strongly recommends that you use formal equivalence tools to verify your netlist. Equivalence checking provides a complete method to verify your netlist and does not require the extensive simulation compute resources that functional verification requires. For a description of the equivalence checking flow, see *Verification using equivalence checking tools* on page 8-3.

You can perform equivalence checking on your post-synthesis or post-layout netlists:

- You must check the post-synthesis netlist against the RTL.
- You can check the post-layout netlist against the post-synthesis netlist if you have checked that netlist against the RTL. Otherwise, you must check the post-layout netlist against the RTL.

If you are unable to perform formal equivalence checking, see Chapter 10 *Sign-Off*.

You must perform functional checks of your intermediate and final netlists using the instruction execution vectors provided (see *Verification using test vectors* on page 8-15). These vectors are not suitable for netlist sign off without additional verification being performed.

You can also run a selection of tests from the validation suite on your netlist. There is no specifically defined subset of the validation suite that must be run on a netlist, because running validation tests is not a part of the recommended sign off verification flow.

To run validation tests on a netlist, see *Functional verification in conjunction with the macrocell RTL model* on page 8-12. With this method you simulate the netlist and RTL in parallel, with both models supplied with the same inputs, and the outputs of both models compared in each cycle for equivalence. You can run the entire validation suite, but this takes a long time because netlist simulations are several times slower than RTL simulations.

8.2 Verification using equivalence checking tools

This section provides information on how to perform equivalence checking of the macrocell Verilog or VHDL RTL and gate-level netlists. It contains:

- *Equivalence checking*
- *Equivalence checking using Formality* on page 8-6
- *Equivalence checking using Verplex* on page 8-8.

8.2.1 Equivalence checking

Equivalence checkers use formal mathematical techniques to verify logic functions between two implementations of a design. This enables you to determine if the design functionality is consistent between the two implementations. You must maintain the functionality of the configured macrocell at each stage of the design process. You can use equivalence checkers to verify that the functionality of the RTL is maintained through successive iterations of the netlist by a process of:

- *Building the design* on page 8-4
- *Mapping the design* on page 8-5
- *Comparing the design* on page 8-5.

You must also consider the design hierarchy, scan chains, and clock gating when you check the equivalence of the macrocell, see *Other considerations* on page 8-5.

Figure 8-1 on page 8-4 shows the supported equivalence checking flow.

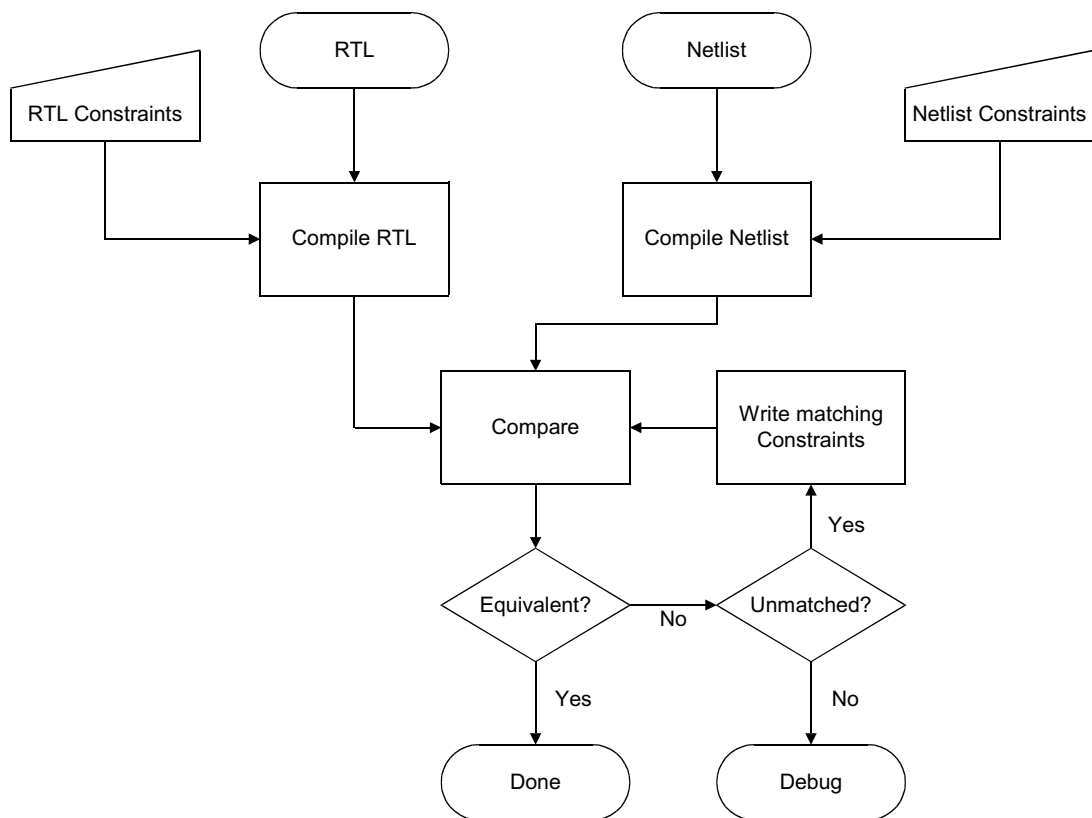


Figure 8-1 Equivalence checking flow

Building the design

To equivalence check the macrocell design that includes memory models at each stage of the design flow, you must instruct the equivalence checker to black-box the memory models from the RTL to netlist comparison. The design process treats memory models as hard macros because they are not synthesized.

The equivalence checker must read in the:

- configured and validated Verilog RTL, along with its memory models, and functional models of the instantiated GTECH components and DesignWare components
- synthesized Verilog netlist, memory models, and cell library.

Black-boxing the memory models has the following effect:

- outputs from the memory models are seen as extra design inputs, that can be set to any value when driving design logic
- inputs to the memory models are seen as extra design outputs, making them part of the equivalence check.

This ensures that the remaining logic is correctly equivalence checked.

Mapping the design

When you have built the Verilog RTL and netlist, the equivalence checker must map the key points in each design. A key point can be:

- Primary inputs
- Primary outputs
- D Flip-Flops
- D latches
- Black boxes.

Comparing the design

After mapping the design the equivalence checker verifies that the logic that drives each compare point is functionally equivalent between the two designs. A compare point can be:

- Primary outputs
- D input of D Flip-Flops or D latches
- Black box inputs.

To prove equivalence between the two designs, the equivalence checker must prove equivalence of every compare point.

Other considerations

When equivalence checking the macrocell, you must consider:

Hierarchical compare versus flat compare

Hierarchical compare is the process of taking two hierarchical designs and comparing them at the module level from the bottom-up. When the equivalence checker has successfully compared a low-level design module, you can black-box that particular module from the design. The compare process can then continue with the next level up.

A hierarchical compare requires that both designs have similar hierarchy. This approach can significantly reduce the time required to verify large designs, and it is recommended for RTL to gate level comparisons.

A flat compare simply performs the compare process at the top level of the two designs.

A flat compare might be required if the two designs are different, for example, if you flatten the netlist during synthesis. You can use flat comparisons to compare two gate level designs where the differences between the two designs are nonfunctional and typically include:

- insertion of scan chains
- insertion of buffer trees for high fanout nets
- re-ordering of the scan chains
- post-layout cell resizing.

Scan Chains

When the design uses scan chains you can add new input and output pins, depending on the scan methodology. The additional pins are typically the scan input, output, and enable pins. Because these additional pins can cause problems with the equivalence checker, when it maps the design, you must instruct the equivalence checker to ignore these compare points.

You must also ensure that the scan chains are disabled by turning off the scan enable signal (**SCANENABLE** set to 0).

Clock Gating

When the synthesis flow uses clock gating, you must ensure that the appropriate mode is set in the equivalence checker.

8.2.2 Equivalence checking using Formality

This section provides information on:

- *Setting up the script* on page 8-7
- *Running Formality equivalence checking* on page 8-7
- *Checking results* on page 8-8.

The version of Formality required for macrocell equivalence checking is 2003.03-1. You do not require simulation vectors because the equivalence checking tools use static techniques to verify the functionality of the macrocell.

Setting up the script

The arm926ejs_fm.tcl script, located in the synopsys/build_dir/scripts directory, controls the macrocell equivalence checking flow. Table 8-1 describes the variables used in this script. You must ensure that these variables are set in the options script to your requirements.

Table 8-1 ARM926EJS_fm.tcl variables

Variable	Default	Description
rm_clock_gating	1	Informs Formality that the design uses clock gating for power saving when set to 1.
rm_vhdl_rtl	0	Informs Formality to use VHDL RTL when set to 1.

————— **Note** —————

You must ensure that the hdlin_dwroot variable references your Synopsys site installation and that you read in your technology library.

The command read_verilog or read_vhdl reads the RTL in to the reference, r, container:

```
if { $rm_vhdl_rtl } {  
    set hdlin_enable_rtlc_vhdl true  
    set hdlin_warn_on_mismatch_message [list FMR_VHDL-1027]  
    source -echo -verbose ./scripts/arm926ejs_vhdl.tcl  
    read_vhdl -container r -libname WORK $rtl_image  
} else {  
    source -echo -verbose ./scripts/arm926ejs_verilog.tcl  
    read_verilog -container r -libname WORK $rtl_image  
}
```

The command read_verilog reads the netlist in to the implementation container, i:
read_verilog -libname WORK -netlist -c i ./synthesis/data/ARM926EJS.v

Running Formality equivalence checking

You must invoke Formality equivalence checking from the synopsys/build_dir directory:

```
make fm
```

To view the status of equivalence checking use the command:

```
tail -f ./log/fm
```

It is recommended that you invoke Formality using command line mode and save the transcript for analysis. However, you can run Formality equivalence checking in graphical mode by sourcing the script from within the *Graphical User Interface* (GUI).

Checking results

To check the results:

1. Check the log file `fm` for errors. If errors exist, you must correct them and rerun the script.
Equivalence checking passes if the log shows `Verification SUCCEEDED` at both the multiplier and the macrocell level:
2. If equivalence fails the following files appear in the `./synthesis/reports` directory giving reasons why:
 - `ARM926EJS-failed.fm`
 - `ARM926EJS-matched.fm`
 - `ARM926EJS-unmatched.fm`
 - `ARM926EJS-candidates.fm`.
 Use this information to help debug the problem.
3. Check that the variables in the `arm926ejs_fm.tcl` and options file are set to your requirements.

8.2.3 Equivalence checking using Verplex

This section provides information on:

- *Setting up the scripts*
- *Running Verplex equivalence checking* on page 8-9
- *Checking results and trouble-shooting* on page 8-10.

Setting up the scripts

Table 8-2 shows the files required for Verplex equivalence checking of the macrocell design, and their location.

Table 8-2 Verplex equivalence checking files

File	Description	Location
<code>uniquifyallmodules.tcl</code>	This is the Verplex-sourced procedure that performs uniquifying on all the modules in the macrocell design	<code>verplex/scripts/tcl</code>
<code>ARM926EJS-tsmc_sicg.vc</code>	This is the Verilog command file used to read in the Verilog netlist	<code>verplex/scripts/vc</code>
<code>ARM926EJS-rtl.vc</code>	This is the Verilog command file used to read in the Verilog RTL	<code>verplex/scripts/vc</code>

To set up the Verplex equivalence checking flow, you must set up the following files :

- *926-rtl-vs-tsmc_sicg.do*
- *ARM926EJS-tsmc_sicg.vc*
- *ARM926EJS-rtl.vc*.

926-rtl-vs-tsmc_sicg.do

Table 8-3 describes the variables to set in the *926-rtl-vs-tsmc_sicg.do* script. You must ensure that these variables are set to your requirements.

Table 8-3 926-rtl-vs-tsmc_sicg.do

Variable	Default	Description
ROOT_MODULE	ARM926EJS	Define the module to be compared.
GOLDEN	rtl	Define the golden design to be RTL.
REVISED	tsmc_sicg	Define the revised design to be netlist.
G_DES_TYPE	Verilog	Define the language of your golden design.
G_LIBERTY_LIB	None	Define the path to your GTECH components.
R_DES_TYPE	Verilog	Define the language of your revised design.
R_LIBERTY_LIB	None	Define the path to the functional model of the cell library.

ARM926EJS-rtl.vc

Update this file to read in your Verilog RTL, changing the paths to your memory models and GTECH components. You must also set your cache size here.

ARM926EJS-tsmc_sicg.vc

Update this file to read in your Verilog netlist, changing the paths to your memory models and Verilog cell library models.

Running Verplex equivalence checking

The Verplex equivalence checker runs from the `./verplex` directory. To run the equivalence checker, type:

```
lec -dofile scripts/do/926-rtl-vs-tsmc_sicg.do -nogui
```

To monitor the Verplex equivalence checking process, type:

```
tail -f log/ARM926EJS-Verilog-rtl-vs-tsmc_sicg.log
```

Checking results and trouble-shooting

The results of the Verplex equivalence checking process are located in:

verplex/log/ARM926EJS-Verilog-rtl-vs-tsmc_sicg.log

If there are errors in the ARM926EJS-Verilog-rtl-vs-tsmc_sicg.log you must fix them before running the tool again.

1. Equivalence checking passes if all the modules compare correctly. The log must show that no modules were non-equivalent, aborted or uncomparing. For example:

```
// Command: report hier_compare result
Total Equivalent modules      = 71
// Command: report hier_compare result -Non_equivalent
// Command: report hier_compare result -Abort
// Command: report hier_compare result -Uncomparing
```

Note

The total number of equivalent modules can vary depending on the execution of the hierarchical compare.

2. If there are non-equivalent, aborted, or uncomparing modules, you must check that the design maps correctly. If the following message appears in the ARM926EJS-Verilog-rtl-vs-tsmc_sicg.log:
Warning: More than 1/3 of the key points have mis-matched names
This message can indicate that the design does not map correctly, but the equivalence checker might still correctly map those key points.
Examine the key point names in the RTL and the netlist and, if required, update the naming rules in the 926-rtl-vs-tsmc_sicg.do file.
3. If the design maps correctly, and there are non-equivalent, aborted, or uncomparing modules, and boundary optimization is switched on, you might have to apply the add no black box command to the modules that fail equivalence checking.

8.3 Netlist verification preparation

Some of the flows described in this chapter use Perl scripts that are supplied with the model. Some changes to environment variables are required to ensure these scripts can run correctly.

Ensure your PATH environment variable includes the `verilog/tools/bin` subdirectory, and the PERL5LIB environment variable includes the `verilog/tools/lib/perl5` subdirectory.

You can use these commands to make changes if you use the C shell:

- `% setenv PATH ${PATH}:/<my_arm926ejs>/verilog/tools/bin`
- if you already have a PERL5LIB environment variable:
`% setenv PERL5LIB ${PERL5LIB}:/<my_arm926ejs>/verilog/tools/lib/perl5`
 otherwise:
`% setenv PERL5LIB /<my_arm926ejs>/verilog/tools/lib/perl5`

Where `<my_arm926ejs>` is the name of the directory where you installed the macrocell deliverables.

The supplied scripts assume that the Perl binary is in the `/usr/local/gnu/bin/perl` directory. If your site has Perl installed in a different location, edit the first line of the scripts to point to the correct location. For example, if Perl is available at `/usr/bin/perl`, change:

```
#!/usr/local/gnu/bin/perl -w
```

to read:

```
#!/usr/bin/perl -w
```

8.4 Functional verification in conjunction with the macrocell RTL model

With this flow, you simulate the synthesized netlist in conjunction with the macrocell RTL within the macrocell validation environment. You apply the core inputs to both designs, and compare the outputs of the netlist with the RTL at the end of each cycle. Using this flow, you can run any of the validation tests on the netlist while checking that the outputs from the netlist are identical to the RTL reference. This approach gives the cycle comparison benefits of extracting test vectors, without the overhead of extracting and replaying vector files. The cosimulation flow also eases debugging problems with the netlist, because the RTL reference is available within the same simulation.

The primary step in this verification flow is to generate a test harness that contains both the RTL and netlist versions of the macrocell design. Use the `verilog_testgen` script to generate this test harness.

8.4.1 Verilog testgen

The `verilog_testgen` script creates a test harness that uses the Verilog netlist of the design to be tested, and the top-level Verilog file of the RTL reference for design. The `verilog_testgen` script creates a new Verilog file, based on these two input files, that contains:

- The module described in the RTL reference file with the module name modified so that it has `_ref` as an extension.
- All the modules described in the netlist file with the module names modified so that they have `_uut` as an extension.
- A new, checking, module description with the same name as the module in the RTL reference file. The new module instantiates the modified RTL reference design, the modified netlist design and a checking module that compares the outputs of the netlist design against the RTL reference design at the end of each clock cycle.

The `verilog_testgen` script modifies the names of the reference and test modules so that there is no name-space conflict when the design is compiled. The checking module description has the same module name and port description as the RTL reference. This means that you can use the checking module as a drop-in replacement for the RTL reference in the validation environment.

If the test module has inputs, such as scan inputs, that are not present on the reference module, they are tied LOW (0).

If the test module has outputs, such as scan outputs, that are not present on the reference module, they are not checked.

If an output of the reference module is X in a given cycle, the corresponding output of the test module is not checked during that cycle.

Usage

```
verilog_testgen
  -ref <ref_file>
  -test <test_file>
  -top <top_module_name>
  -clk <clock_name>
  [-rst <reset_name:reset_level>]
  [-nofail]
  [-help]
```

-ref Specify the Verilog file containing the reference module to compare against.

-test Specify the Verilog file containing the test module that is to be checked.

-top Specify the top-level module name of the design.

-clk Specify the name of the clock input for the system.

Note

The verilog_testgen script can handle only designs with a single clock input.

-rst Specify a reset input, and its active level. Output comparisons are disabled if the reset input is in its active state.

-nofail Specify that the simulation continues even if an output mismatch occurs. The default behavior is to stop the simulation immediately if a mismatch occurs.

Note

- The verilog_testgen script generates a file named <ref_file>_check.v.
 - Before running verilog_testgen ensure your environment is set up as described in *Netlist verification preparation* on page 8-11.
-

8.4.2 Verifying the macrocell design

To verify a netlist of the macrocell design:

1. Ensure that the MAIN_ARM926EJS directory is set up as described in *Integrating your RAM blocks* on page 2-22 and corresponds to the netlist being verified.
2. Change directory to the verilog/MAIN_ARM926EJS directory and type the following command:

```
verilog_testgen -ref ARM926EJS.v -test <verilog_netlist> -top ARM926EJS -clk CLK -rst HRESETn:0
```

where <verilog_netlist> includes the full path and filename of the Verilog netlist being verified. The script generates a file named ARM926EJS_check.v.

3. Modify the verilog/tbenchfpga/tbench926.vc, to compile the netlist and checker module for simulation. It is advisable to make a backup of this file before making the modifications to enable you to rebuild the standard RTL model if necessary.

When the backup has been made, add lines to reference the standard cell models used by the netlist.

In addition, change:

```
-v verilog/MAIN_ARM926EJS/ARM926EJS.v
```

to:

```
-v verilog/MAIN_ARM926EJS/ARM926EJS_check.v
```

4. Rebuild the model for your chosen simulator as described in *Building the model* on page 3-10.
5. Rename, or backup, the existing validation logs otherwise they will be overwritten.
6. Change directory to the validation directory and run validation tests as described in *Running the RTL validation* on page 3-12.

The following message is displayed at the beginning of the simulation to indicate that the test harness is active:

```
TESTGEN COMPARISON ACTIVE
```

If an output comparison fails an error message is displayed and the simulation terminates. An example failure message is:

```
TESTGEN COMPARISON FAILURE: PORT InMREQ: CYCLE 10332: TIME 155344.5ns  
ref = 0  
uut = 1
```

Note

If your RAM model performs setup and hold timing checks, you must disable the checks using the -simopts"+notimingchecks" option when running validation.

8.5 Verification using test vectors

The purpose of the test vectors is to enable you to perform quick netlist checks prior to performing formal or functional verification. The vector capture uses the source RTL as a reference, so vector replay ensures that your netlist matches the cycle-by-cycle behavior of the RTL. The test vectors only cover a small amount of macrocell functionality and are not suitable for netlist sign-off without additional verification.

8.5.1 Test vectors directory structure

The macrocell test vectors are located in three main directories:

verilog/tools Contains tools for converting the supplied vectors into a format suitable for Verilog simulation. Figure 8-2 shows its hierarchy.

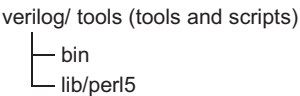


Figure 8-2 Verilog tools directory

vectors Contains the reference vectors. Figure 8-3 shows its hierarchy.

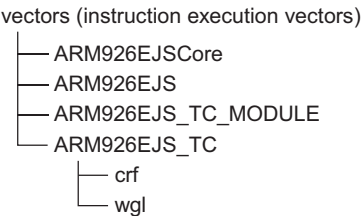


Figure 8-3 Vectors directory

verilog Contains the Verilog source for the macrocell and a Verilog testbench for replaying the test vectors. Figure 8-4 on page 8-16 shows its hierarchy.

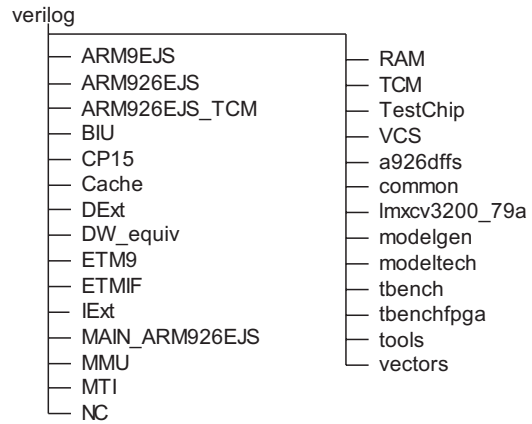


Figure 8-4 Verilog testbench directory

8.5.2 Test vector files

For each test vector there are two associated files:

- the vector extraction log file, that contains a log of the original vector extraction, `<test_vector>.capture.Z`
- the vector replay log file, that contains a log of the replay of the vectors on the source RTL, `<test_vector>_verilog.replay.Z`.

All files are compressed using the UNIX compress tool.

8.6 Replaying test vectors in a Verilog environment

This section describes how you can use the supplied testbench, ARM926EJS_tb_verilog.v to replay test vectors under Verilog simulation. The testbench is located in the verilog/vectors/ARM926EJS directory.

Note

The flow is the same for test vector replay on all other levels of hierarchy.

8.6.1 Replaying test vectors on the RTL

You must use the following steps to replay the test vectors on the RTL using the ModelTech tools in the verilog/vectors/ARM926EJS directory:

1. At the ARM926EJS level there are vectors for several cache sizes. Therefore, this step is only required when replaying vectors on the ARM926EJS.

Go to your Verilog/vectors/ARM926EJS directory. Edit the dcachesize and icachesize entries in the Makefile to match the cache sizes you are using.

Note

The supplied script, verilog/tools/bin/Crf2Vrf makes a reference to /usr/local/gnu/bin/perl. You must change this to point to your site-specific Perl installation.

2. Set up the environment variable A926VLOG as the path to your verilog directory, for example:

```
setenv A926VLOG /<root_dir>/<my_arm926ej-s>/verilog
```

3. Set up the paths to the required tools:

```
source dotcshrc
```

4. Replay the vectors, type:

```
make do_tests
```

Note

- It is possible to replay a single test as follows:
make <testname>.test
If you are using a simulator other than Modeltech then you must modify the file, Makefile.common.
 - If your RAM model performs setup and hold timing checks, you must disable the checks, by adding the +notimingchecks option to the vsim command in Makefile.common.
-

8.6.2 Replaying test vectors on the Netlist

To replay the test vectors on the netlist using the ModelTech tools in the `verilog/vectors/ARM926EJS` directory:

1. At the ARM926EJS level there are vectors for several cache sizes. Therefore, this step is only required when replaying vectors on the ARM926EJS.
Go to your `Verilog/vectors/ARM926EJS` directory. Edit the `dcachesize` and `icachesize` entries in the `Makefile` to match the cache sizes you are using.
2. Set up the environment variable `A926VLOG` as the path to your Verilog directory, for example:

```
setenv A926VLOG /<root_dir>/<my_arm926ej-s>/verilog
```
3. Set up the paths to the required tools:

```
source dotcshrc
```
4. Compile the netlist and vector replay testbench. For example, using the ModelTech tools:

```
vlib work  
vlog -f netlist.vc ARM926EJS_tb_verilog.v
```


Where `netlist.vc` contains the paths to the Verilog netlist and your Verilog library simulation models.
5. To replay the vectors, type:

```
make do_tests
```

8.6.3 Checking the vectors pass

The testbench prints a summary at the end of the simulation reporting any errors encountered in the replay of the vectors.

————— Note —————

All of the supplied vectors must replay with zero errors.

—————

Chapter 9

Production Test

This chapter describes how to do production testing for the macrocell. It contains the following sections:

- *About production test* on page 9-2
- *Test flow with non-Synopsys test tools* on page 9-3
- *Macrocell test ports* on page 9-4
- *Test wrapper insertion* on page 9-5
- *Performing ATPG using TetraMAX* on page 9-6.

9.1 About production test

Applying *Automatic Test Pattern Generation* (ATPG) vectors is the industry standard way of production testing complex SoCs. Scan chains, that are inserted into the design, apply the ATPG vectors to the design under test.

The macrocell is a fully synchronous design with all registers clocked off the rising clock edge, and therefore you can achieve a very high test coverage. The macrocell design flow uses Synopsys test tools to perform scan ready synthesis, scan chain insertion, and to produce ATPG vectors.

This chapter describes:

- the macrocell test features that test the macrocell and its surrounding environment
- how to perform ATPG.

If you do not use Synopsys test tools, see *Test flow with non-Synopsys test tools* on page 9-3.

9.2 Test flow with non-Synopsys test tools

If you do not use Synopsys test tools, you cannot perform scan ready and scan chain inserted synthesis. You must use your own test tools to perform scan chain insertion and produce ATPG vectors.

Before performing scan chain insertion, you must read *Macrocell test ports* on page 9-4. This describes the macrocell test ports that you can use for scan insertion.

9.3 Macrocell test ports

Table 9-1 describes the macrocell test ports.

Table 9-1 Macrocell test ports

Port	Direction	Description
TESTMODE	Input	This signal ensures the clocks are free-running during scan test. TESTMODE must be: • tied HIGH throughout the duration of scan testing • tied LOW during functional mode. If you have to add any test logic to the macrocell, for example to test your RAMs, you can use the TESTMODE signal to control your test logic.
SCANENABLE	Input	This signal enables serial shifting of vectors through the scan chains. This signal must be tied LOW during functional mode.
SI	Input	Inputs to scan chains.
SO	Output	Outputs from scan chains.

————— **Note** —————

- **SI, SO, WRP_SI, and WRP_SO** do not exist as ports in the macrocell RTL. These ports are automatically generated during scan ready synthesis and exist in your synthesized netlist.
- Because the design no longer uses the **INTEST** and **EXTEST** wrapper, the signals **INTEST** and **EXTEST** are tied to zero by the `set_logic_zero` command in the `arm926ejs_compile.tcl` script.

9.4 Test wrapper insertion

A test wrapper is provided that you can use to gain access to the inputs and outputs of the macrocell in production test. This gives increased test coverage when access to the primary inputs and outputs is impossible because the macrocell is deeply embedded in your SoC design.

All of the I/O pins are wrapped except those shown in Table 9-2.

Table 9-2 Pins that are not wrapped by the test wrapper

Port	Direction	Description
WRP_CLOCK	Input	Wrapper clock. Clocks the sequential elements in the wrapper cell.
WRP_SHIFT	Input	Wrapper shift. Enables shifting through the sequential element of the wrapper cell.
WRP_SAFE	Input	Wrapper safe mode enable. Only used if a safe wrapper cell is used. Currently untested.
WRP_IF	Input	Wrapper inward facing mode enable.
WRP_OF	Input	Wrapper outward facing mode enable.
WRP_SI	Input	Inputs to wrapper chains.
WRP_SO	Output	Outputs from wrapper chains.

The synthesis script, described in Chapter 5 *Synthesis*, produces a macrocell with either:

- no test wrapper
- a test wrapper using Synopsys SoCTest.

Note

The majority of the macrocell I/Os are registered. Synopsys SoCTest is currently unable to reuse these registers but shared wrapper support will be in future releases of the tool. This produces a small overhead in registers.

9.5 Performing ATPG using TetraMAX

As part of the synthesis flow, the `arm926ejs_physopt.tcl` script generates a test model and protocol files for the macrocell. The `rm_create_test_model` variable, in the options file, controls this process. This is similar to the way in which the `rm_create_test_model` variable controls wrapper insertion.

To generate the necessary test patterns for the completed test model, use TetraMAX to generate them automatically. You can use the example script, `arm926ejs_tmax.tcl` located in the `synopsys/build_dir/scripts` directory, to run TetraMAX. TetraMAX runs on your scan inserted netlist.

The test model, that the `physopt` script writes out, includes 4 test modes:

Internal_scan

This mode only deals with the internal scan chains of the macrocell and ignores the wrapper. You might notice reduced fault coverage, when you generate patterns for this mode, because of logic that exists between the wrapper and the pins of the macrocell. You cannot test this logic in this mode.

Mission_mode This is the normal functional mode of the macrocell.

WRP_IF This is the internal wrapper test mode and you use it to test the core and isolate the surrounding user defined logic. You can use this mode to apply input vectors to the core and observe the core response at the output.

WRP_OF This is the external wrapper test mode and you can use it to test the user defined logic surrounding the core and isolate the core itself.

The default script only generates patterns for the internal test, or `WRP_IF` mode. The `arm926ejs_tmax.tcl` script includes example code to generate the `Internal_scan` patterns, but this code is commented out.

Historically, the logic between the last register in the macrocell and the RAM caused test shadows surrounding the RAM blocks. TetraMAX can resolve these test shadows because it can read and write into the RAM blocks. Example TetraMAX RAM models are located in the `synopsys/build_dir/tmax_rams` directory.

The example TetraMAX script, `arm926ejs_tmax.tcl`, currently only supports the stuck-at fault model.

Before running the TetraMAX script, you must change the 'read netlist' commands to point to:

- the Verilog library for the target process
- the TetraMAX RAM models for the RAM blocks used in your macrocell design
- your macrocell netlist.

Note

You might have to change the 'set atpg' command to meet your requirements.

To run TetraMAX, from the synopsys/build_dir directory, use the command:

```
make tmax
```

TetraMAX writes its output to the logfile, tmax, in the synopsys/build_dir/logs directory. For more details on the make command, see *Makefile* on page 5-12.

Chapter 10

Sign-Off

This chapter describes the sign-off checks that you must do in addition to your normal *System-on Chip* (SoC) flow sign-off checks. You must satisfy certain verification criteria before you sign off the macrocell design. This chapter describes the sign-off criteria. It contains the following sections:

- *RTL integration phase* on page 10-2
- *Post synthesis, and post place and route phases* on page 10-3.

10.1 RTL integration phase

It is strongly recommended that you verify the macrocell RTL deliverables before you begin the synthesis stage. To verify the RTL run the entire macrocell validation suite on the configured model. This confirms that the macrocell RTL has been successfully installed.

———— **Note** ————

RTL validation must be completed on your configured RTL before you can sign off your design.

————

10.2 Post synthesis, and post place and route phases

You must verify the functionality and timing of your placed and routed netlist.

10.2.1 Functional signoff

You must verify the functionality of the final placed-and-routed netlist before you sign off the macrocell design. Two alternative methods are supported:

- Logical equivalence checking of netlist to configured RTL as described in *Verification using equivalence checking tools* on page 8-3 and functional test vector replay as described in *Verification using test vectors* on page 8-15.
- Functional test vector replay as described in *Verification using test vectors* on page 8-15, followed by *Functional verification in conjunction with the macrocell RTL model* on page 8-12 by running the validation test suites. (See *RTL validation test suites* on page 3-4.)

It is strongly recommended that you use the first of these methods. You can perform equivalence checking of the post-place and route netlist to RTL in stages, for example, by verifying equivalence of the post-synthesis netlist to RTL and the post-place and route to post-synthesis netlists.

Note

Running the entire validation suite on the netlist can take a long time. It also consumes a considerable amount of CPU resource.

10.2.2 Timing signoff

You must verify the timing of the final placed-and-routed netlist using STA, before you sign off the netlist. Extra confidence might be gained by replaying test vectors with timing back-annotated.

Sign-Off

Appendix A

Synthesizable Register File Replacement

This appendix describes the ARM926EJ-S synthesizable register file replacement. It contains the following sections:

- *About synthesizable register file replacement* on page A-2
- *Replacing the synthesizable register file* on page A-3.

A.1 About synthesizable register file replacement

The default implementation of the ARM9EJ-S register file uses an HDL description to synthesize the register file using discrete D-type registers. If a suitable macrocell is available, you can replace the synthesized register file. This results in significant area and power improvements.

———— **Note** ————

If you replace the synthesized register file, you must run all of the RTL validation suites to ensure that you have integrated your register file correctly.

—————

A.2 Replacing the synthesizable register file

This section describes where the ARM9EJ-S register file is located in the hierarchy, and the requirements that a macrocell replacement must satisfy for you to replace the synthesized register file.

A.2.1 a9ejsRegBank

The a9ejsRegBank is the top level of the ARM9EJ-S register file, and is instantiated inside the a9ejsCoreDP block. The a9ejsRegBank block contains register decode logic, forwarding multiplexor logic, and an instantiated register file block, uREGBANKMEM (a9ejsRam3r2wSDff).

a9ejsRam3r2wSDff

The a9ejsRam3r2wSDff block is a synchronous 3-read-port, 2-write-port register file, containing thirty 32-bit registers. Table A-1 shows the interface. The function of the a9ejsRam3r2wSDff is best understood by examining the RTL.

Table A-1 a9ejsRam3r2wSDff interface description

Signal	Direction	Description
CLK	Input	Clock
Mode[3:0]	Input	Core mode
Wr0	Input	Write port 0 enable
Wr1	Input	Write port 1 enable
Wr0Addr[4:0]	Input	Write port 0 register address select
Wr1Addr[4:0]	Input	Write port 1 register address select
Wr0Data[31:0]	Input	Write port 0 data
Wr1Data[31:0]	Input	Write port 1 data
Rd0Addr[4:0]	Input	Read port 0 register address select
Rd1Addr[4:0]	Input	Read port 1 register address select
Rd2Addr[4:0]	Input	Read port 2 register address select
Rd2FUser	Input	Force user mode read from C port
Rd0Data[31:0]	Output	Read port 0 data

Table A-1 a9ejsRam3r2wSDff interface description (continued)

Signal	Direction	Description
Rd1Data[31:0]	Output	Read port 1 data
Rd2Data[31:0]	Output	Read port 2 data
NewPC[31:0]	Input	New value of PC
ZeroABus1	Input	Zero A bus bit 1

Register address decoding

The register address selects (**Wr0Addr**, **Wr1Addr**, **Rd0Addr**, **Rd1Addr**, and **Rd2Addr**) are 5-bit encoded signals. There are only 30 registers required, so encodings 0x0 to 0x1D only are used. Encodings 0x1E and 0x1F are not generated by the control logic during valid operation.

Write ports

There are two write ports to the register file. Each port is controlled by a register write address (**WrXAddr**) and a register write enable (**WrX**), and has a register write data port (**WrXData**). The write port interface is set up to the rising edge of **CLK**. If the write enable is asserted (HIGH) at the rising edge of **CLK**, the register addressed by the register address has the data presented on the register write data port written into it.

If an attempt is made to write to the same register from both write ports in the same cycle, the data from **Wr0Data** is written.

Read ports

There are three read ports from the register file. Each port is controlled by a register read address (**RdXAddr**), and has an associated register read data port (**RdXData**). The read ports are combinational, the register address is decoded and the addressed register data is presented on the relevant read port. The data read from the read ports has to pass through forwarding array multiplexors, and to be set up to the forwarding array registers in the same cycle that the register read address becomes active.

Glossary

This glossary describes some of the terms used in this manual. Where terms can have several meanings, the meaning presented here is intended.

Advanced High-performance Bus (AHB)

The AMBA Advanced High-performance Bus system connects embedded processors such as an ARM core to high-performance peripherals, DMA controllers, on-chip memory, and interfaces. It is a high-speed, high-bandwidth bus that supports multi-master bus management to maximize system performance.

See also Advanced Microcontroller Bus Architecture and AHB-Lite.

Advanced Microcontroller Bus Architecture (AMBA)

AMBA is the ARM open standard for multi-master on-chip buses, capable of running with multiple masters and slaves. It is an on-chip bus specification that details a strategy for the interconnection and management of functional blocks that make up a *System-on-Chip* (SoC). It aids in the development of embedded processors with one or more CPUs or signal processors and multiple peripherals. AMBA complements a reusable design methodology by defining a common backbone for SoC modules. AHB conforms to this standard.

See also Advanced High-performance Bus and AHB-Lite.

AHB

See Advanced High-performance Bus.

AHB-Lite	AHB-Lite is a subset of the full AHB specification. It is intended for use in designs where only a single AHB master is used. This can be a simple single AHB master system or a multi-layer AHB system where there is only one AHB master on a layer.
AMBA	<i>See</i> Advanced Microcontroller Bus Architecture.
Architecture	The organization of hardware and/or software that characterizes a processor and its attached components, and enables devices with similar characteristics to be grouped together when describing their behavior, for example, Harvard architecture, instruction set architecture, ARMv6 architecture.
ARM state	A processor that is executing ARM (32-bit) instructions is operating in ARM state. <i>See also</i> Thumb state.
Big-endian	Memory organization in which the least significant byte of a word is at a higher address than the most significant byte. <i>See also</i> Little-endian and Endianness.
Block address	An address that comprises a tag, an index, and a word field. The tag bits identify the way that contains the matching cache entry for a cache hit. The index bits identify the set being addressed. The word field contains the word address that can be used to identify specific words, halfwords, or bytes within the cache entry.
Cache	A block of on-chip or off-chip fast access memory locations, situated between the processor and main memory, used for storing and retrieving copies of often used instructions and/or data. This is done to increase the average speed of memory accesses and therefore to increase processor performance.
Cache hit	A memory access that can be processed at high speed because the instruction or data that it addresses is already held in the cache.
Cache line	The basic unit of storage in a cache. It is always a power of two words in size (usually 4 or 8 words), and is required to be aligned to a suitable memory boundary. <i>See also</i> Cache terminology.
Cache line index	The number associated with each cache line in a cache way. Within each cache way, the cache lines are numbered from 0 to (set-associativity) -1. <i>See also</i> Cache terminology.
Cache set	A cache set is a group of cache lines (or blocks). A set contains all the ways that can be addressed with the same index. The number of cache sets is always a power of two.
Cache way	A group of cache lines (or blocks). It is 2 to the power of the number of index bits in size.
Cast out	<i>See</i> Victim.

Central Processing Unit (CPU)

The part of a processor that contains the ALU, the registers, and the instruction decode logic and control circuitry. Also commonly known as the processor core.

Clean

A cache line that has not been modified while it is in the cache is said to be clean. To clean a cache is to write dirty cache entries into main memory. If a cache line is clean, it is not written on a cache miss because the next level of memory contains the same data as the cache.

See also Dirty.

Clock gating

Gating a clock signal for a macrocell with a control signal, and using the modified clock that results to control the operating state of the macrocell.

Cold reset

Also known as power-on reset. Starting the processor by turning power on. Turning power off and then back on again clears main memory and many internal settings. Some program failures can lock up the processor and require a cold reset to enable the system to be used again. In other cases, only a warm reset is required.

See also Warm reset.

Coprocessor

A processor that supplements the main CPU. It carries out additional functions that the main CPU cannot perform. Usually used for floating-point math calculations, signal processing, or memory management.

Core reset

See Warm reset.

CPU

See Central Processing Unit.

Data Cache (DCache)

A block of on-chip fast access memory locations, situated between the processor and main memory, used for storing and retrieving copies of often used data. This is done to greatly increase the average speed of memory accesses and therefore to increase processor performance.

DCache

See Data Cache.

Direct Memory Access

An operation that accesses main memory directly, without the processor performing any accesses to the data concerned.

Dirty

A cache line in a Write-Back cache that has been modified while it is in the cache is said to be dirty. A cache line is marked as dirty by setting the dirty bit. If a cache line is dirty, it must be written to memory on a cache miss because the next level of memory contains data that has not been updated. The process of writing dirty data to main memory is called cache cleaning.

See also Clean.

DMA	<i>See</i> Direct Memory Access.
Embedded Trace Macrocell (ETM)	A hardware macrocell that outputs instruction and data trace information on a trace port.
Endianness	Byte ordering. The scheme that determines the order in which successive bytes of a data word are stored in memory. <i>See also</i> Little-endian and Big-endian.
ETM	<i>See</i> Embedded Trace Macrocell.
Fast Context Switch Extension (FCSE)	This enables cached processors with an MMU to present different addresses to the rest of the memory system for different software processes even when those processes are using identical addresses.
FCSE	<i>See</i> Fast Context Switch Extension.
ICache	<i>See</i> Instruction Cache.
Implementation- defined	A feature that is not architecturally defined, and which might vary between implementations. The feature is defined and documented for each individual implementation.
Index	<i>See</i> Cache line index.
Index register	A register specified in some load or store instructions. The value of this register is used as an offset to be added to or subtracted from the base register value to form the virtual address, which is sent to memory. Some addressing modes optionally enable the index register value to be shifted prior to the addition or subtraction.
Instruction Cache (ICache)	A block of on-chip fast access memory locations, situated between the processor and main memory, used for storing and retrieving copies of often used instructions. This is done to increase the average speed of memory accesses and therefore to increase processor performance.
Little-endian	Memory organization where the least significant byte of a word is at a lower address than the most significant byte. <i>See also</i> Big-endian and Endianness.
Jazelle architecture	The ARM Jazelle architecture extends the Thumb and ARM operating states by adding a Java state to the processor. Instruction set support for entering and exiting Java applications, real-time interrupt handling, and debug support for mixed Java/ARM applications is present. When in Java state, the processor fetches and decodes Java bytecodes and maintains the Java operand stack.

Macrocell	A complex logic block with a defined interface and behavior. A typical VLSI system comprises several macrocells (such as an ARM processor, an Embedded Trace Macrocell, and a memory block) plus application-specific logic.
Memory bank	One of two or more parallel divisions of interleaved memory, usually one word wide, that enable reads and writes of multiple words at a time, rather than single words. All memory banks are addressed simultaneously and a bank enable or chip select signal determines which of the banks is accessed for each transfer. Accesses to sequential word addresses cause accesses to sequential banks. This enables the delays associated with accessing a bank to occur during the access to its adjacent bank, speeding up memory transfers.
Memory Management Unit (MMU)	Hardware that controls caches and access permissions to blocks of memory, and translates virtual addresses to physical addresses.
MMU	See Memory Management Unit.
Modified Virtual Address (MVA)	A virtual address produced by the integer unit can be changed by the current Process ID to provide a <i>Modified Virtual Address</i> (MVA) for the MMUs and caches.
MVA	See Modified Virtual Address.
PA	See Physical Address.
Physical Address (PA)	The MMU performs a translation on <i>Modified Virtual Addresses</i> (MVA) to produce the <i>Physical Address</i> (PA) which is given to AHB to perform an external access. The PA is also stored in the Data Cache to avoid requiring address translation when data is cast out of the cache.
Power-on reset	See Cold reset.
Processor	A contraction of microprocessor. A processor includes the CPU or core, plus additional components such as memory, and interfaces. These are combined as a single macrocell, that can be fabricated on an integrated circuit.
Read	Reads are defined as memory operations that have the semantics of a load. That is, the ARM instructions LDM, LDRD, LDC, LDR, LDRT, LDRSH, LDRH, LDRSB, LDRB, LDRBT, LDREX, RFE, STREX, SWP, and SWPB, and the Thumb instructions LDM, LDR, LDRSH, LDRH, LDRSB, LDRB, and POP. Java instructions that are accelerated by hardware can cause a number of reads to occur, according to the state of the Java stack and the implementation of the Java hardware acceleration.
Region	A partition of instruction or data memory space.
Register	A temporary storage location used to hold binary data until it is ready to be used.

Scan chain	A scan chain is made up of serially-connected devices that implement boundary scan technology using a standard JTAG TAP interface. Each device contains at least one TAP controller containing shift registers that form the chain connected between TDI and TDO , through which test data is shifted. Processors can contain several shift registers to enable you to access selected parts of the device.
Set-associative cache	In a set-associative cache, lines can only be placed in the cache in locations that correspond to the modulo division of the memory address by the number of sets. If there are n ways in a cache, the cache is termed n -way set-associative. The set-associativity can be any number greater than or equal to 1 and is not restricted to being a power of two.
Tag	The upper portion of a block address used to identify a cache line within a cache. The block address from the CPU is compared with each tag in a set in parallel to determine if the corresponding line is in the cache. If it is, it is said to be a cache hit and the line can be fetched from cache. If the block address does not correspond to any of the tags it is said to be a cache miss and the line must be fetched from the next level of memory.
TAP	<i>See</i> Test Access Port.
Test Access Port (TAP)	The collection of four mandatory terminals and one optional terminal that form the input/output and control interface to a JTAG boundary-scan architecture. The mandatory terminals are TDI , TDO , TMS , and TCK . The optional terminal is TRST .
Thumb instruction	A halfword that specifies an operation for an ARM processor in Thumb state to perform. Thumb instructions must be halfword-aligned.
Thumb state	A processor that is executing Thumb (16-bit) halfword aligned instructions is operating in Thumb state.
VA	<i>See</i> Virtual Address.
Victim	A cache line, selected to be discarded to make room for a replacement cache line that is required as a result of a cache miss. The way in which the victim is selected for eviction is processor-specific. A victim is also known as a cast out.
Virtual Address (VA)	The MMU uses its page tables to translate a Virtual Address into a Physical Address. The processor executes code at the Virtual Address, which might be located elsewhere in physical memory. <i>See also</i> FCSE, MVA, and PA.
Warm reset	Also known as a core reset. Initializes the majority of the processor excluding the debug controller and debug logic. This type of reset is useful if you are using the debugging features of a processor.
Way	<i>See</i> Cache way.

WB	<i>See</i> Write-back.
Write	Writes are defined as operations that have the semantics of a store. That is, the ARM instructions SRS, STM, STRD, STC, STRT, STRH, STRB, STRBT, STREX, SWP, and SWPB, and the Thumb instructions STM, STR, STRH, STRB, and PUSH. Java instructions that are accelerated by hardware can cause a number of writes to occur, according to the state of the Java stack and the implementation of the Java hardware acceleration.
Write-back (WB)	In a write-back cache, data is only written to main memory when it is forced out of the cache on line replacement following a cache miss. Otherwise, writes by the processor only update the cache. (Also known as copyback).
Write-through (WT)	In a write-through cache, data is written to main memory at the same time as the cache is updated.
WT	<i>See</i> Write-through.

Index

The items in this index are listed in alphabetical order. The references given are to page numbers.

A

Antennas in routing 6-11
ATPG 9-6
Automated Test Pattern Generation
 see ATPG

B

Block diagram 1-9
Buffering high fanout nets 5-30
Building and validating 3-7

C

Case analysis, STA 7-11
Clock gating constraints 5-20
Clock tree
 routing 6-9
 synthesis 6-8

Clocks 5-20
 and high fanout nets 5-22
 specifying 7-10
Configuration file
 dotcshrc 3-3
 RTL validation 3-3
Constraining the design 5-20
Constraints
 clock gating 5-20
 design rule 5-21
 port timing 5-21
Conventions
 numerical xv
 signal naming xiv
 typographical xiv
Cross talk analysis 7-11
CTS and routing flow 6-3

D

Data cache data RAM 2-12
Data cache dirty RAM 2-17

Data cache tag RAM 2-14
Data cache valid RAM 2-16
Design context 5-21, 7-10
 operating conditions 5-23
 port characteristics 5-22
Design flow 1-4
Design hierarchy 1-7
Design rule
 check 5-28
Design rule constraints 5-21
Design, constraining 5-20
Design, exporting 5-31
Design, saving 5-30
Development
 platform 1-2
 tools 1-2
Directory structure 1-6
 release 1-7
 synthesis 5-2
 test vectors 8-15
dotcshrc file 3-3

E

- Electromigration
 - checks 6-10
 - in floorplanning 4-5
- Electrostatic discharge 6-12
- Environment setup, validation 3-9
- Equivalence checking
 - building the design 8-4
 - comparing the design 8-5
 - flat and hierarchical design 8-5
 - flow 8-4
 - mapping the design 8-5
 - other considerations 8-5
 - scan chains 8-6
 - tools 8-3
 - using Formality 8-6
 - using Verplex 8-8
- ESD 6-12
- Example clock tree 6-8
- Export design 5-31
- Extracted timing model, verifying 7-14

F

- Floorplan 4-1
 - electromigration 4-5
 - example 4-8
 - guidelines 4-4
 - placement 4-7
 - producing 4-5
 - RAM interface 4-6
 - supported flow 4-3
- Formality equivalence checking
 - running 8-7
 - setting the script up 8-7

G

- Gate level checks 6-10

H

- High fanout net buffering 5-30
- High fanout nets
 - routing 6-9

I

- Instruction cache data RAM 2-7
- Instruction cache tag RAM 2-8
- Instruction cache valid RAM 2-11
- Integrating RAM
 - automatic integration 2-25
 - manual integration 2-22
- Interface timing 7-12

L

- Liberty file conversion 7-13
- Library cell
 - timing model extraction 7-13
 - timing model verification 7-14

M

- Macrocell test ports 9-4
- Makefile 5-12
- Metal fill, routing 6-12
- MMU
 - options 2-6
 - RAM 2-6
- Model
 - building 3-10
 - compiling 3-10
 - wire load 5-23
- Module block diagram 1-9

N

- Naming files in synthesis 5-30
- Netlist verification 8-1
 - directory structure 8-15
 - functional 8-1
 - preparation 8-11
 - test vector replay, Verilog 8-18
 - Verilog testgen script 8-12
- Numerical conventions xv

O

- Operating conditions
 - STA 7-11
 - synthesis 5-23
- Options
 - STA 7-5
 - synthesis 5-8
 - technology 5-10

P

- Parasitic extraction 6-12
- Platform, tools, and deliverables 1-2
- Port
 - characteristics, synthesis 5-22
- Port timing constraints 5-21
- Post-routing 6-10
- PrimeTime script 7-4
- Production test 9-1, 9-2
 - macrocell test ports 9-4
 - test flow, non-Synopsys tools 9-3
 - wrapper insertion 9-5

R

- RAM
 - block integration 2-22
 - byte-write memory 2-18
 - data cache data 2-12
 - data cache dirty 2-17
 - data cache tag 2-14
 - data cache valid 2-16
 - DCBISTA connections 2-34
 - floorplanning 4-6
 - instruction cache data 2-7
 - instruction cache tag 2-8
 - instruction cache valid 2-11
 - integrated test logic 2-34
 - integration 2-1
 - integration testbench 2-29
 - large memory from smaller RAM
 - blocks 2-19
 - manual integration 2-22
 - MMU 2-6
 - optimizing for power 2-19
 - optimizing for speed 2-20

RAM (continued)
 options, instruction valid 2-11
 organization 2-4
 requirements 2-2
 supported integration flow 2-3
 test logic 2-31
 test ports 2-32
 Release directory structure 1-7
 Reports
 STA 7-17
 synthesis 5-34
 Routing
 clock trees 6-9
 metal fill 6-12
 signals 6-9
 RTL analysis 5-15
 RTL validation
see validation

S

Scan chains
 configuration 5-27
 equivalence checking 8-6
 insertion 5-27
 stitching 5-28
 Scan ports, macrocell 9-4
 Script file
 constraints 5-7
 generation, validation 3-25
 physical optimization 5-7
 technology 5-8
 usage, validation 3-19, 3-22
 SDF and post-layout optimization 7-12
 Signal naming conventions xiv
 Signals, routing 6-9
 Sign-off 10-1
 functional 10-3
 integration phase 10-2
 timing 10-3
 Silicon revision 1-10
 Simulator and platform support 3-8
 STA 7-2
 case analysis 7-11
 control options 7-6
 controlling 7-9
 flow, supported 7-3
 options script 7-5

ST (continued)
 outputs 7-17
 reading in design 7-9
 reports 7-17
 running 7-7
 setup 7-6
 sign-off model interconnect 7-10
 technology script 7-5
 timing 7-12
 Static timing analysis
see STA
 Synopsys
 directories 7-17
 Synthesis 5-1
 control file 5-6
 control summary 5-12
 design, reading 5-15
 directory structure 5-2
 file to control 5-6
 flow, supported 5-4, 5-5
 naming conventions 5-30
 optimization 5-25
 options 5-8
 process 5-15
 reports 5-34
 RTL analysis 5-15
 running 5-12, 5-14
 script associations 5-6
 setup 5-9
 Synthesizable register file, replacing
 A-2

T

Technology
 library 5-8, 5-9
 options 5-10
 parameters 5-9
 script 5-8
 script, STA 7-5
 Test clocks 5-27
 Test ports 9-4
 Test suites 3-12
 validation 3-4
 Test vectors
 files 8-17
 simulation world, building 8-18
 verification 8-15
 Verilog environment, replay in 8-18

Test wrapper
 insertion 5-27, 9-5
 Testbench, RAM integration 2-29
 Timing
 exceptions, STA 7-11
 library cell, STA 7-13
 STA 7-12
 Timing model
 generation 7-8
 qualification script 7-4
 Tools 1-2
 synthesis 5-3
 validation 3-2
 Typographical conventions xiv

U

Uniquifying the hierarchy 5-17

V

Validation 3-2
 configuration files 3-3, 3-5
 debugging 3-18
 flow, supported 3-7
 model 3-10
 report for logs 3-16
 report logs 3-17
 run status 3-15
 running 3-12
 script 3-19
 script file generation 3-25
 script, build options 3-20
 script, debug options 3-21
 script, run options 3-20
 setup 3-9
 simulation options 3-19
 simulator support 3-8
 suite 3-2
 suite configuration 3-5
 test suites 3-4
 test suites, running 3-12
 tests 3-2
 tools 3-2, 3-3
 tools, val_report 3-3
 val_report script usage 3-22

- val_report
 - example 3-15
 - filter options 3-22
 - output options 3-24
 - script usage 3-22
 - submission options 3-23
- Verification
 - Equivalence checking 8-3
 - macrocell design 8-13
 - netlist, preparation 8-11
 - RTL model 8-12
 - test vectors 8-15
 - testgen script 8-12
- Verplex equivalence checking
 - results, checking 8-10
 - running 8-9
 - scripts, setup 8-8
- Voltage drop checks 6-11

W

- Wire load model in synthesis 5-23